



Références

- <https://git-scm.com/> : site officiel
- <https://marklodato.github.io/visual-git-guide/index-en.html> : Illustrations des commandes.
- <https://git-scm.com/book/en/v2> : Livre « *Pro Git* », gratuit.
- <http://git-school.github.io/visualizing-git>

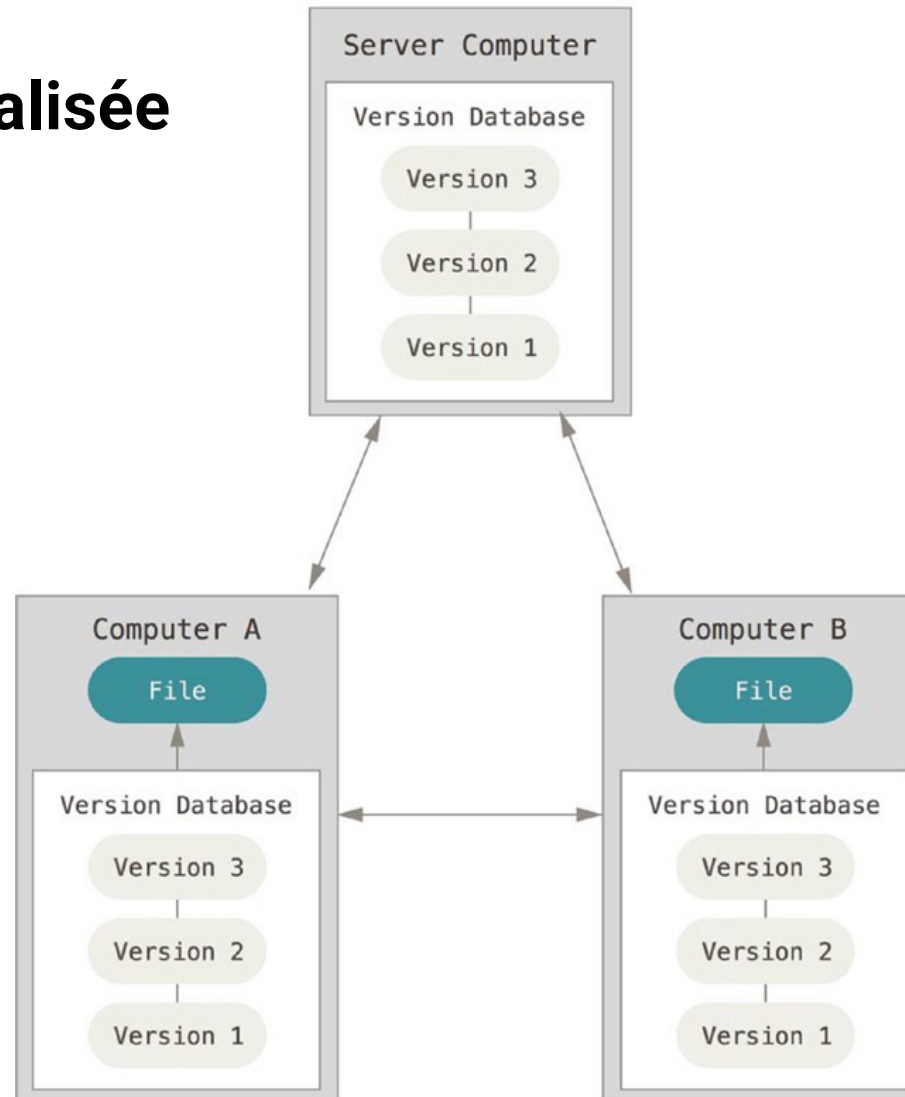
Définition

*"Un logiciel de gestion de versions (ou VCS en anglais, pour Version Control System) est un logiciel qui permet de **stocker** un ensemble de fichiers en conservant la chronologie de toutes les modifications qui ont été effectuées dessus."-Wikipédia*

Avantages

- **Gérer plusieurs versions d'un projet**
 - Sauvegarder l'évolution du code (plusieurs versions)
 - Pouvoir revenir en arrière (version antérieure)
 - Comparer des versions
- **Partage de code entre développeurs**
 - Travailler à plusieurs sur un projet
 - Pouvoir réaliser des modifications sans impacter le travail des autres
- **Coordination du travail en équipe**
- **Déploiement d'applications (Dépôt distant)**

Architecture décentralisée ou distribuée





Git est un logiciel de gestion de versions décentralisée (DVCS-Distributed Version Control System).

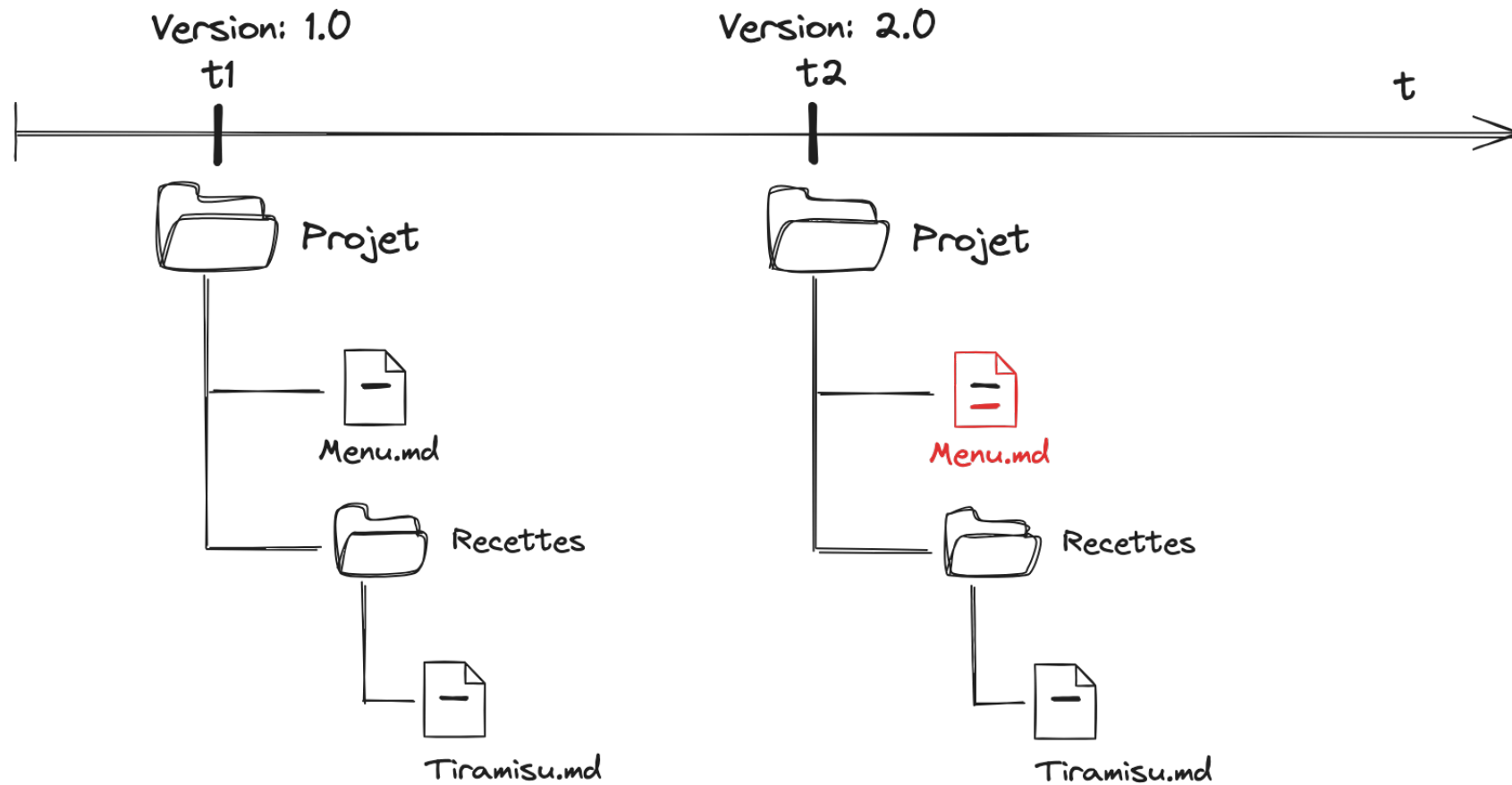
Fonctionnement interne

git est un...

Distributed Revision Control System



Paolo Perrotta

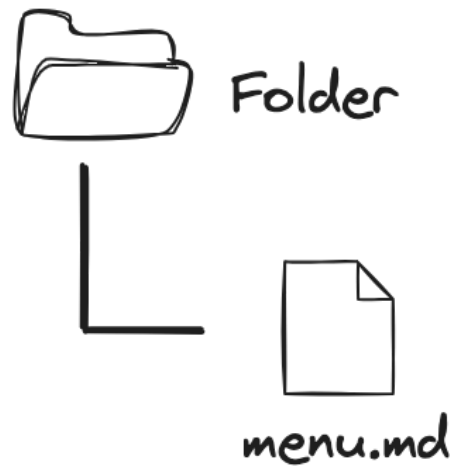




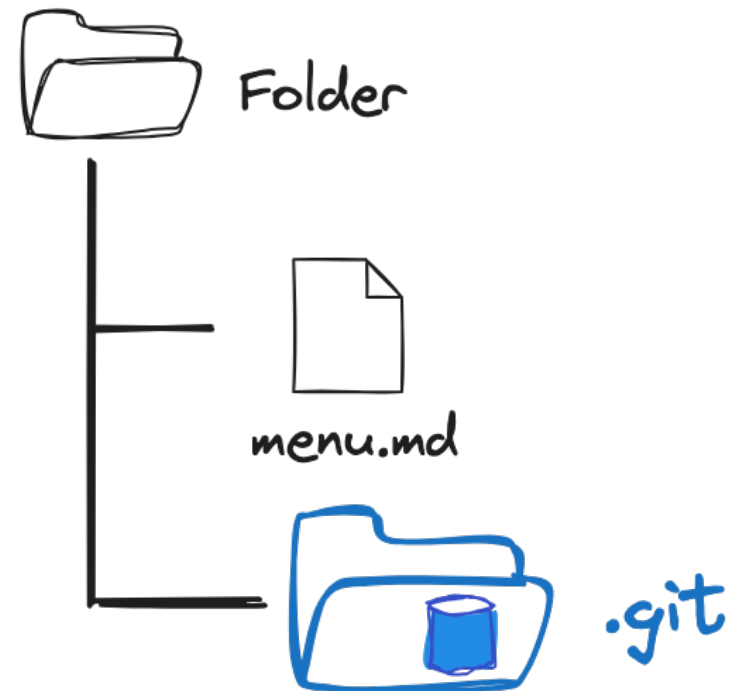
« **Markdown** est un [langage de balisage léger](#) créé en [2004](#) par [John Gruber](#), avec l'aide d'[Aaron Swartz](#)^{1,2}, dans le but d'offrir une [syntaxe](#) facile à lire et à écrire en l'état dans sa forme non formatée. Markdown est principalement utilisé dans des [blogs](#), des sites de [messagerie instantanée](#), des [forums](#) et des pages de documentation de logiciels. » Wikipédia

Un document rédigé par Markdown peut être converti en HTML, en PDF ou en d'autres formats.

Répertoire "classique"



Dépôt local git

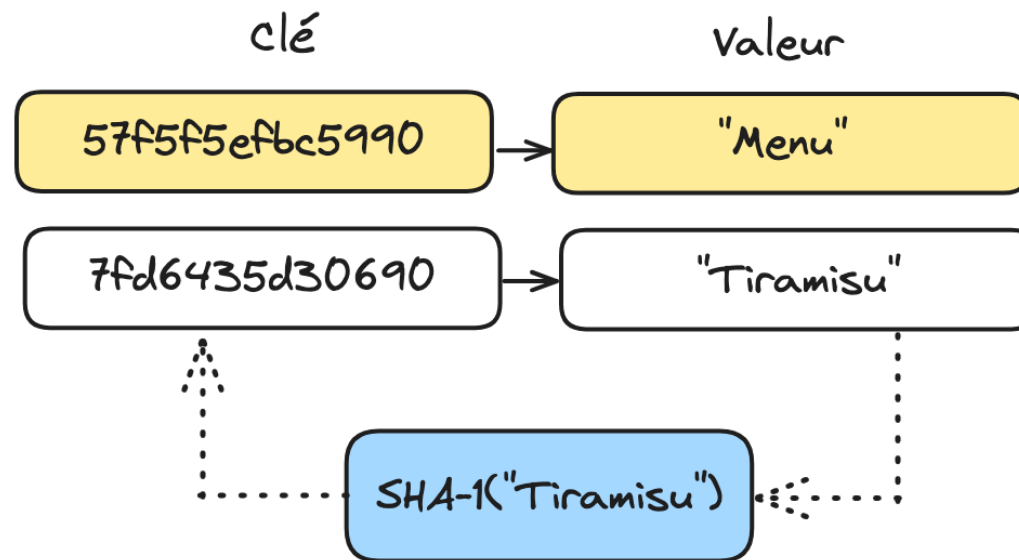


git est une...

Persistent Map



Système de stockage => Base de données **clé-valeur** -> Map
-> dictionnaire (Python)

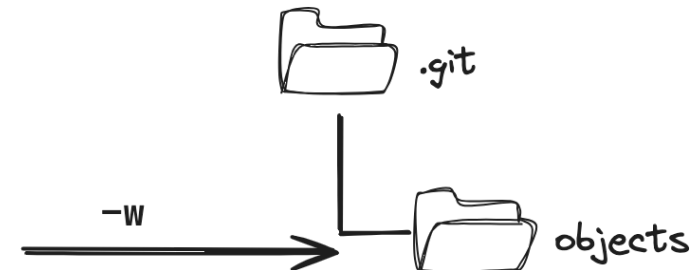


SHA-1(contenu) => 2160 possibilités => **garantie de l'unicité**

*SHA-1 (Secure Hash Algorithm) est une **fonction de hachage Cryptographique**.*

Commandes de plomberie = commandes de bas niveau

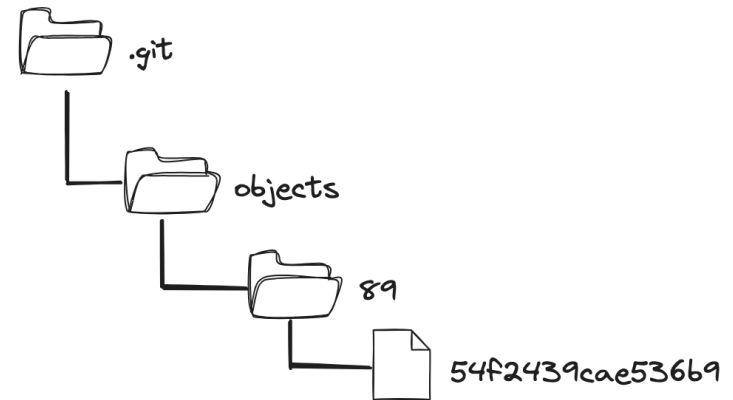
```
$ echo 'Menu' | git hash-object -w --stdin
```



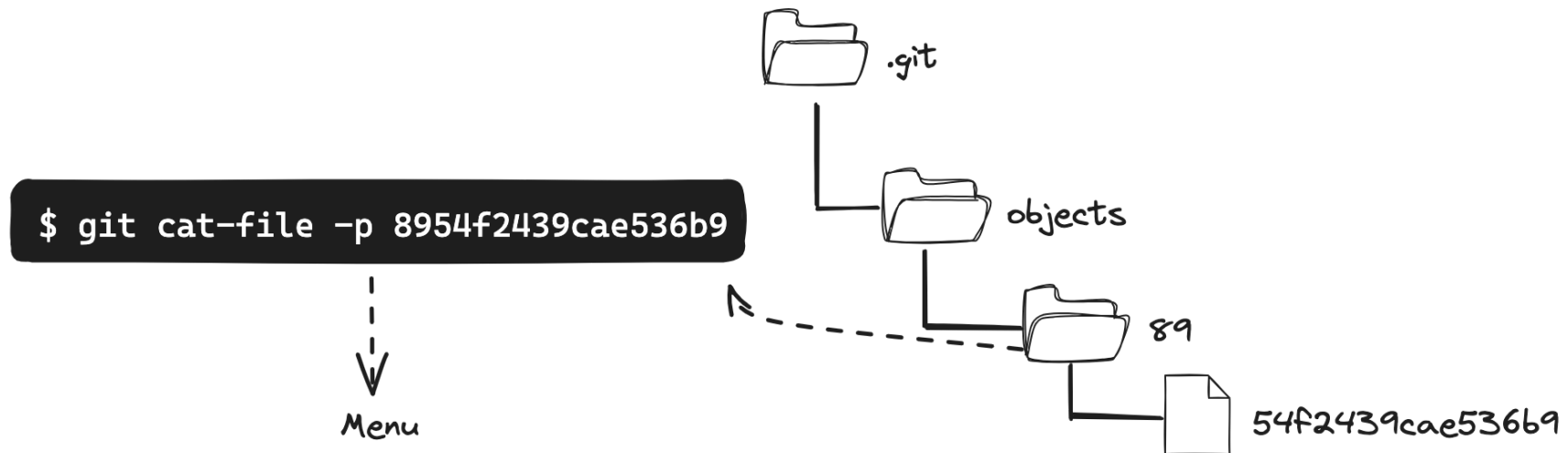
git hash-object :

prend des données, les stocke dans votre répertoire **.git**, puis retourne la clé sous laquelle les données sont stockées.

8954f2439cae536b9b35ac39e8d173b90f979ca4



git cat-file :
récupère le contenu à partir de la clé.



git est...

Stupid Content Tracker



```
$ man git
```

NAME

git - the stupid content tracker

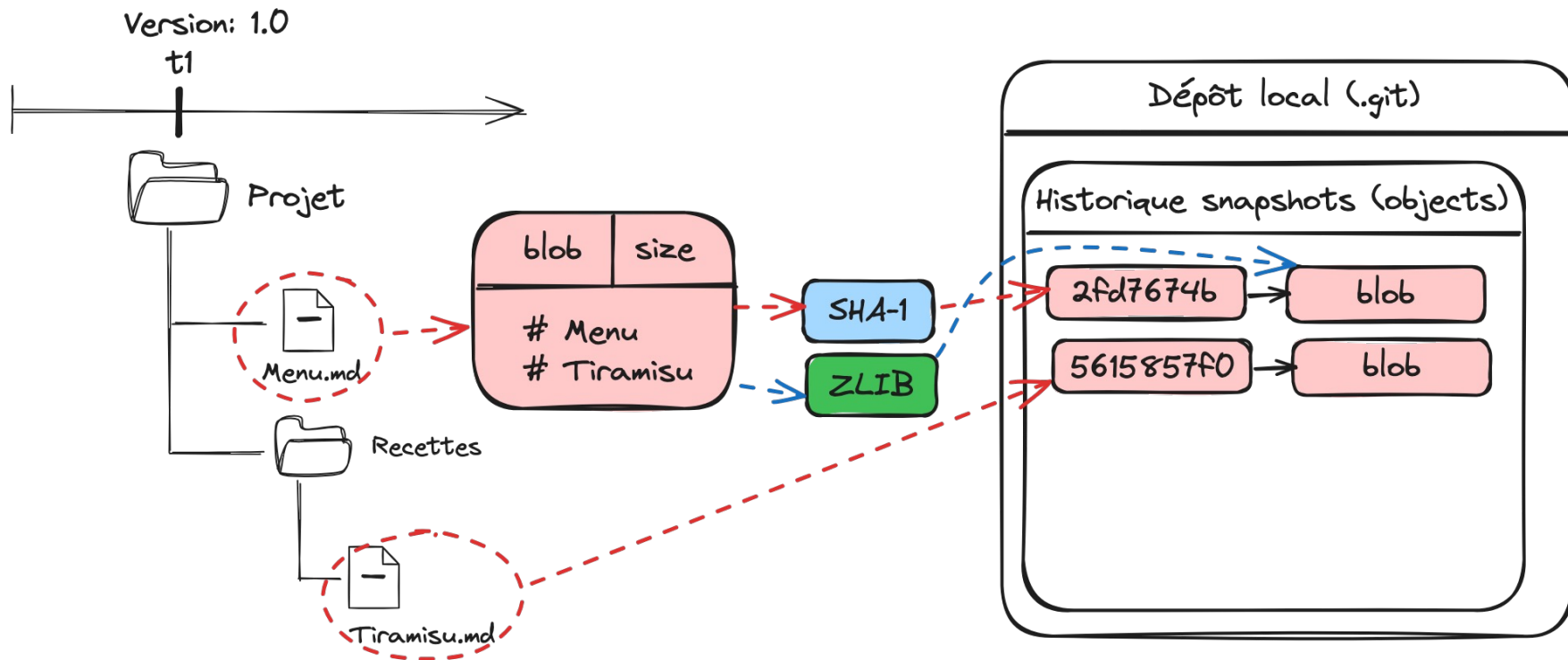
SYNOPSIS

```
git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]  
      [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]  
      [-p|--paginate|-P|--no-pager] [--no-replace-objects] [--bare]  
      [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]  
      [--super-prefix=<path>] [--config-env=<name>=<envvar>]  
      <command> [<args>]
```

DESCRIPTION

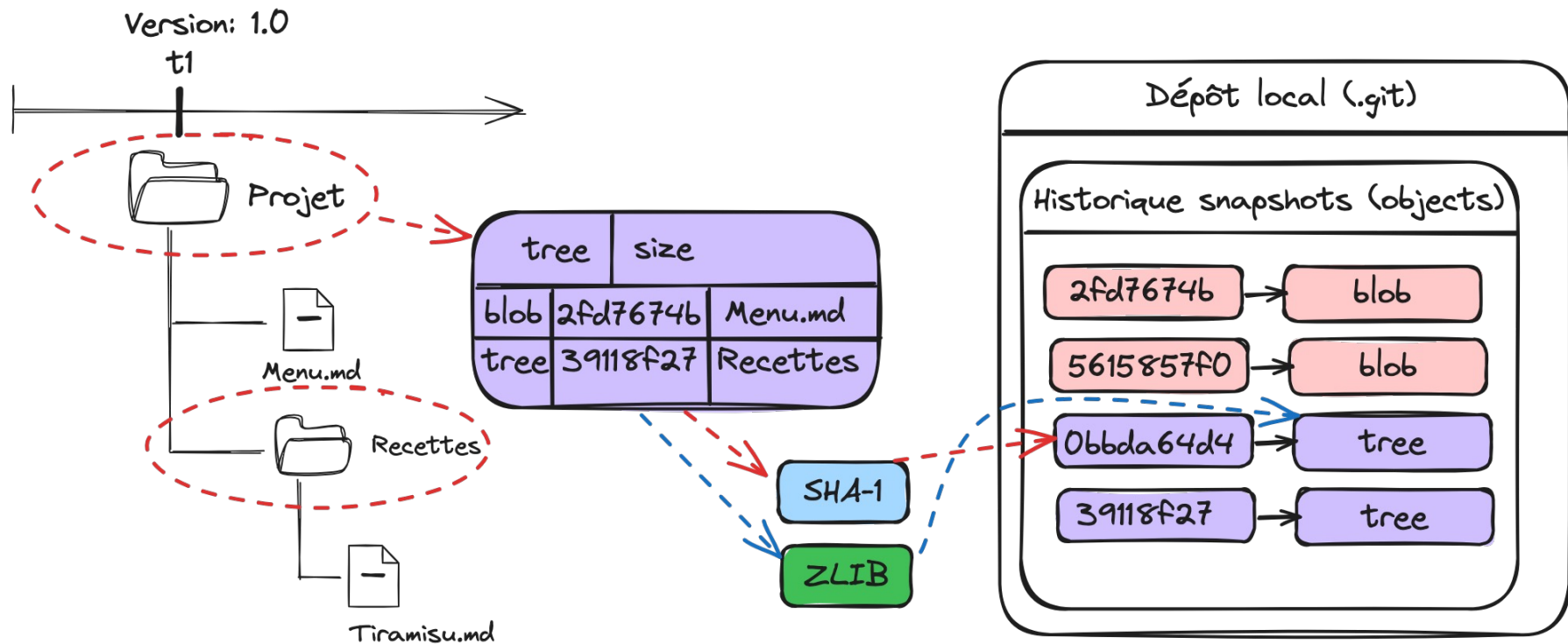
Git is a fast, scalable, distributed revision control system with an unusually rich com

Blob : binary large object (gros objet binaire) = **stocke le contenu** d'un fichier

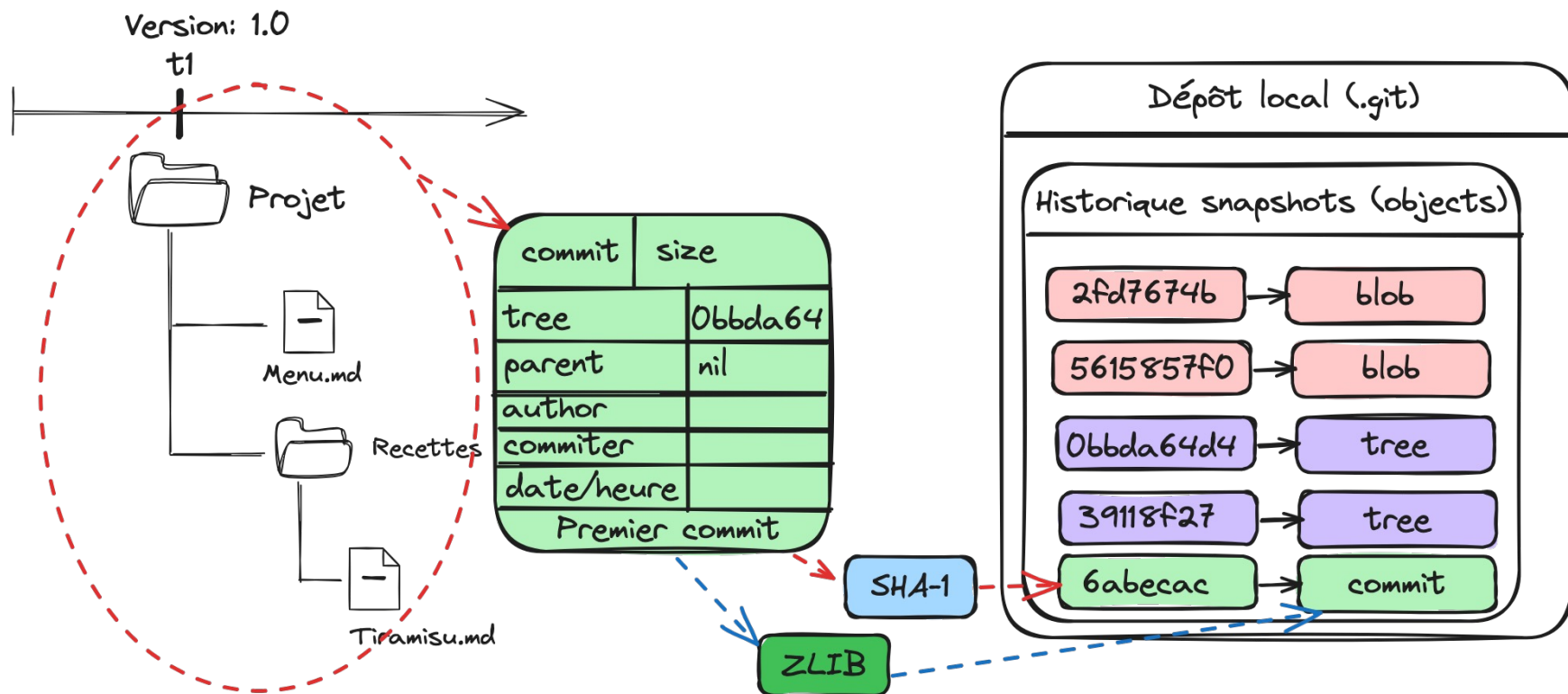


Zlib : bibliothèque logicielle de compression de données.

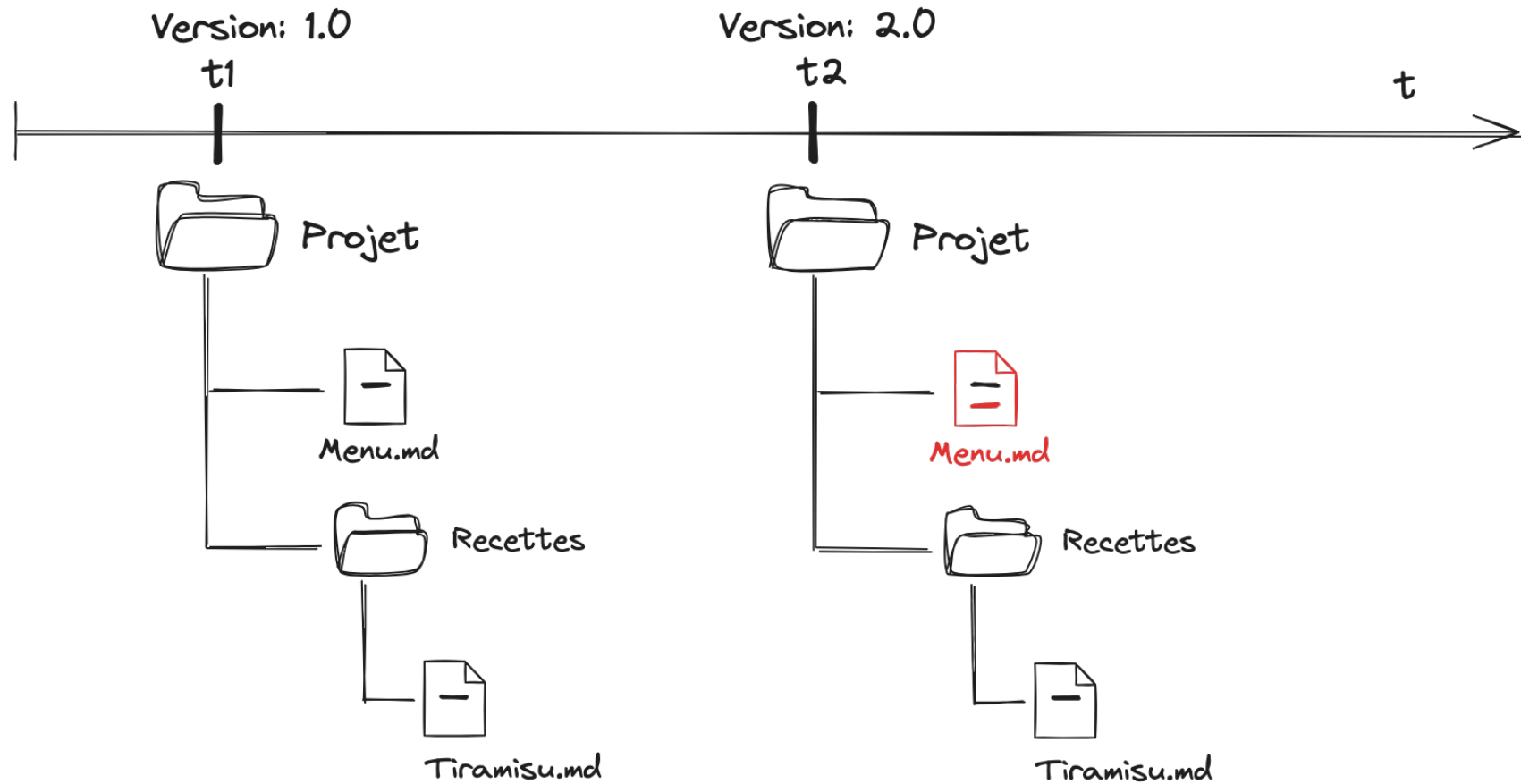
Tree = Stocke l'arborescence du projet

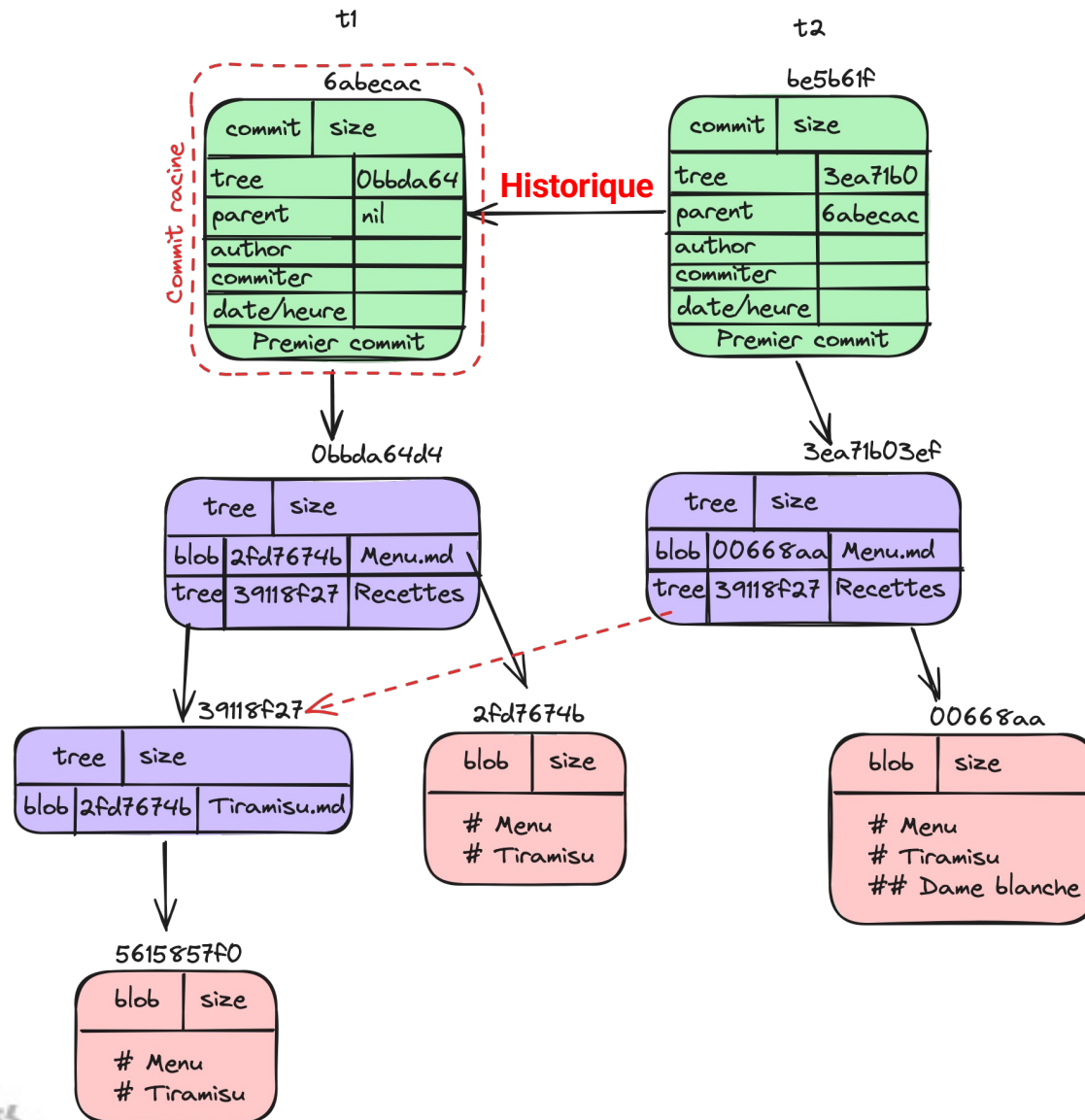


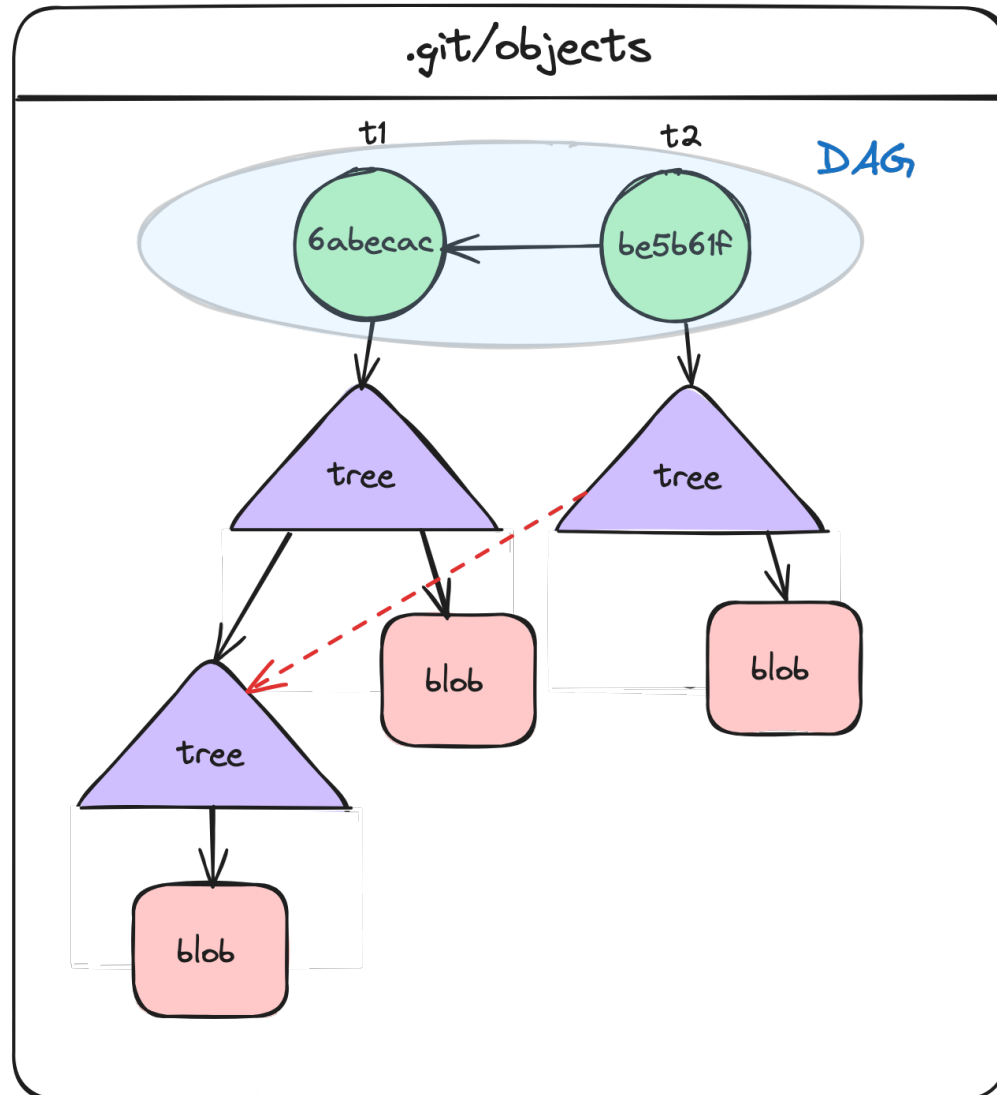
Commit = Stocke la référence d'un tree (arborescence) + nom de l'auteur + horodatage + message



Comment stocker les différents instantanés ?

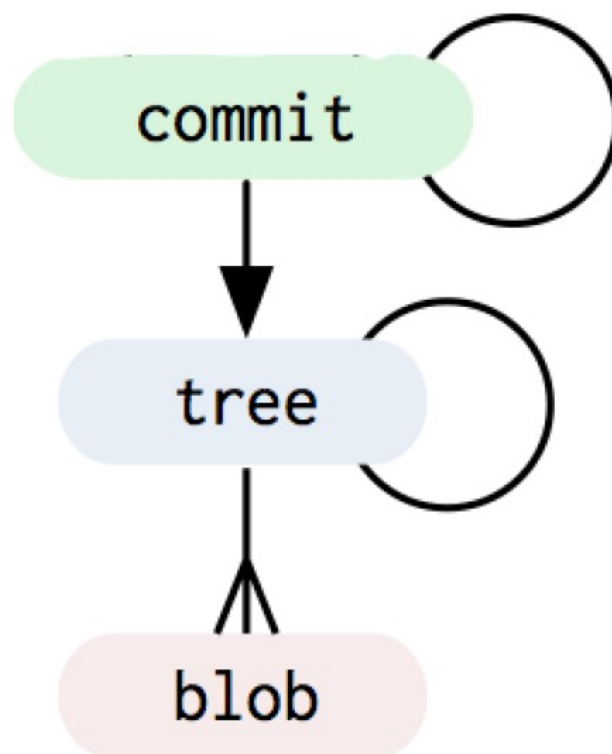


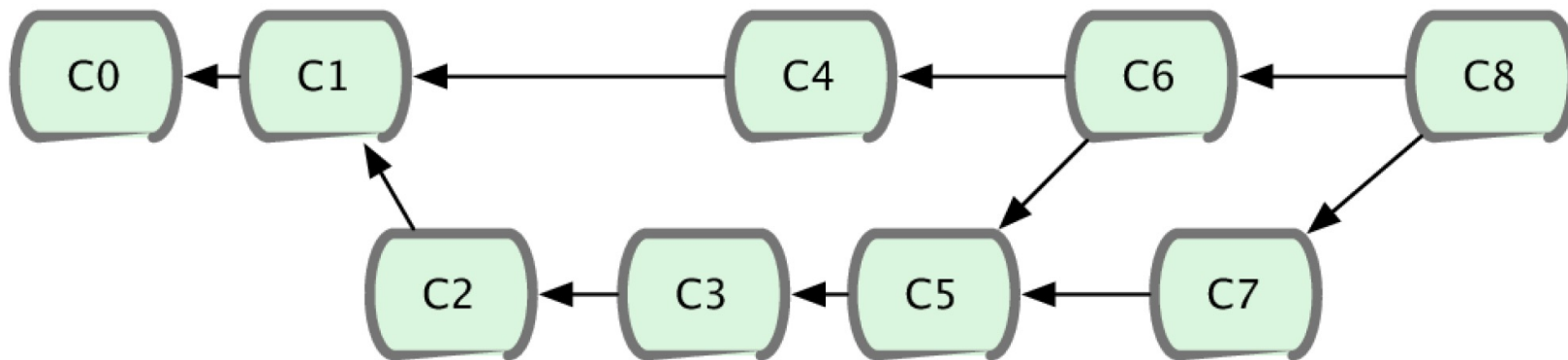




Structure de données => DAG :

Directed Acyclic Graph (Graphe orienté acyclique) => graphe sans boucle fermée





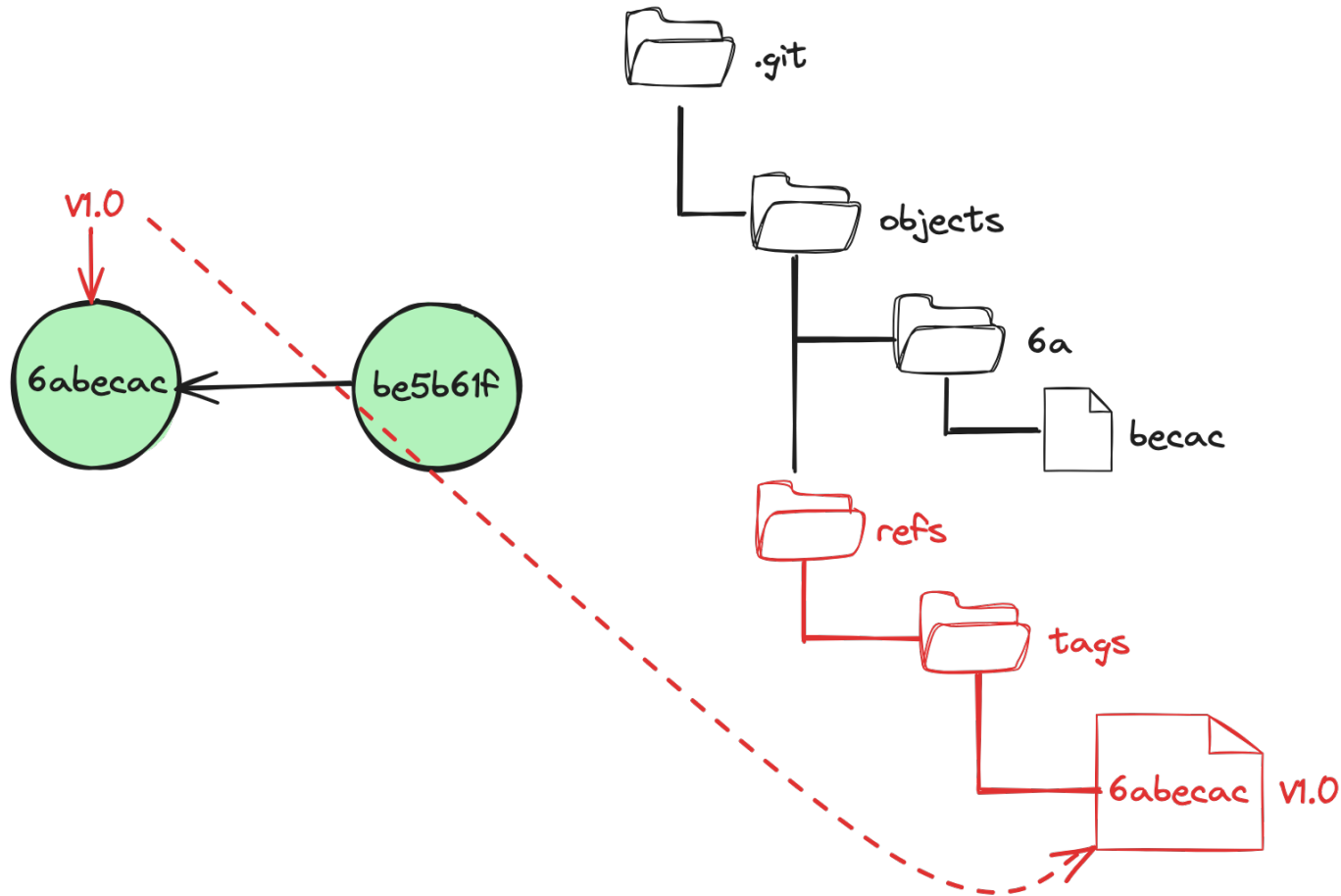
Graph (DAG) = l'ensemble des sauvegardes (commits)

git est un...

Revision Control System

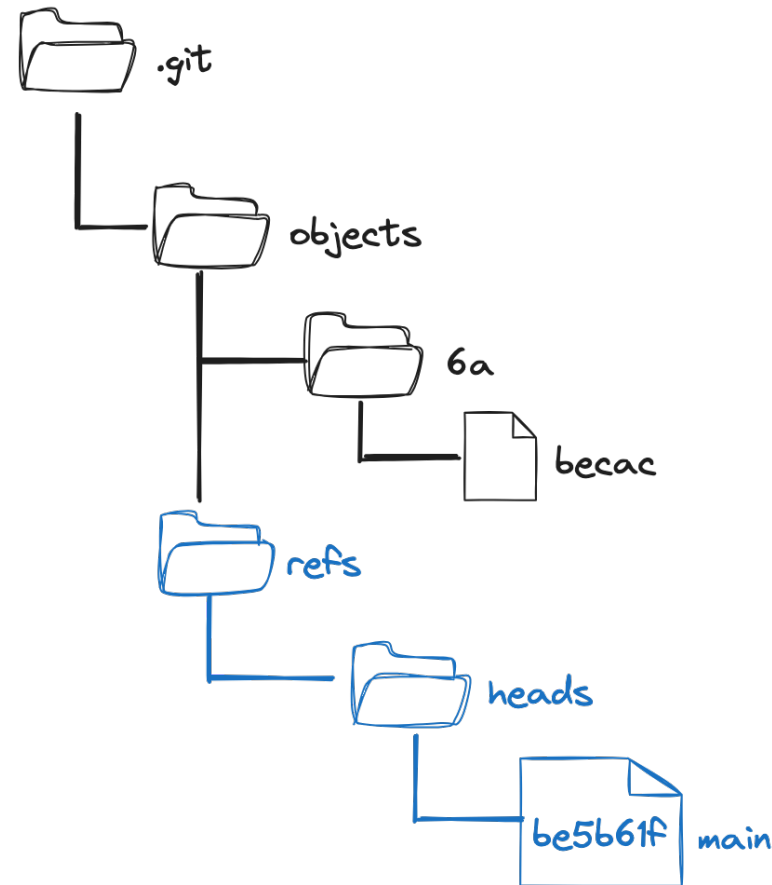
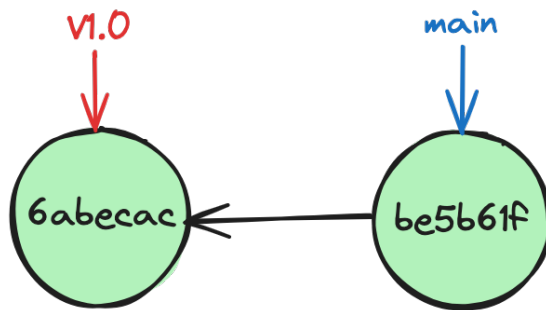


Tag



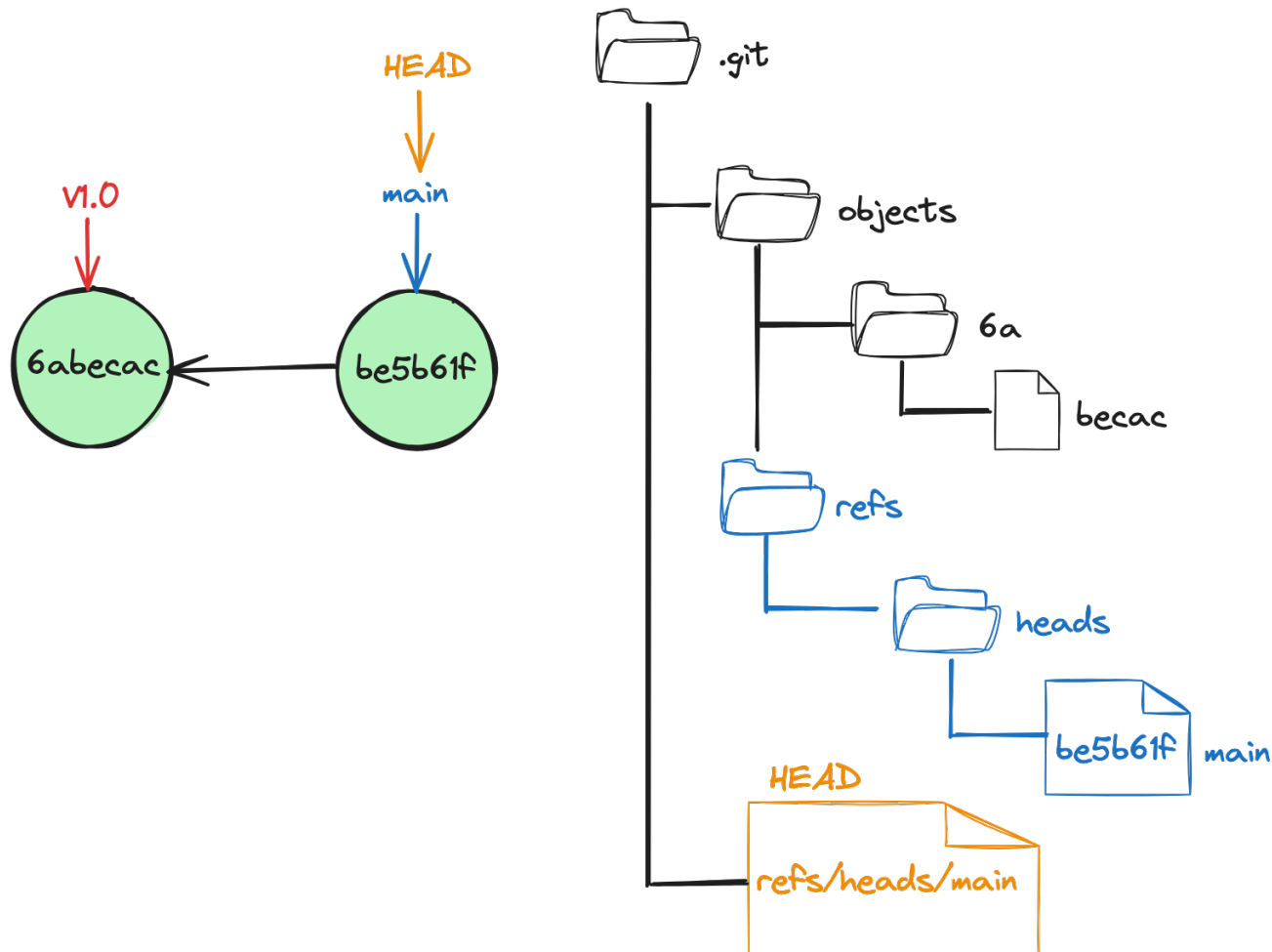
- Permet de **marquer un commit** particulier (p.e. : numéro de version)
- **Pointeur fixe**
- Tag = **fichier texte** contenant le **SHA-1 d'un commit** spécifique.

Branche



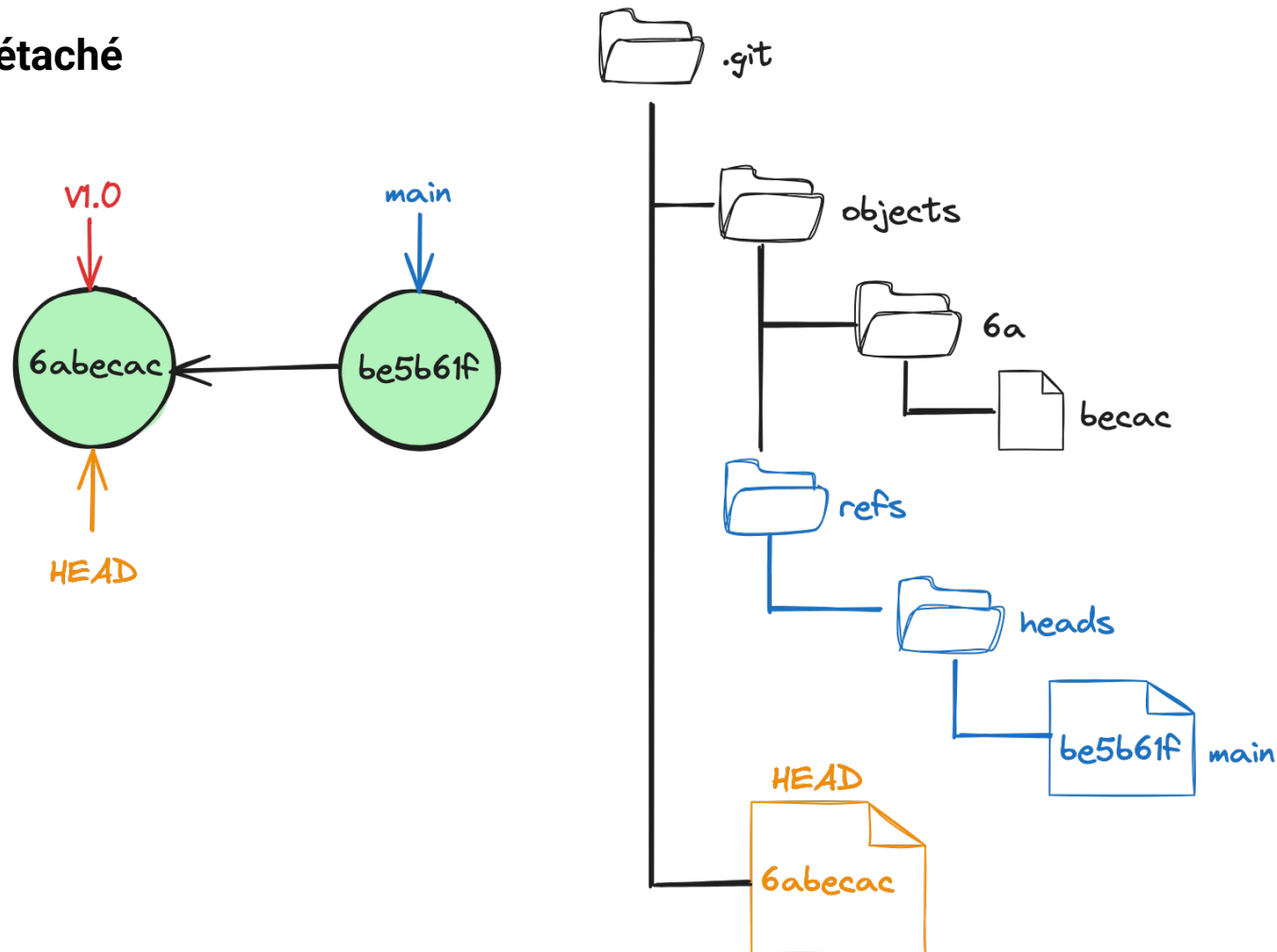
- Branche = **référence** qui pointe sur le SHA-1 du **dernier commit**.
- Une branche = un **fichier texte** de 20 octets contenant le **SHA-1** du commit
- Branche **main** (master) = Branche principale (**par défaut**).

HEAD

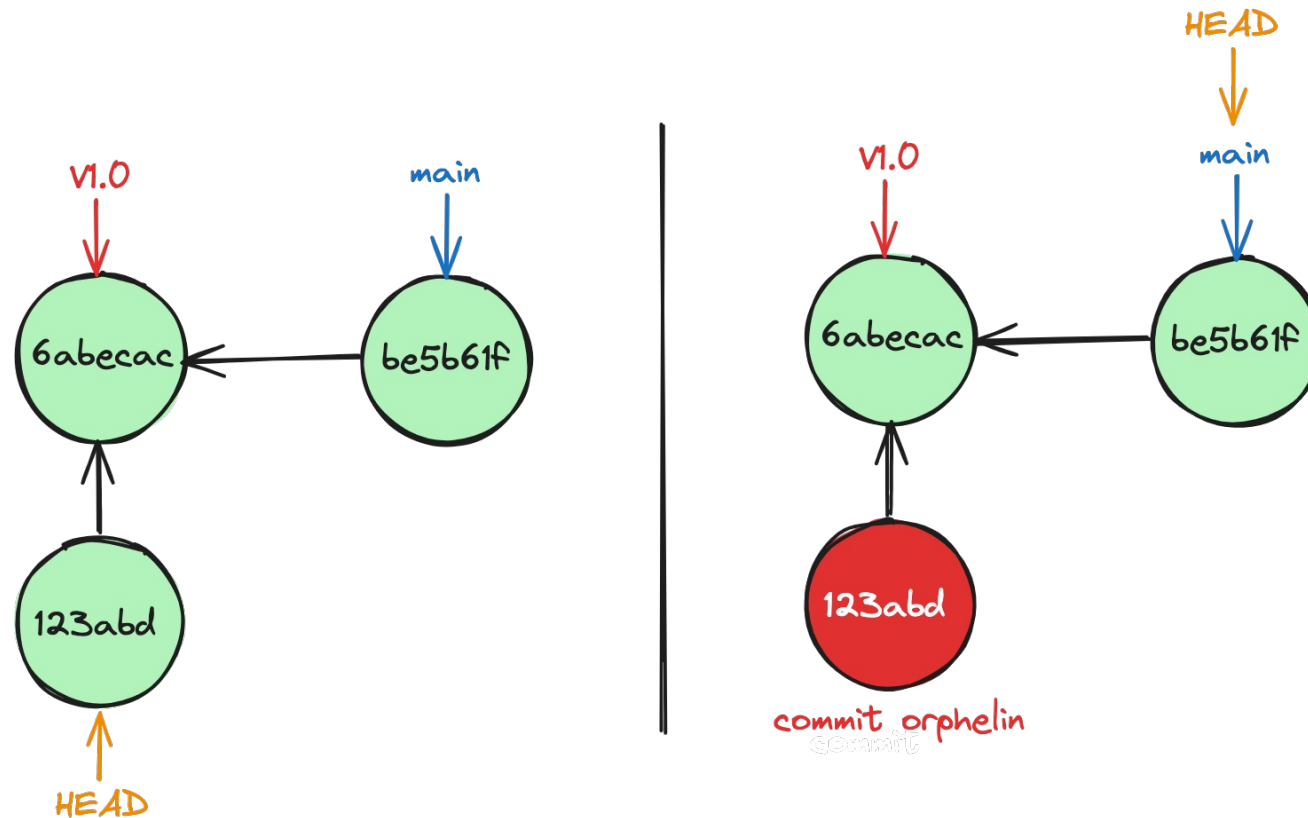


- Référence HEAD = **fichier texte** contenant la **référence d'une branche**.
- Permet de **naviguer** dans le graphe **en s'attachant à une branche**

HEAD détaché

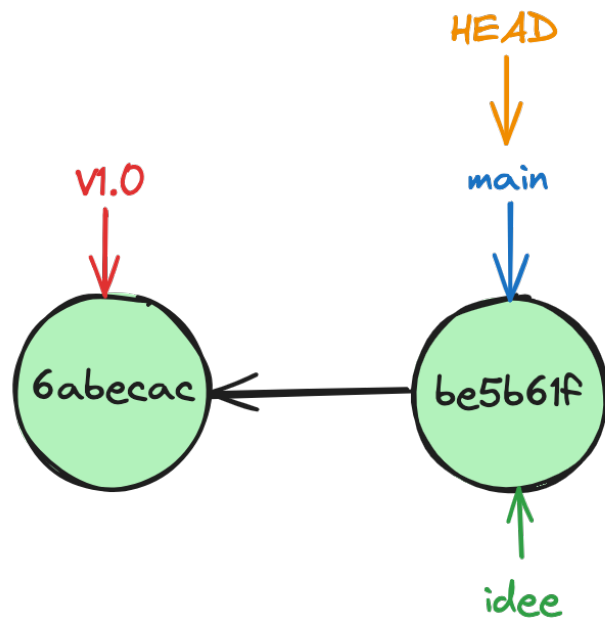


HEAD détaché = HEAD qui pointe directement sur un commit

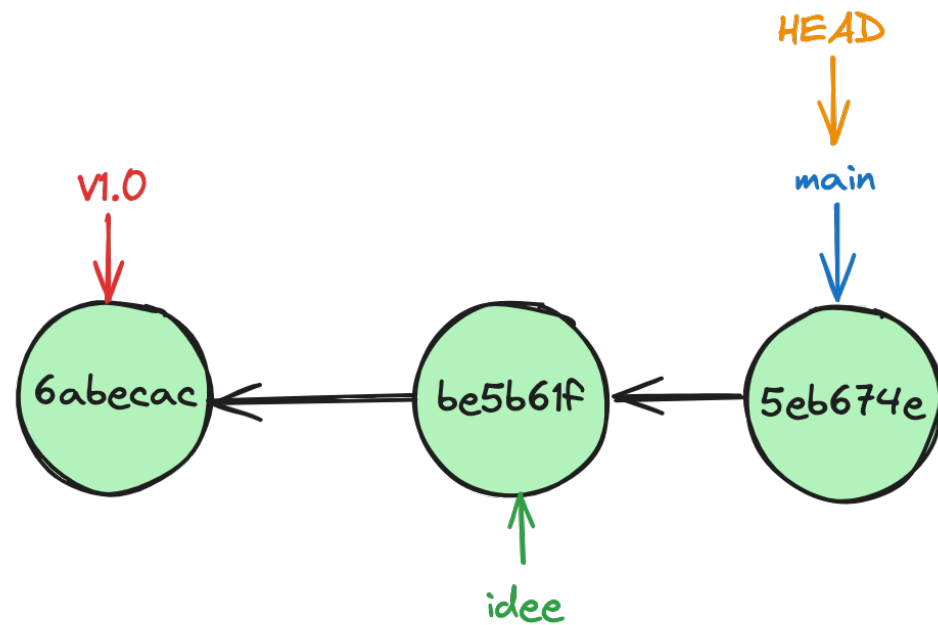


Commit orphelin= Commit qui n'est plus référencé (branche ou commit)
Au-delà de 90 jours, les commits orphelins sont supprimés par le
garbage collector.

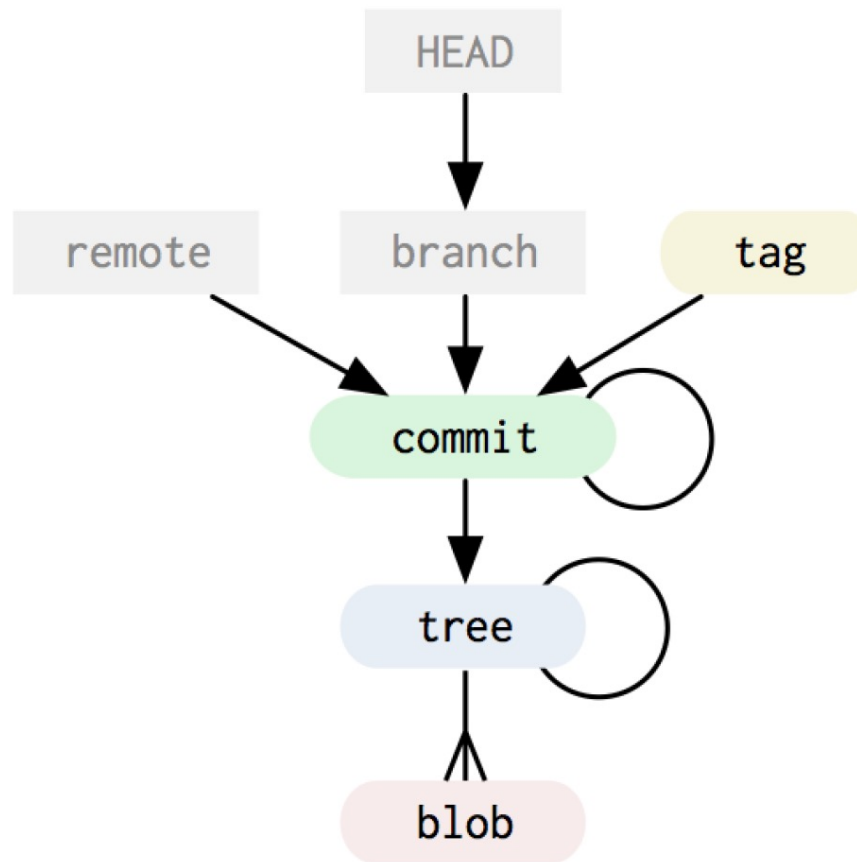
Nouveau commit



git branch idee



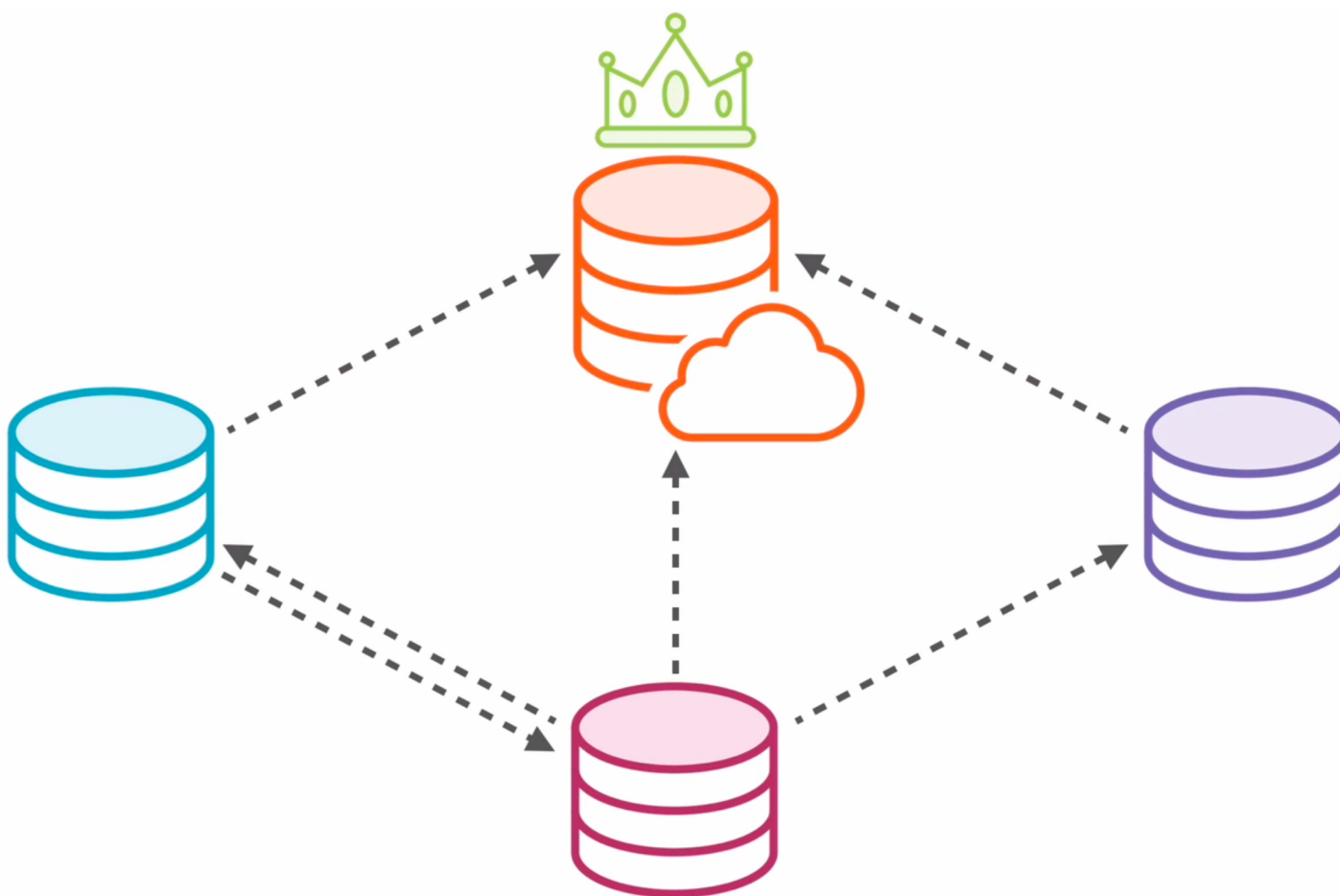
Résumé objets & références

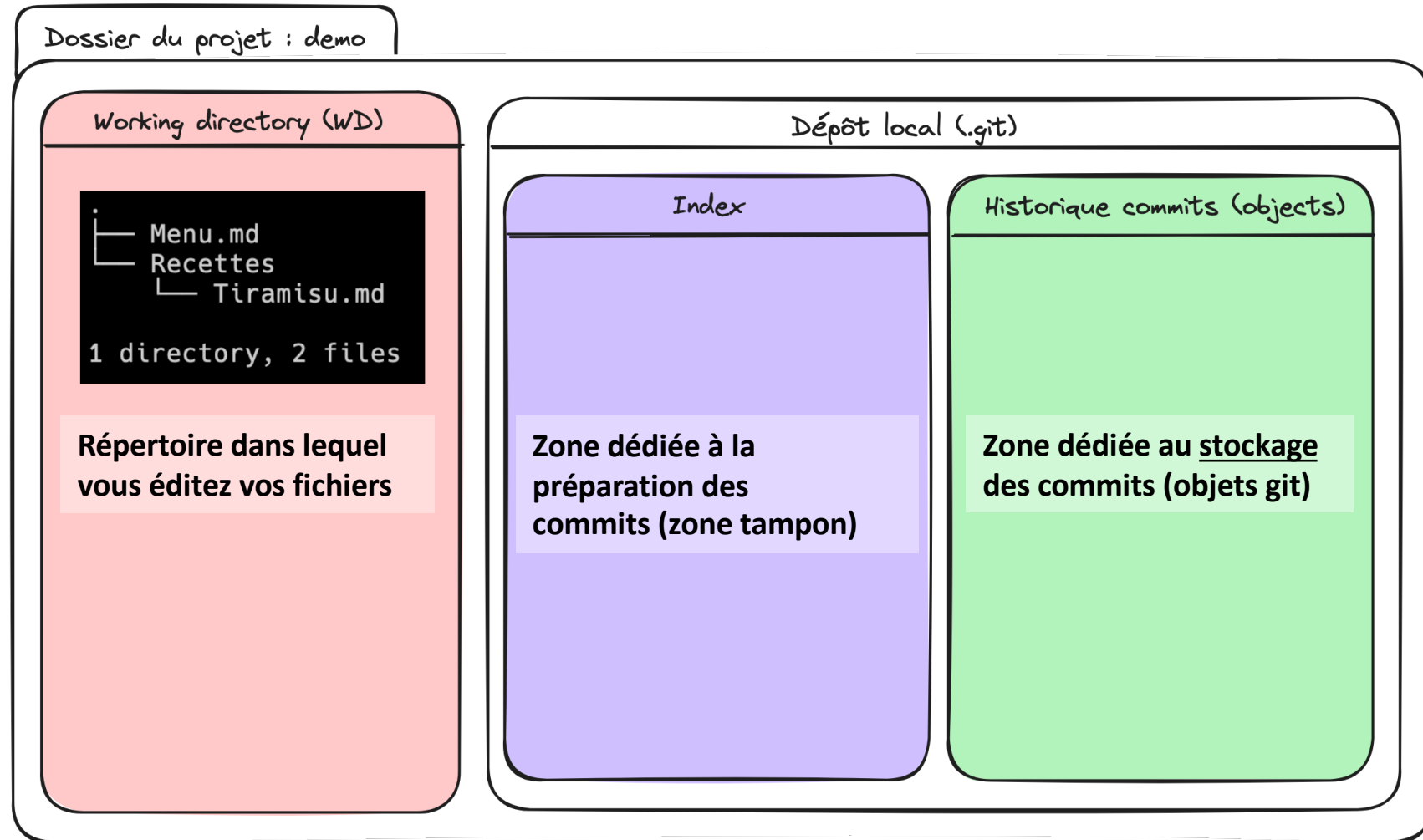


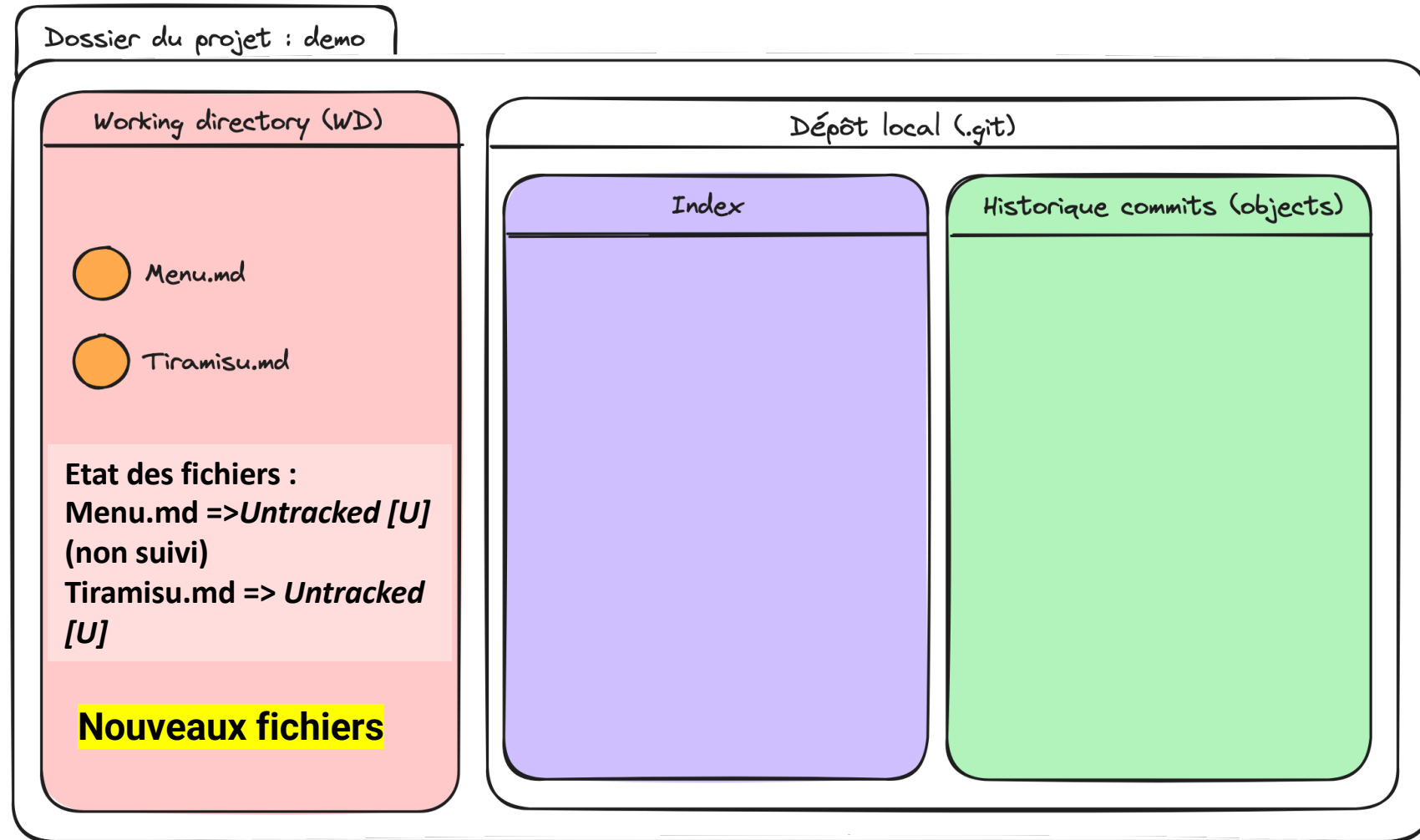
git est un...

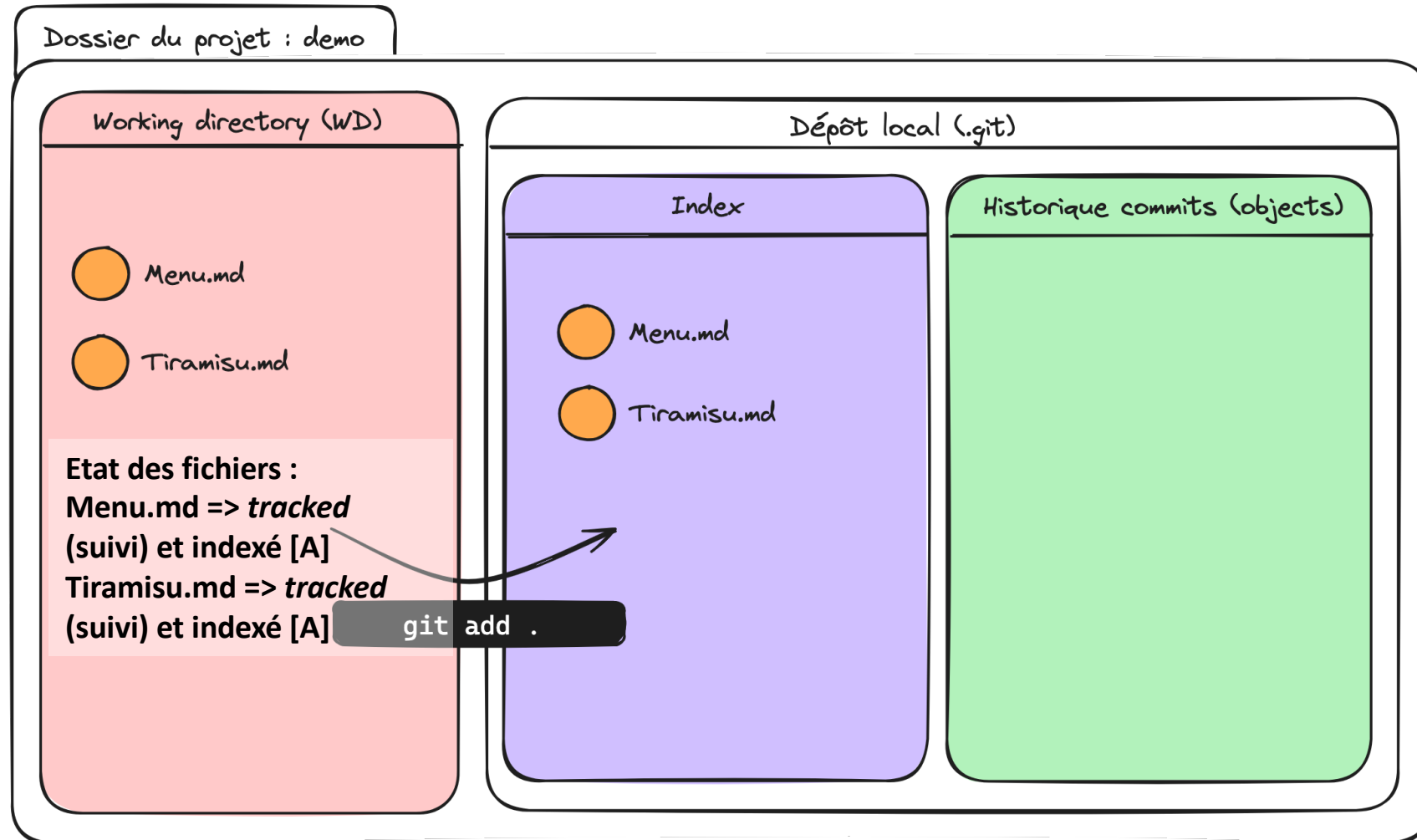
Distributed Revision Control System

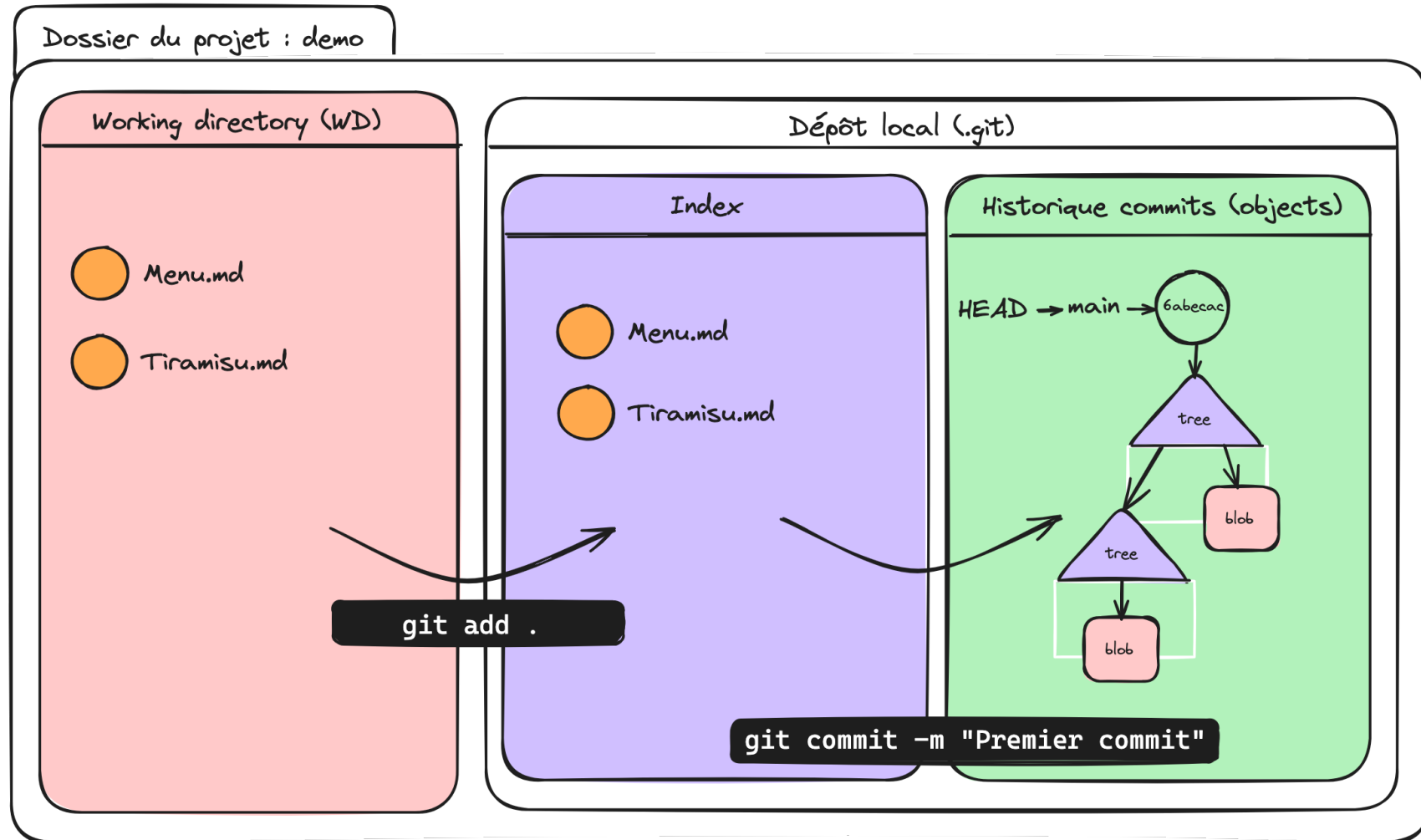


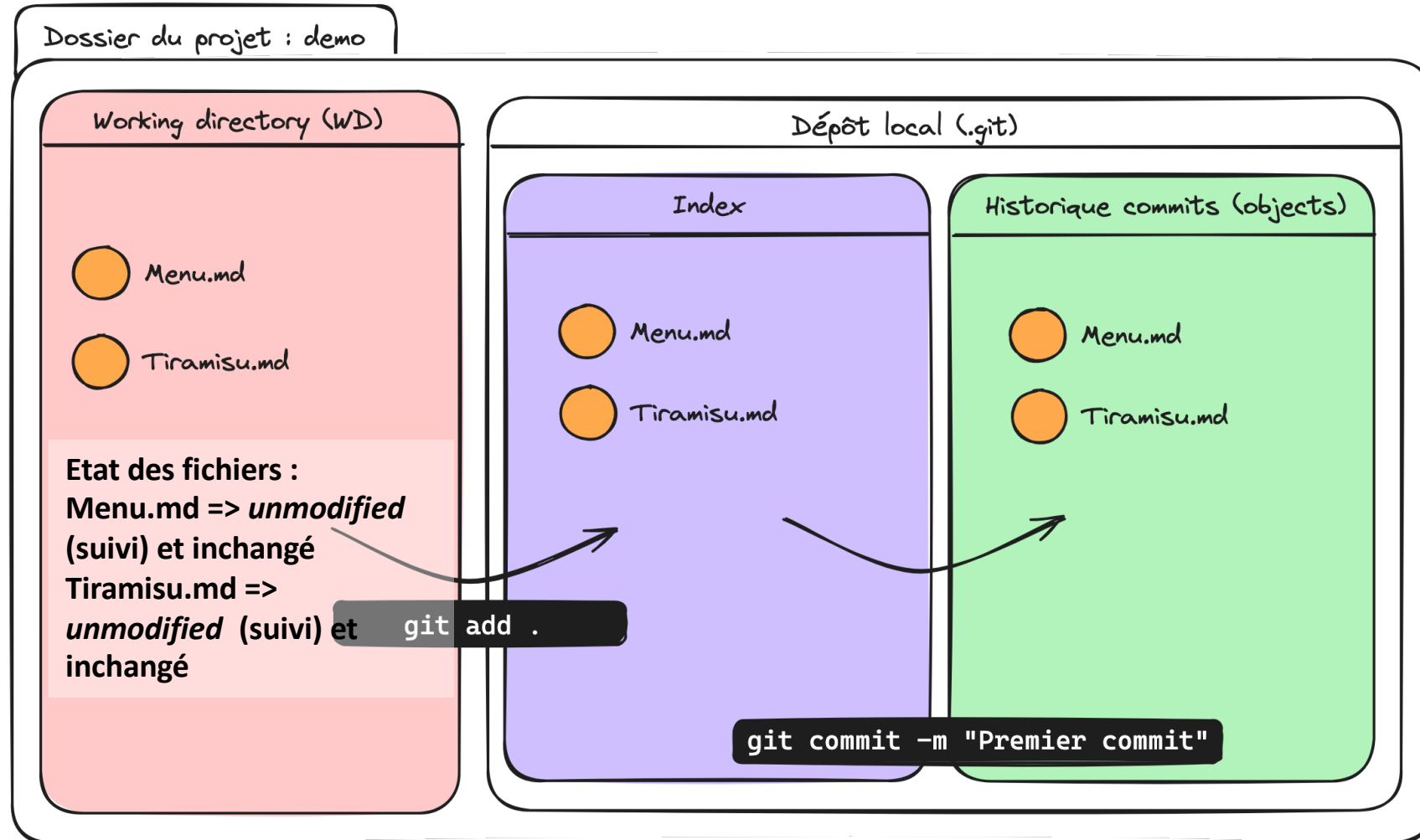


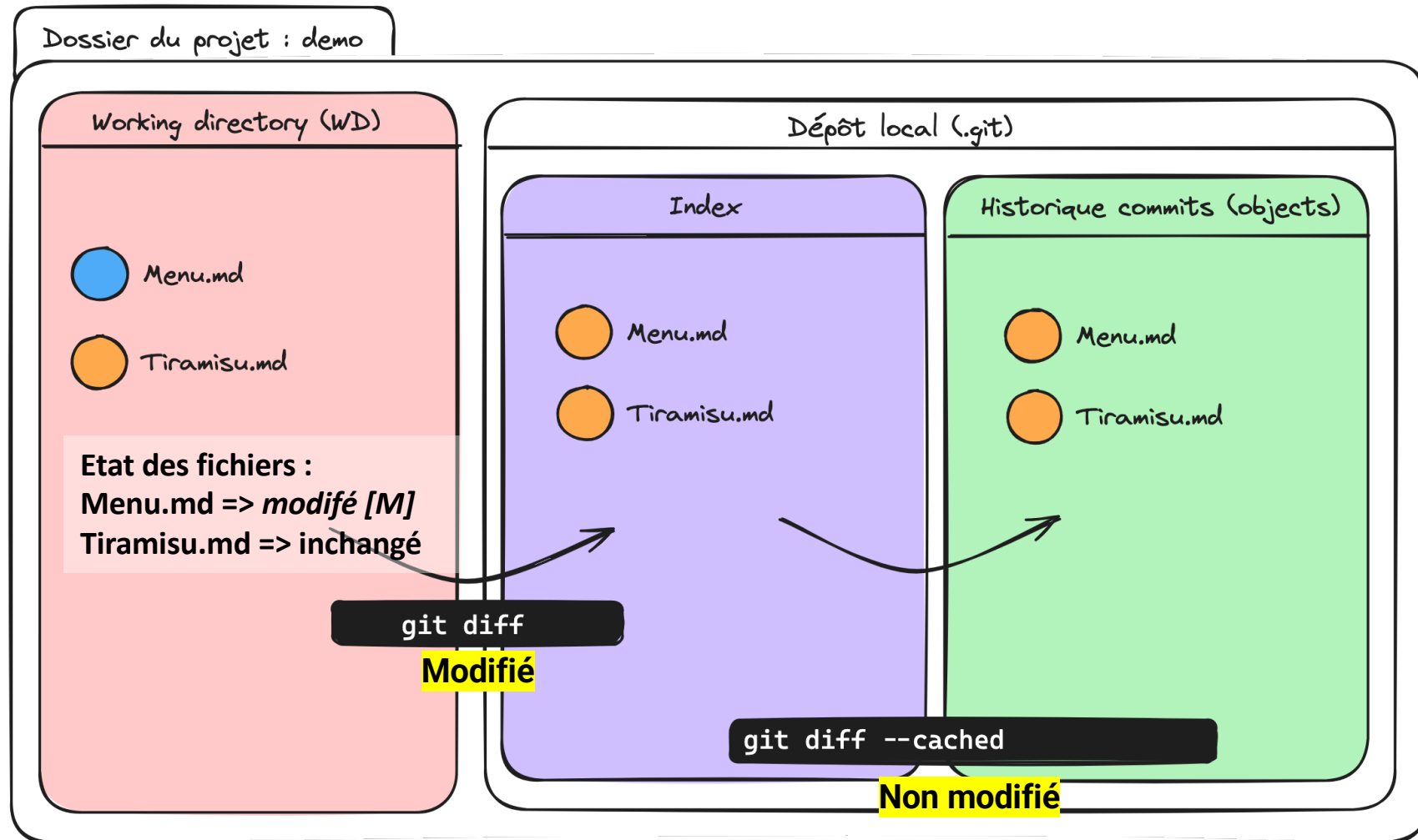


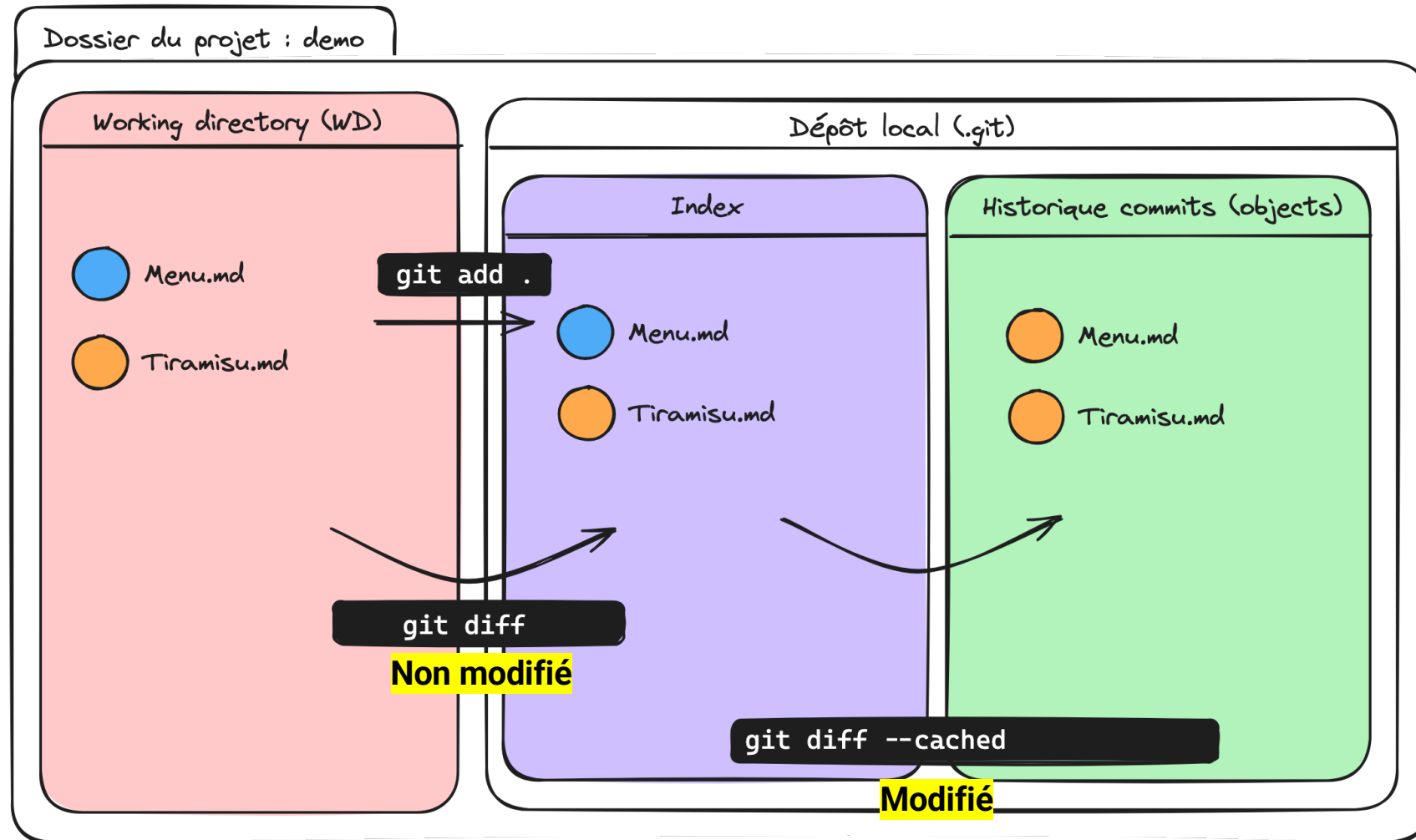


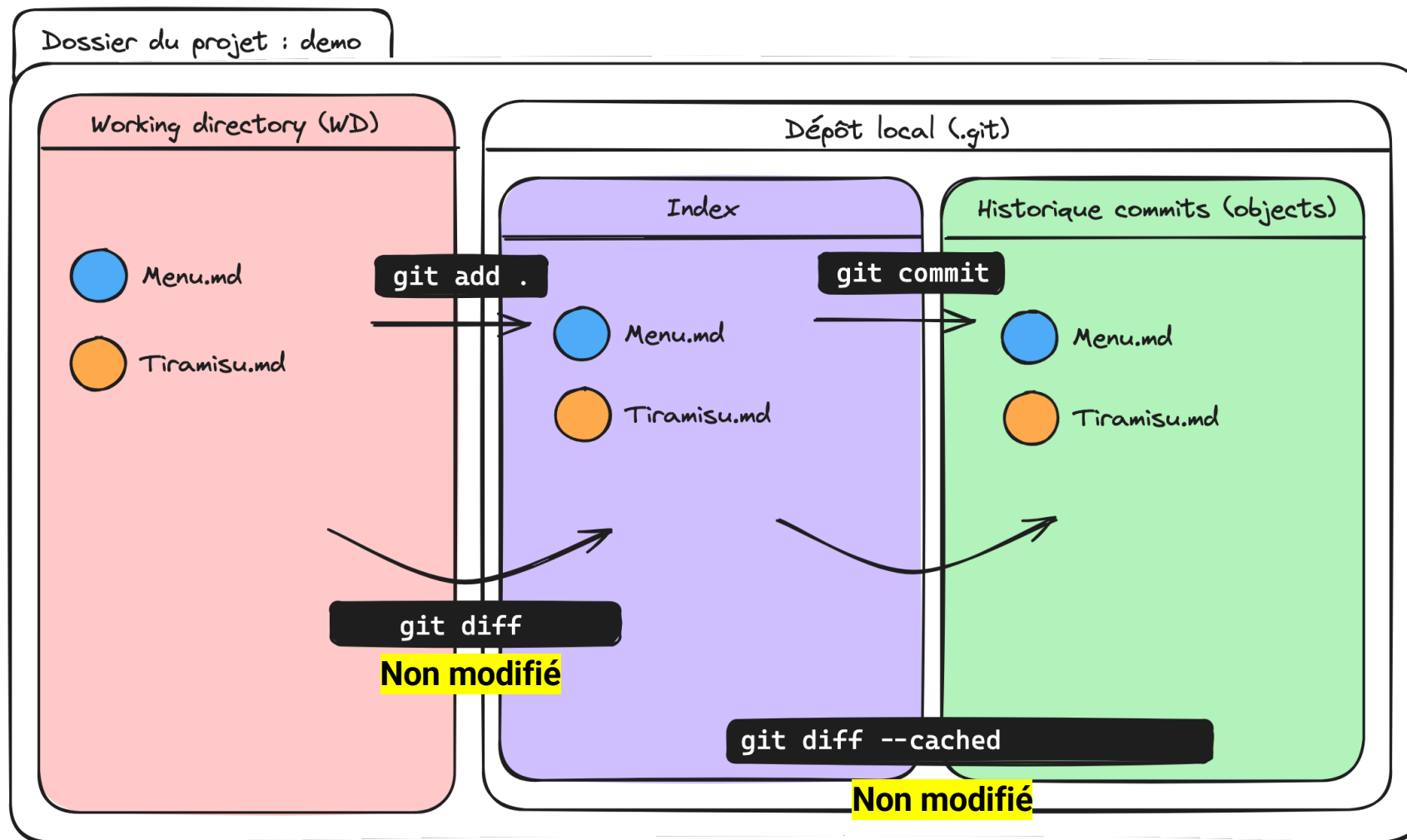


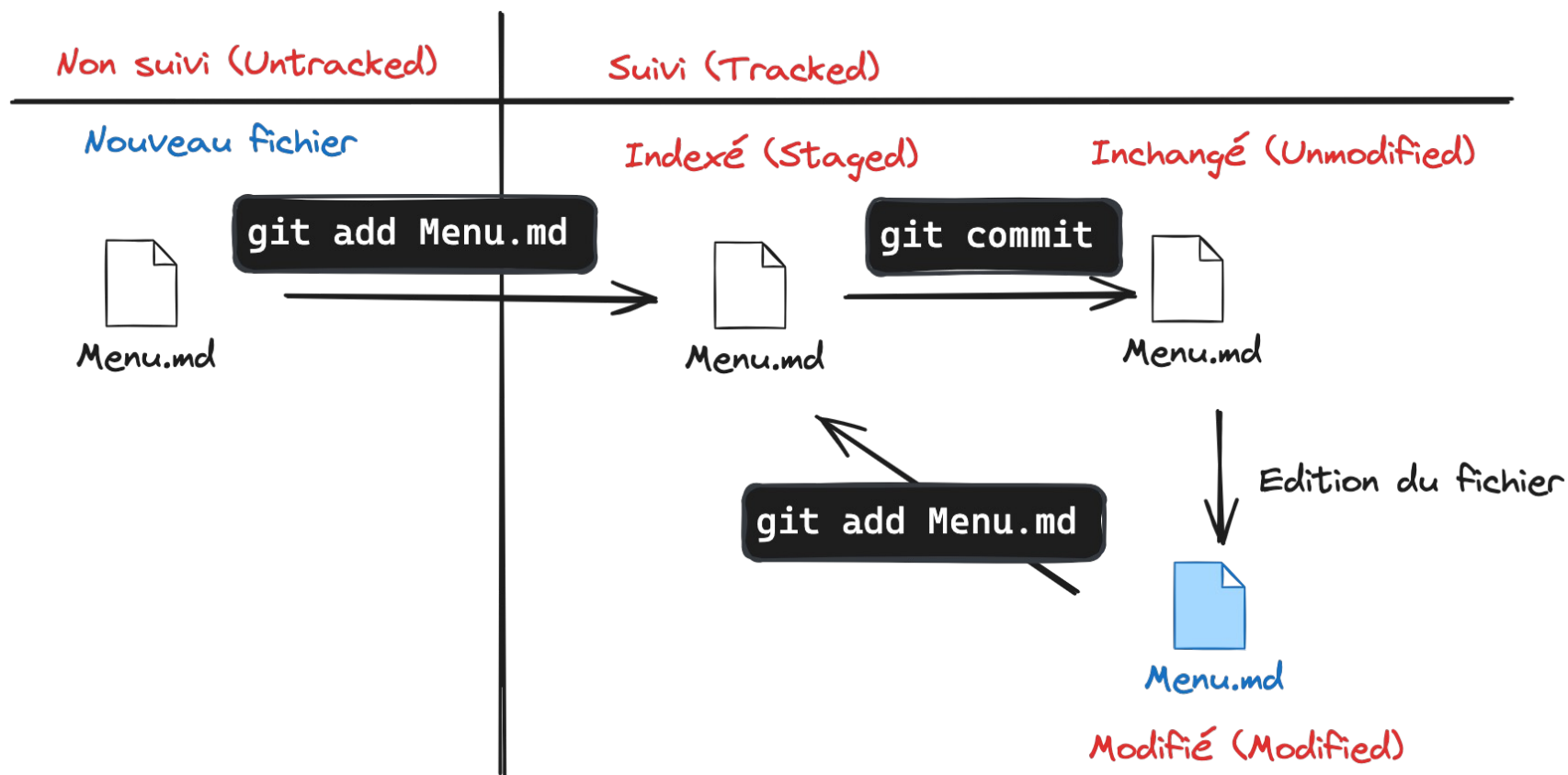




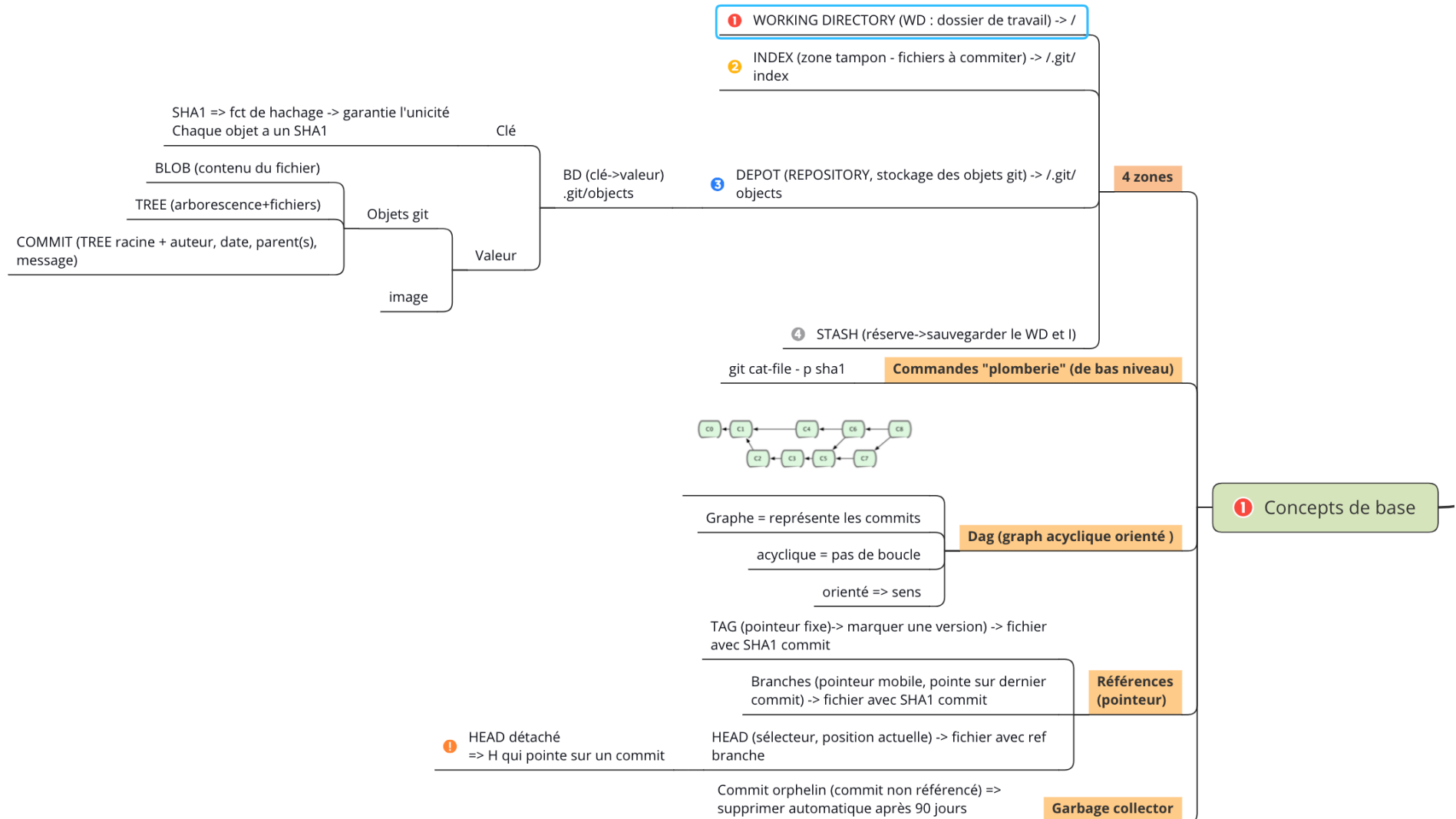








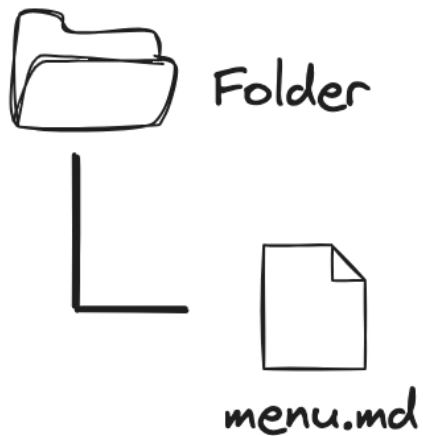
Fonctionnement interne— Synthèse



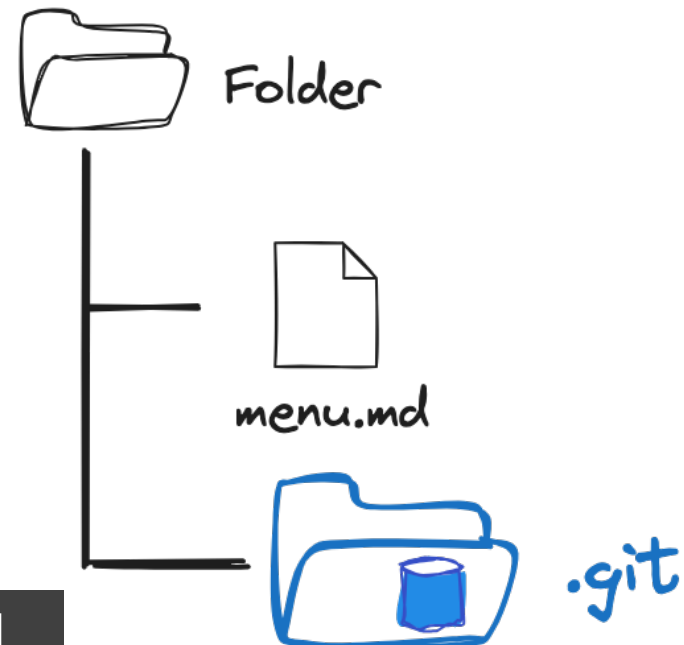
Commandes

Crée un dépôt local vide dans le dossier courant ou en créer un avec le nom spécifié.

Répertoire "classique"



Dépôt local git



```
git init [nom_dépôt]
```

```
{21:52:25}~/tests/monProjet (master #) $ tree -a
.
├── .git
│   ├── HEAD
│   ├── config
│   ├── description
│   ├── hooks
│   │   ├── applypatch-msg.sample
│   │   ├── commit-msg.sample
│   │   ├── post-update.sample
│   │   ├── pre-applypatch.sample
│   │   ├── pre-commit.sample
│   │   ├── pre-push.sample
│   │   ├── pre-rebase.sample
│   │   ├── prepare-commit-msg.sample
│   │   └── update.sample
│   ├── info
│   │   └── exclude
│   ├── objects
│   │   ├── info
│   │   └── pack
│   └── refs
│       ├── heads
│       └── tags
└──
```

- **Système** : paramètres liés à tous les utilisateurs du système (fichier : `/etc/gitconfig`)

Commande : `git config --system <paramètre> <valeur>`

- **Global** : paramètres spécifiques à chaque utilisateur (`~/.gitconfig`)

Commande : `git config --global <paramètre> <valeur>`

- **Local** : paramètres spécifiques à un unique dépôt (`.git/config`)

Commande : `git config <paramètre> <valeur>`

`<paramètre>` : `section.nomParametre`

Identité

Commande :

```
$ git config --global user.email gianni.tricarico@heh.be
```

```
$ git config --global user.name «TRIGianni»
```

Fichier : (~/.gitconfig)

[user]

name = TRIGianni

email = gianni.tricarico@heh.be

section

Paramètre = valeur

Configuration minimale pour réaliser des commits

Afficher la configuration de git : `git config --list`

```
init.defaultbranch=main
alias.lg=log --color --graph --pretty=format:'%Cred%h%Creset -
%C(yellow)%d%Creset %s %Cgreen(%cr) %C(yellow)<%an>%Creset' --
abbrev-commit
alias.chk=checkout
user.name=HeH-Gianni
user.email=gianni.tricarico@heh.be
```


Fichier : (`~/.gitconfig`)

```
[alias]
lg = log --color --graph --pretty=format:'%Cred%h%Creset -
%C(yellow)%d%
Creset %s %Cgreen(%cr) %C(yellow)<%an>%Creset' --abbrev-commit
chk = checkout
```

git chk équivalent à git checkout

```
commit 125722c8f4099d6dc022c9bd3de21a90ad767376 (HEAD -> master, origin/master)
Merge: 838ba3d b91f86b
Author: TRIGianni <gianni.tricarico@heh.be>
Date:   Wed Sep 21 12:19:53 2022 +0200

    fusion

commit 838ba3d72cee08b064eed2a6dabcc729b410ed85
Author: TRIGianni <gianni.tricarico@heh.be>
Date:   Wed Sep 21 12:02:31 2022 +0200

    L7 cours - L10 HeH

commit b91f86b8b8a34287e5209d1fe4d5de5fdf1bc294 (origin/style, style)
Author: TRIGianni <gianni.tricarico@heh.be>
Date:   Wed Sep 21 11:36:03 2022 +0200

    Ajout fichier style.css

commit aa089decf64d1632affbaa2d95dae251af3e9c71
Author: TRIGianni <gianni.tricarico@heh.be>
Date:   Wed Sep 21 11:35:17 2022 +0200
```

git log

```
* 125722c - (HEAD -> master, origin/master) fusion (12 months ago) <TRIGianni>
| \
| * b91f86b - (origin/style, style) Ajout fichier style.css (12 months ago) <TRIGianni>
| * aa089de - Ajout <link> dans index.html (12 months ago) <TRIGianni>
* | 838ba3d - L7 cours - L10 HeH (12 months ago) <TRIGianni>
| /
* 25ca693 - Ajout h1 (1 year ago) <TRIGianni>
* c394506 - Premier commit (1 year ago) <TRIGianni>
```

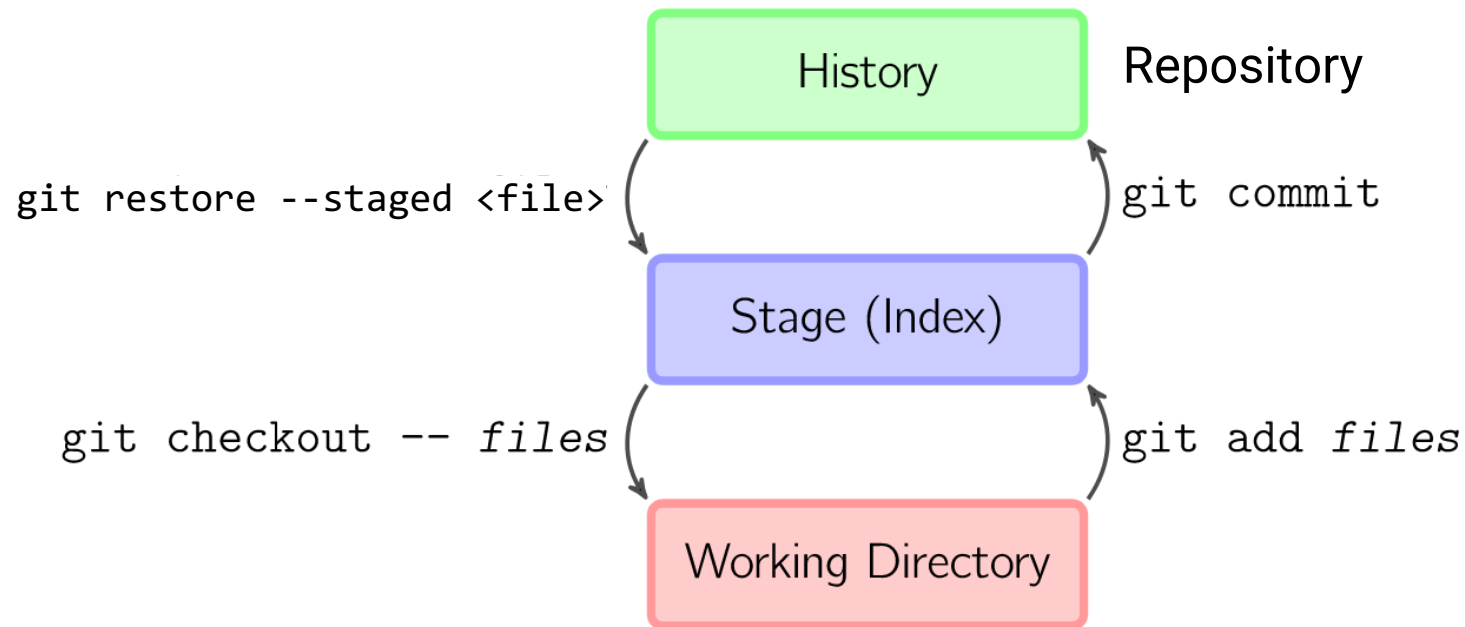
git lg

Editer le fichier : */projet/.gitignore*

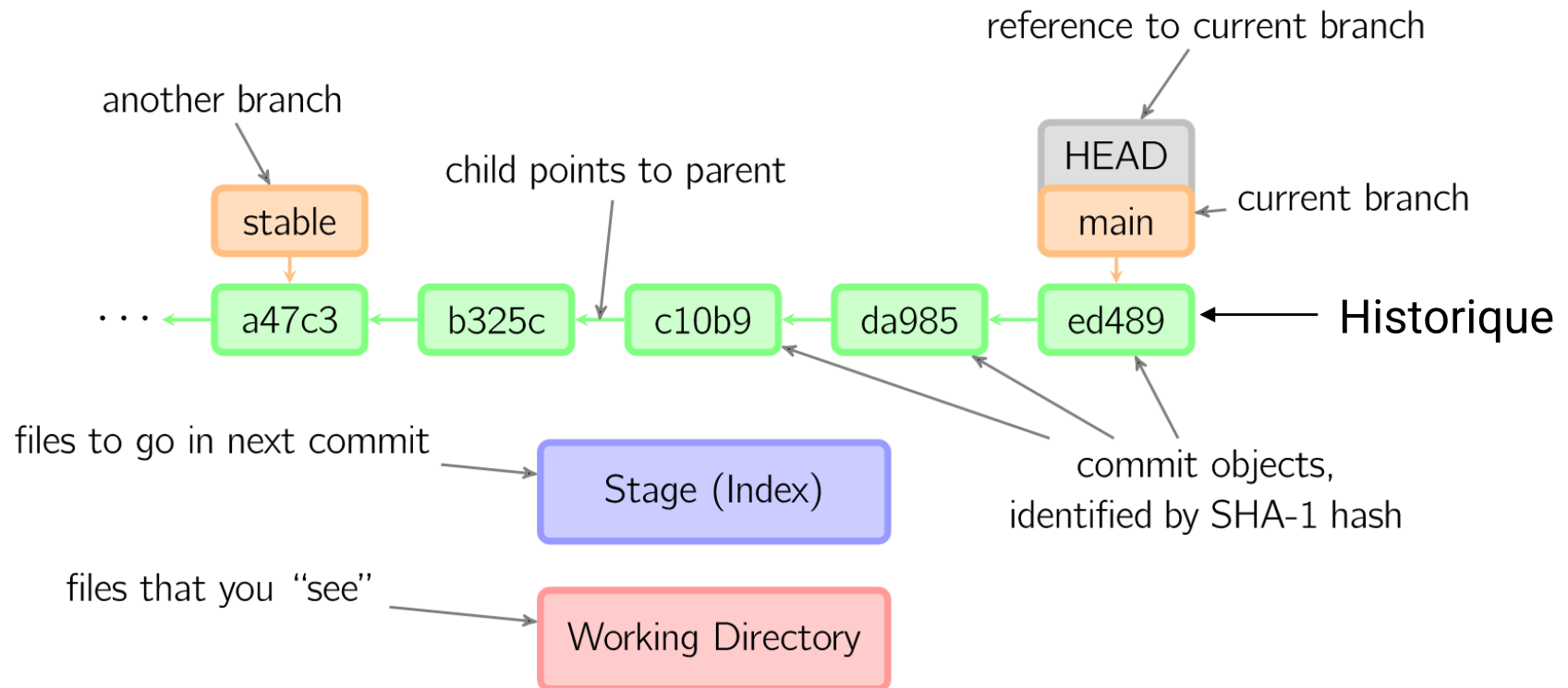
```
cat .gitignore
```

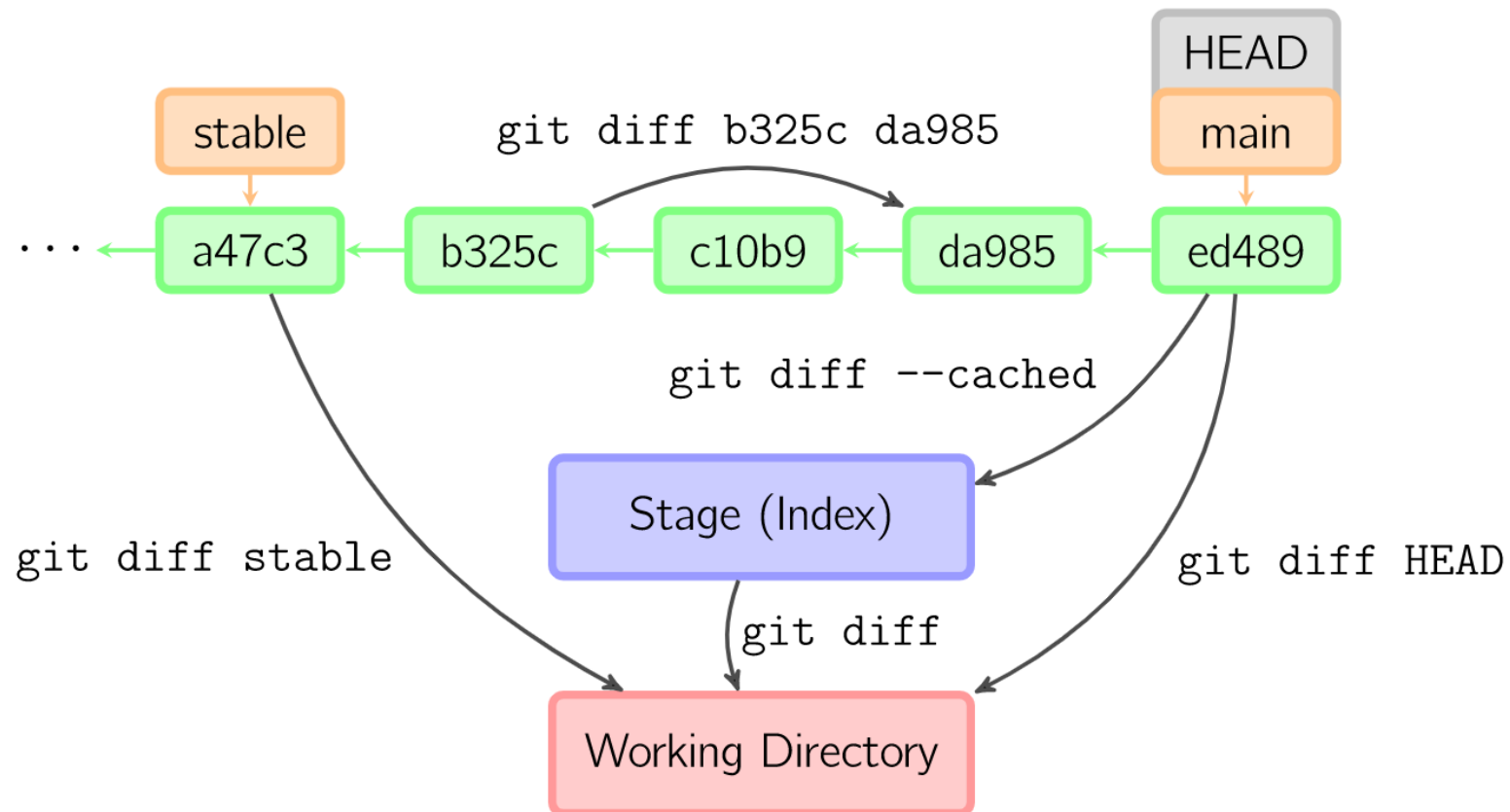
```
# pas de fichier .a
*.a
# mais suivre lib.a malgré la règle précédente
!lib.a
# ignorer uniquement le fichier TODO à la racine du projet
/TODO
# ignorer tous les fichiers dans le répertoire build
build/
# ignorer doc/notes.txt, mais pas doc/server/arch.txt
doc/*.txt
# ignorer tous les fichiers .txt sous le répertoire doc/
doc/**/*.txt
```

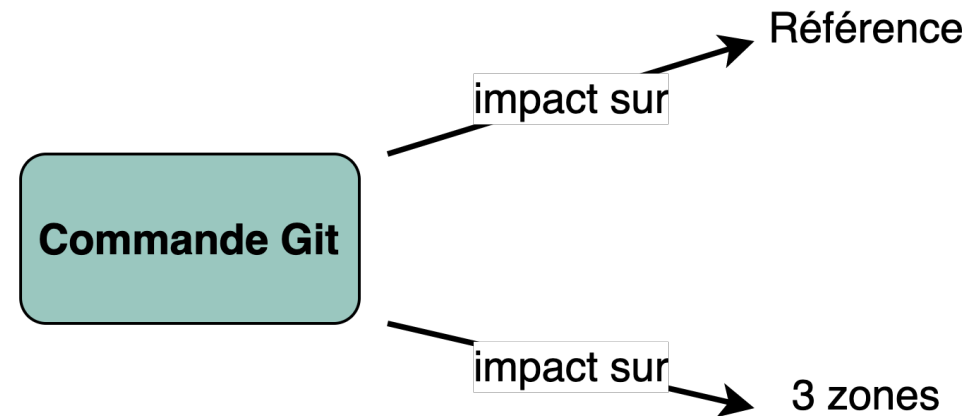
Règles prédéfinies : <https://github.com/github/gitignore>



- *git add fichiers* : copie les *fichiers* vers le stage.
- *git commit* : fait un "cliché" du stage sous la forme d'un commit.
- *git reset -- fichiers* : supprime les fichiers du stage ; i.e. elle copie les *fichiers* du dernier commit vers le stage.
- *git checkout -- fichiers* : copie les *fichiers* du stage vers la working directory



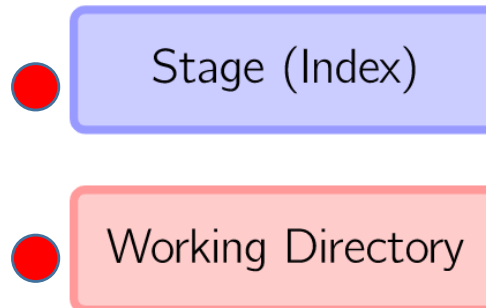
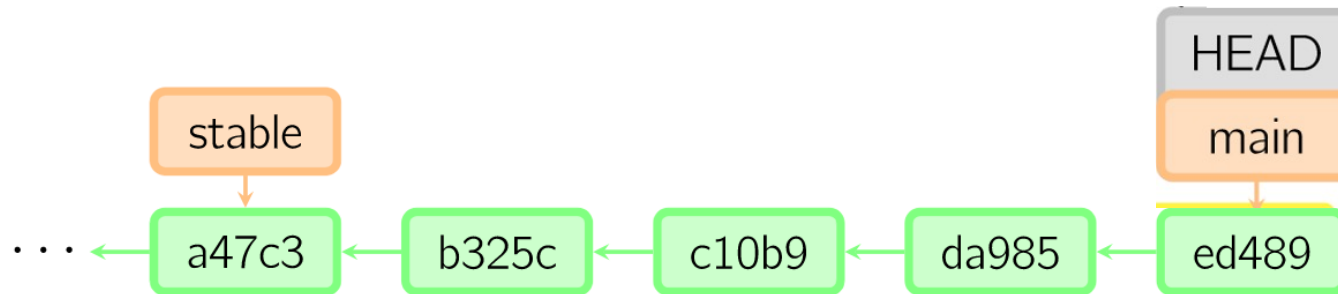




Il n'existe aucune commande qui permet de supprimer ou modifier un commit.

Seul le garbage collector est capable de supprimer un commit orphelin.

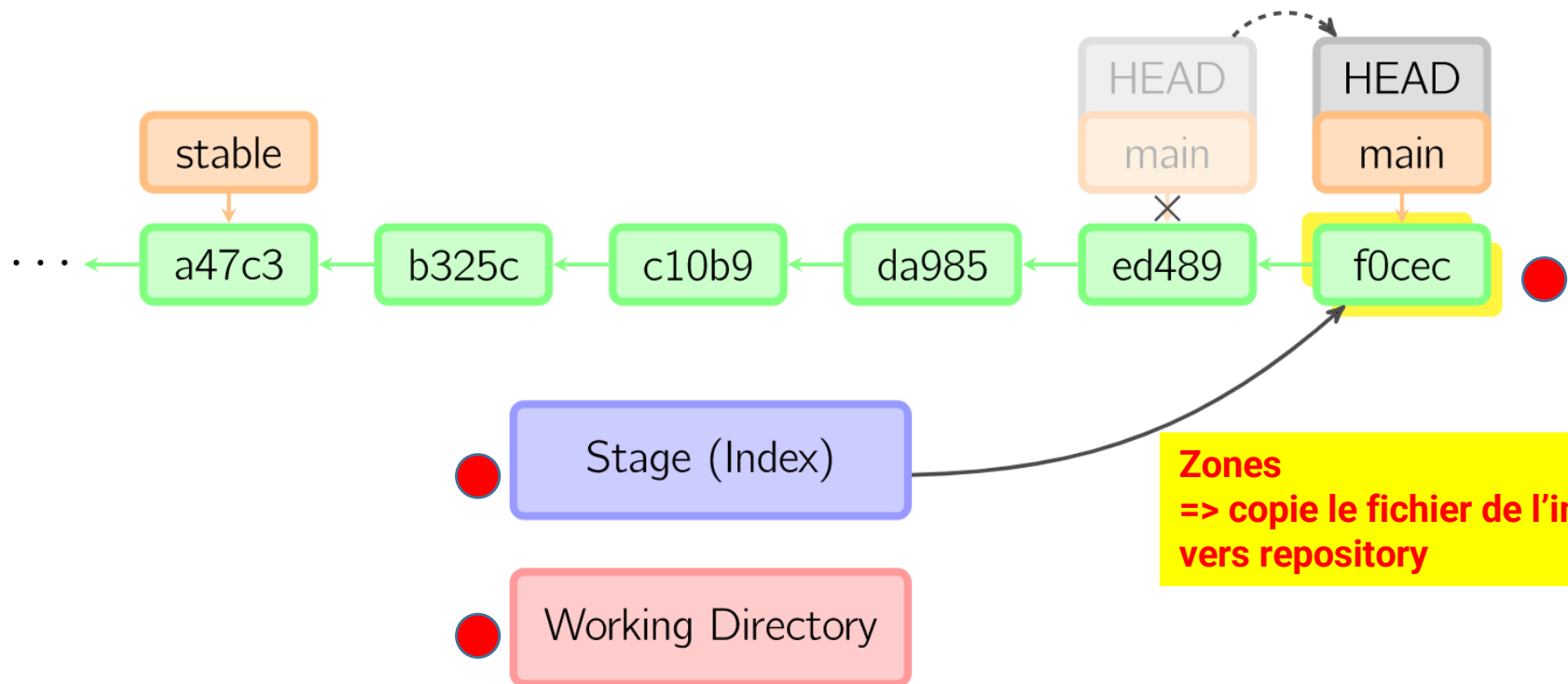
`git commit - m « message »`



Copie les fichiers présents dans la zone d'index vers le repository.

`git commit -m « message »`

Référence
=> déplacement branche + HEAD



Zones
=> copie le fichier de l'index
vers repository

Gestion sémantique de versions (SemVer)

MAJOR.MINOR.PATCH

Il faut incrémenter :

- 1.le numéro de version **MAJEUR** quand il y a des changements non rétrocompatibles,
- 2.le numéro de version **MINEUR** quand il y a des ajouts de fonctionnalités rétrocompatibles,
- 3.le numéro de version de **CORRECTIF** quand il y a des corrections d'anomalies rétrocompatibles.

<https://semver.org/>

Convention de nommage des commits

Le message du commit doit être structuré comme suit :

```
<type>[étendue optionnelle]: <description>  
  
[corps optionnel]  
  
[pied optionnel]
```

<https://www.conventionalcommits.org/fr/v1.0.0/>

<https://github.com/angular/angular/blob/main/CONTRIBUTING.md#commit>

Un commit DOIT être préfixé par un *type* :

- **fix**: un commit de *type* fix corrige un bogue dans le code (cela est en corrélation avec PATCH en gestion sémantique de versions).
- **feat**: un commit de *type* feat introduit une nouvelle fonctionnalité dans le code (cela est en corrélation avec MINOR en gestion sémantique de versions).
- **ci** : Modifications apportées à nos fichiers de configuration et scripts CI (exemples de scopes : Travis, Circle, BrowserStack, SauceLabs)
- **docs** : la documentation modifie uniquement
- **perf** : Un changement de code qui améliore les performances
- **refactor** : Un changement de code qui ne corrige pas de bug ni n'ajoute de fonctionnalité
- **style** : Modifications qui n'affectent pas la signification du code (espace blanc, formatage, points-virgules manquants, etc.)
- **test** : Ajout de tests manquants ou correction de tests existants

BREAKING CHANGE: un commit qui contient dans son *pied* le mot-clé BREAKING CHANGE:, ou dont le *type/étendue* est *suffixé d'un !*, introduit une rupture de compatibilité dans l'API (cela est en corrélation avec MAJOR en gestion sémantique de versions). Un BREAKING CHANGE peut faire partie des commits de n'importe quel *type*.

```
feat: allow provided config object to extend other configs
```

```
BREAKING CHANGE: `extends` key in config file is now used for extending other config files
```

```
feat!: send an email to the customer when a product is shipped
```

fix: prevent racing of requests

Introduce a request id and a reference to latest request. Dismiss incoming responses other than from latest request.

Remove timeouts which were used to mitigate the racing issue but are obsolete now.

Reviewed-by: Z

Refs: #123

git log

```
commit 6cd1a3779636182eef50c84efd0da2550b651d10 (HEAD -> master, origin/master)
Merge: 7ea6394 d426263
Author: TRIGianni <gianni.tricarico@heh.be>
Date: Thu Sep 24 14:21:25 2020 +0200

    Merge pull request #2 from Multydrive/master

    test

commit d4262633c1e98298138c5021024f9e6aa7edf92e
Author: Andreas <andreas.basso@std.heh.be>
Date: Thu Sep 24 14:14:23 2020 +0200

    git@github.com:Multydrive/devDemo.gitgit@github.com:Multydrive/devDemo.gitgi
    t@github.com:Multydrive/devDemo.git

    test

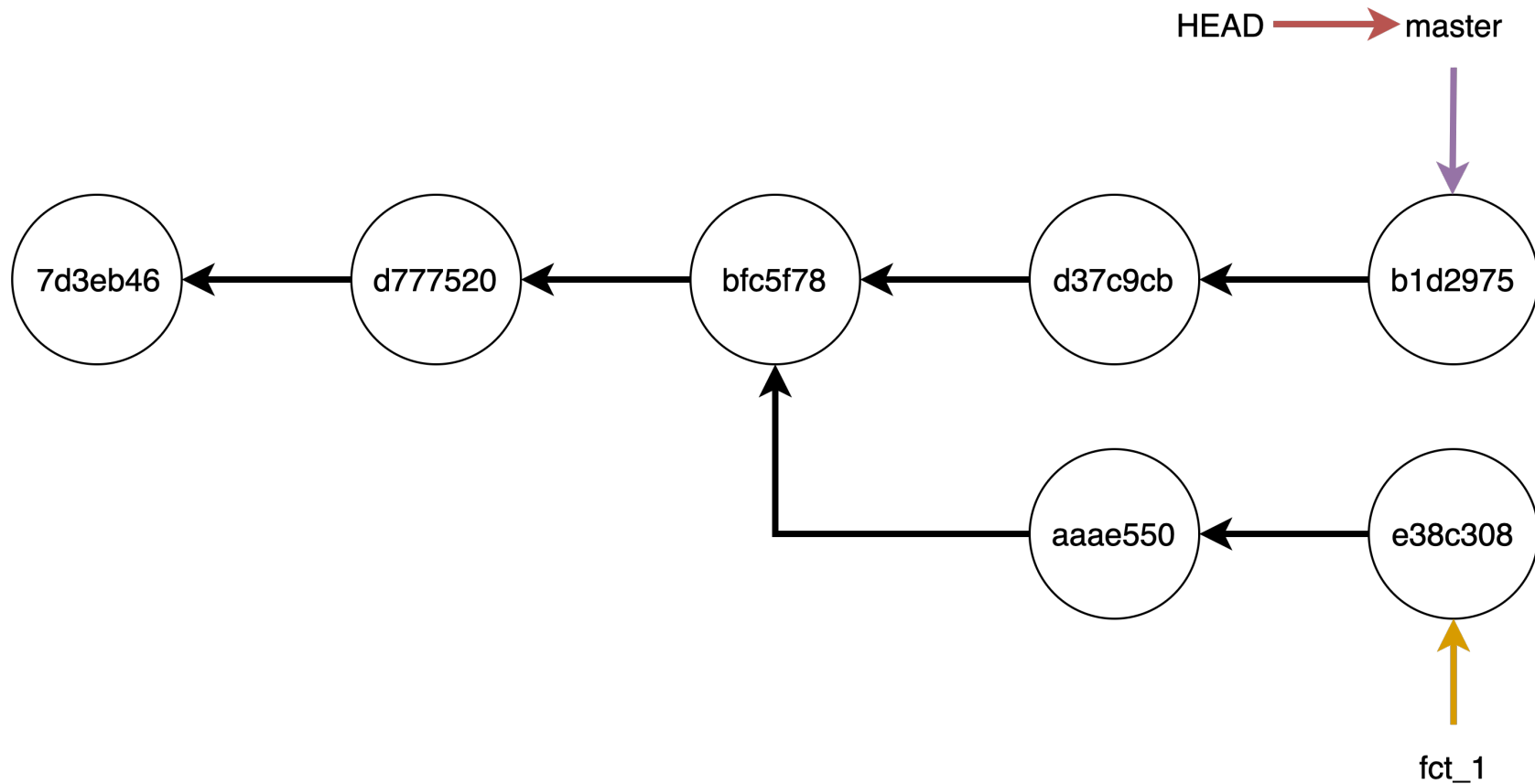
commit 7ea639403f260279a4b11c4637b80aae545097cb
Author: Gianni Tricarico <giannitricarico@MacBook-Pro-de-Gianni.local>
Date: Wed Sep 23 10:11:07 2020 +0200

    ajout <p>
```

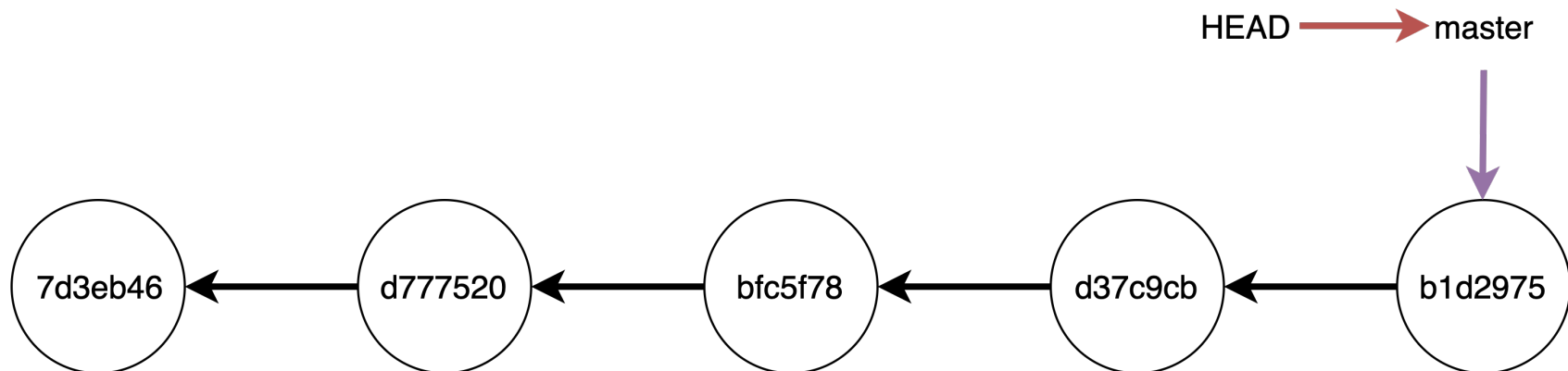
```
Git log --color --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset  
%s %Cgreen(%cr) %C(bold blue) %Creset' --abbrev-commit
```

```
* 6cd1a37 - (HEAD -> master, origin/master) Merge pull request #2 from Multydrive/master (1 year, 8  
months ago) <TRIGianni>  
|\  
| * d426263 - git@github.com:Multydrive/devDemo.gitgit@github.com:Multydrive/devDemo.gitgit@github.com  
:Multydrive/devDemo.git test (1 year, 8 months ago) <Andreas>  
|/  
* 7ea6394 - ajout <p> (1 year, 8 months ago) <Gianni Tricarico>  
* bd6fbff - fusion (1 year, 8 months ago) <Gianni Tricarico>  
|\  
| * 277ece5 - (style) ajout # (1 year, 8 months ago) <Gianni Tricarico>  
| * e38c308 - background-color : green (1 year, 8 months ago) <Gianni Tricarico>  
| * aae550 - style.css + p (1 year, 8 months ago) <Gianni Tricarico>  
* | b1d2975 - ajout * (1 year, 8 months ago) <Gianni Tricarico>  
* | d37c9cb - correction bug (1 year, 8 months ago) <Gianni Tricarico>  
|/  
* bfc5f78 - ajout <p> (1 year, 8 months ago) <Gianni Tricarico>  
* d777520 - ajout balise h1 (1 year, 8 months ago) <Gianni Tricarico>  
* 7d3eb46 - (tag: V1.0) création du fichier index.html (1 year, 8 months ago) <Gianni Tricarico>
```


Commandes – Visualiser l'historique des commits



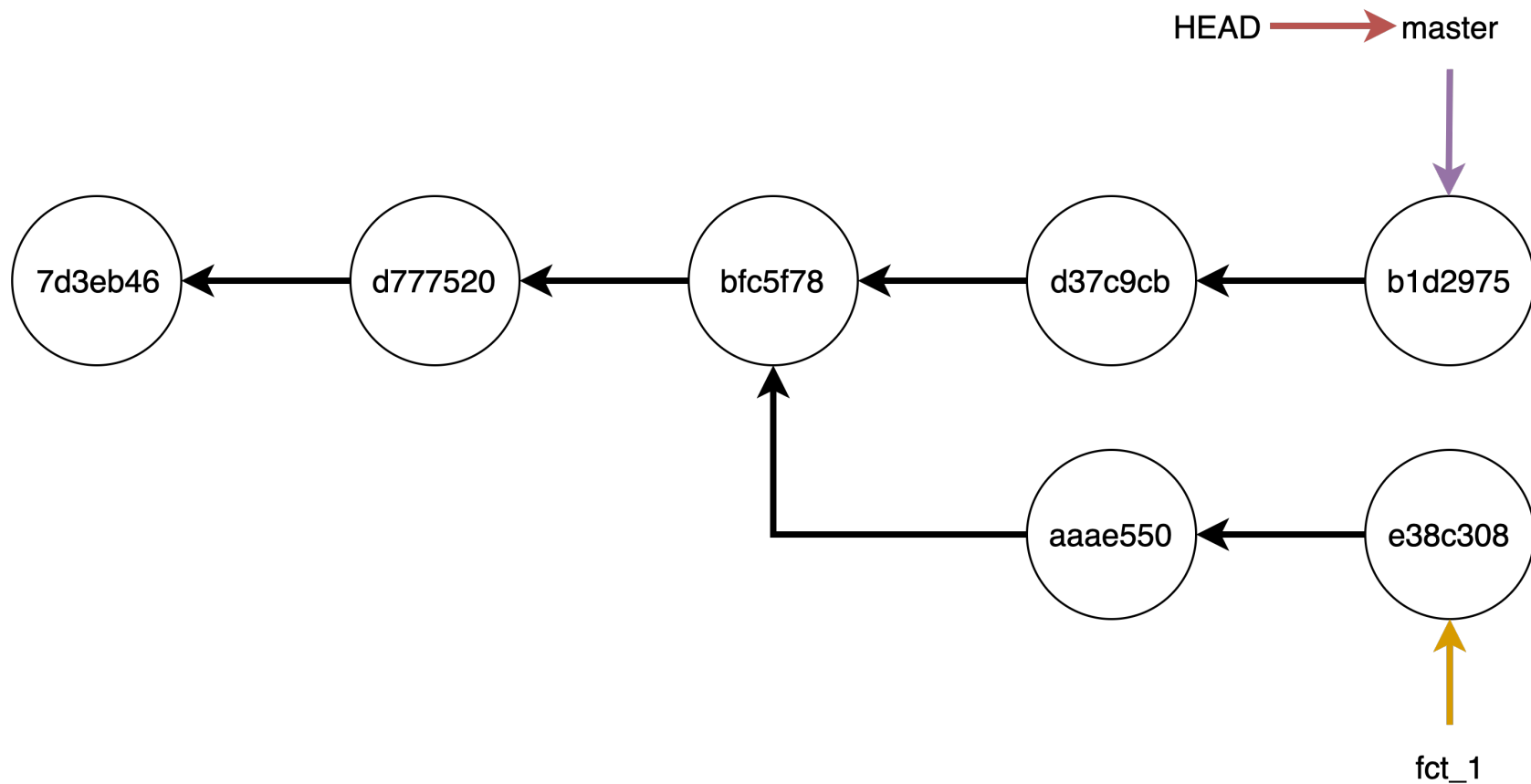
`git log`

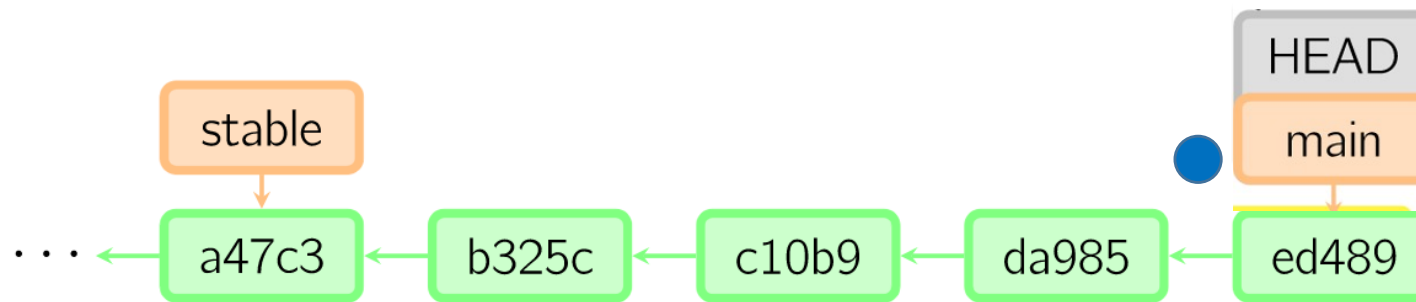


Visualiser l'historique des commits de la branche courante.

Commandes – Visualiser l'historique des commits

`git log --all`

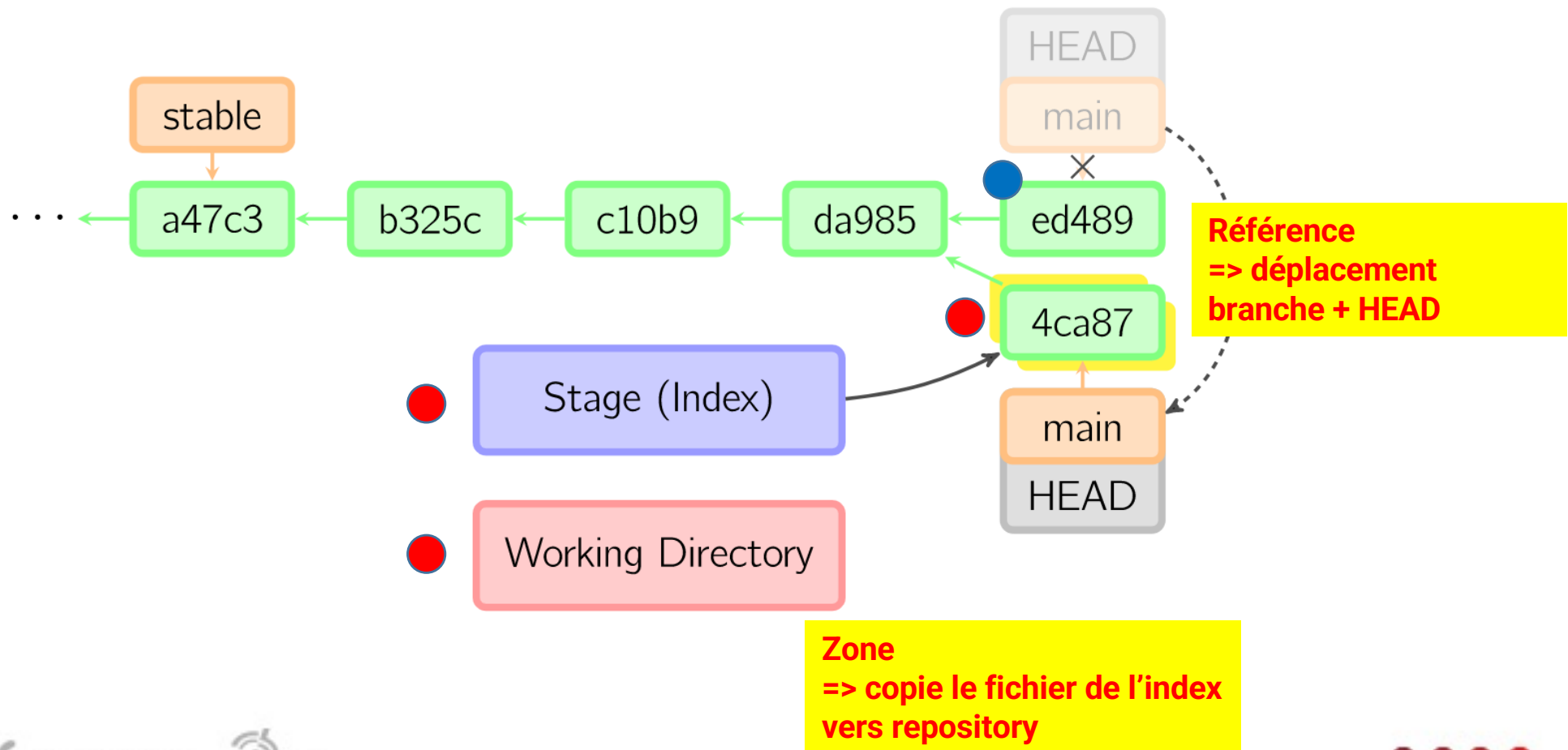




- Stage (Index)
- Working Directory

Permet de corriger (contenu et/ou message) le dernier commit.

`git commit --amend`



Git conserve un journal (fichier situé : `.git/logs/HEAD`) de tous les déplacements de la référence HEAD, accessible grâce à la commande :

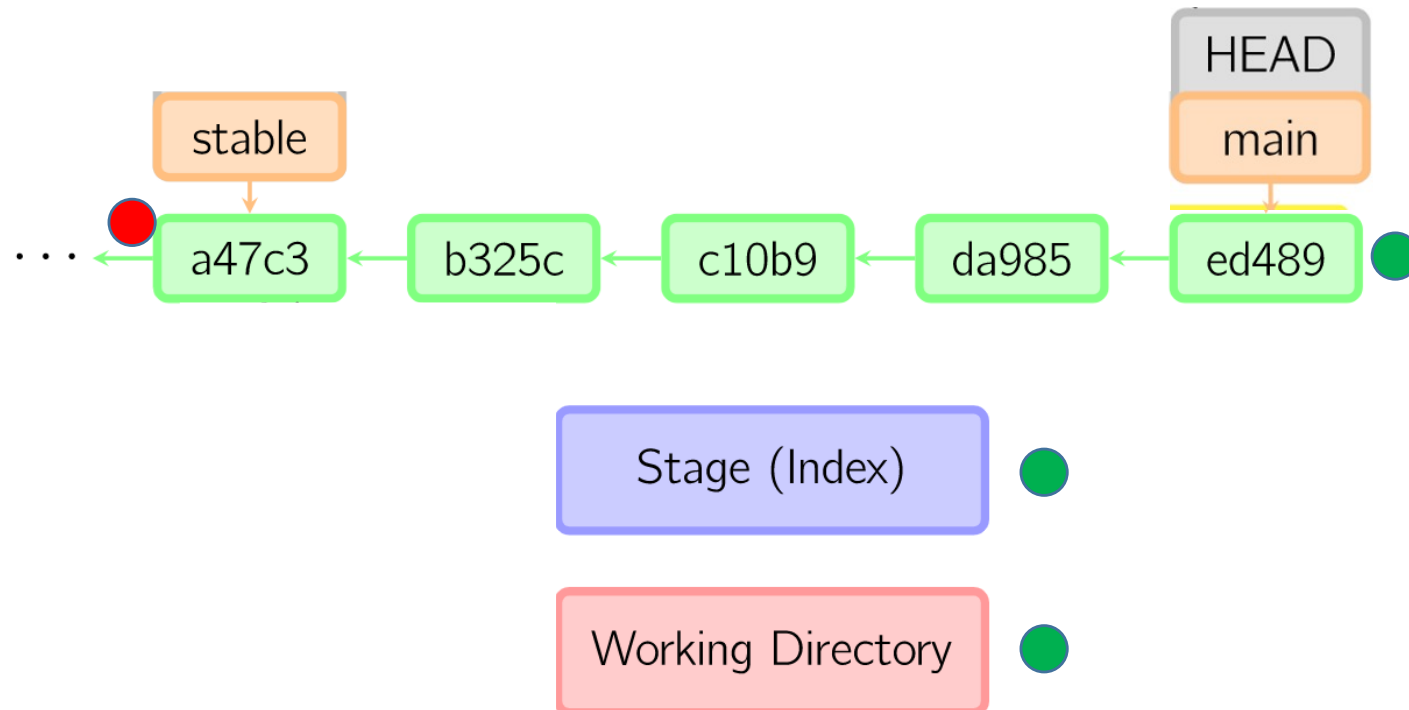
```
git reflog
```

Commande permettant de retrouver des commits orphelins.

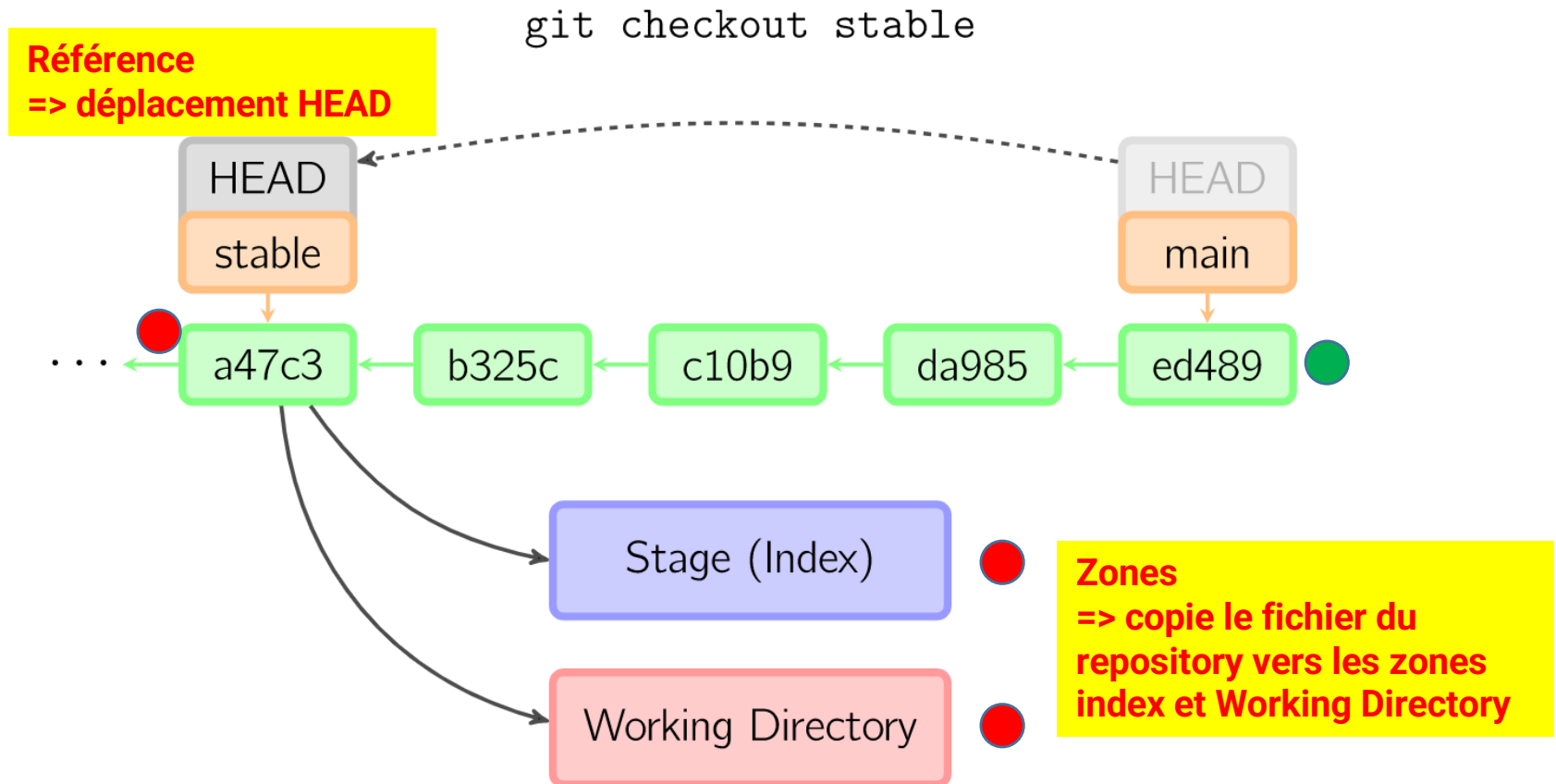
```
6cd1a37 (HEAD -> master, origin/master) HEAD@{0}: checkout: moving from 6cd1a3779636182eef50c84efd0da2550b651d10 to master
6cd1a37 (HEAD -> master, origin/master) HEAD@{1}: checkout: moving from master to origin/master
6cd1a37 (HEAD -> master, origin/master) HEAD@{2}: merge origin/master: Fast-forward
7ea6394 HEAD@{3}: commit: ajout <p>
bd6fbff HEAD@{4}: commit (merge): fusion
b1d2975 HEAD@{5}: checkout: moving from style to master
277ece5 (style) HEAD@{6}: commit: ajout #
e38c308 HEAD@{7}: checkout: moving from master to style
b1d2975 HEAD@{8}: commit: ajout *
d37c9cb HEAD@{9}: checkout: moving from bfc5f78b89870db0663b6615ce5472034f39e005 to master
bfc5f78 HEAD@{10}: checkout: moving from master to bfc5
d37c9cb HEAD@{11}: reset: moving to d37c9cb
627023d HEAD@{12}: merge style: Merge made by the 'recursive' strategy.
d37c9cb HEAD@{13}: checkout: moving from style to master
e38c308 HEAD@{14}: checkout: moving from master to style
d37c9cb HEAD@{15}: commit: correction bug
bfc5f78 HEAD@{16}: checkout: moving from style to master
e38c308 HEAD@{17}: checkout: moving from master to style
bfc5f78 HEAD@{18}: checkout: moving from style to master
e38c308 HEAD@{19}: reset: moving to HEAD
e38c308 HEAD@{20}: checkout: moving from master to style
bfc5f78 HEAD@{21}: checkout: moving from style to master
e38c308 HEAD@{22}: checkout: moving from master to style
```

```
git checkout [tag | sha1 | branche]
```

```
git checkout stable
```

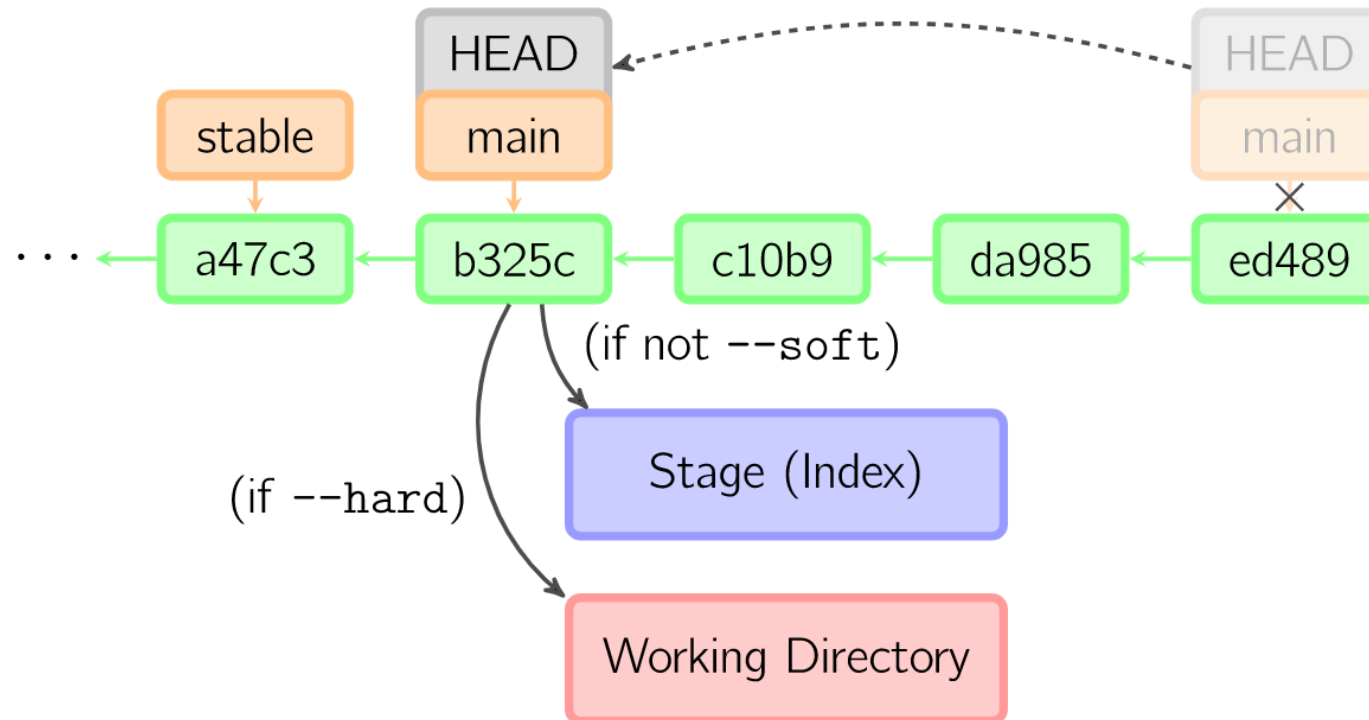


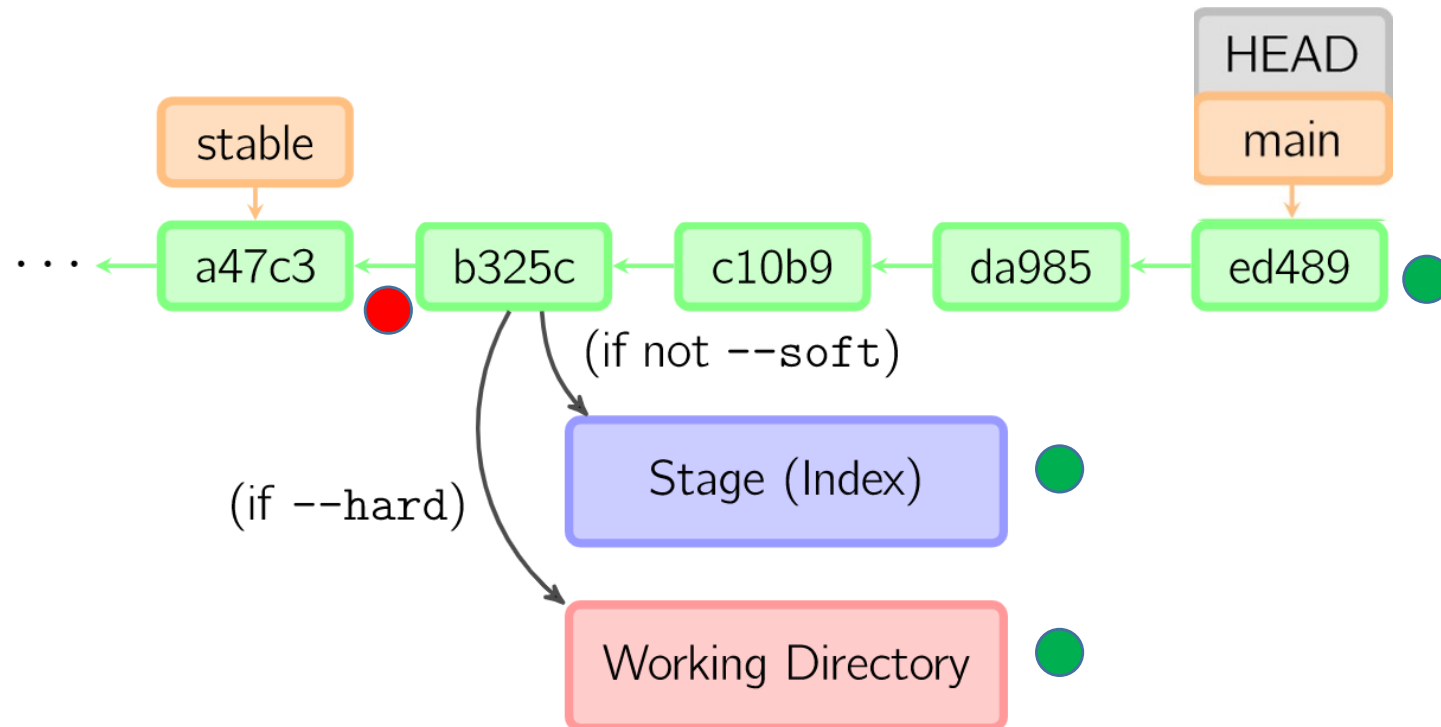
`git checkout [tag | sha1 | branche]`



- Commit parent : HEAD[^], A34D34F[^], branche[·], HEAD~1
- Commit grand-parent : HEAD^{^^} ou HEAD~2
- Commit grand-grand parent : HEAD^{^^^} ou HEAD~3
=> HEAD~n (nombre de génération)

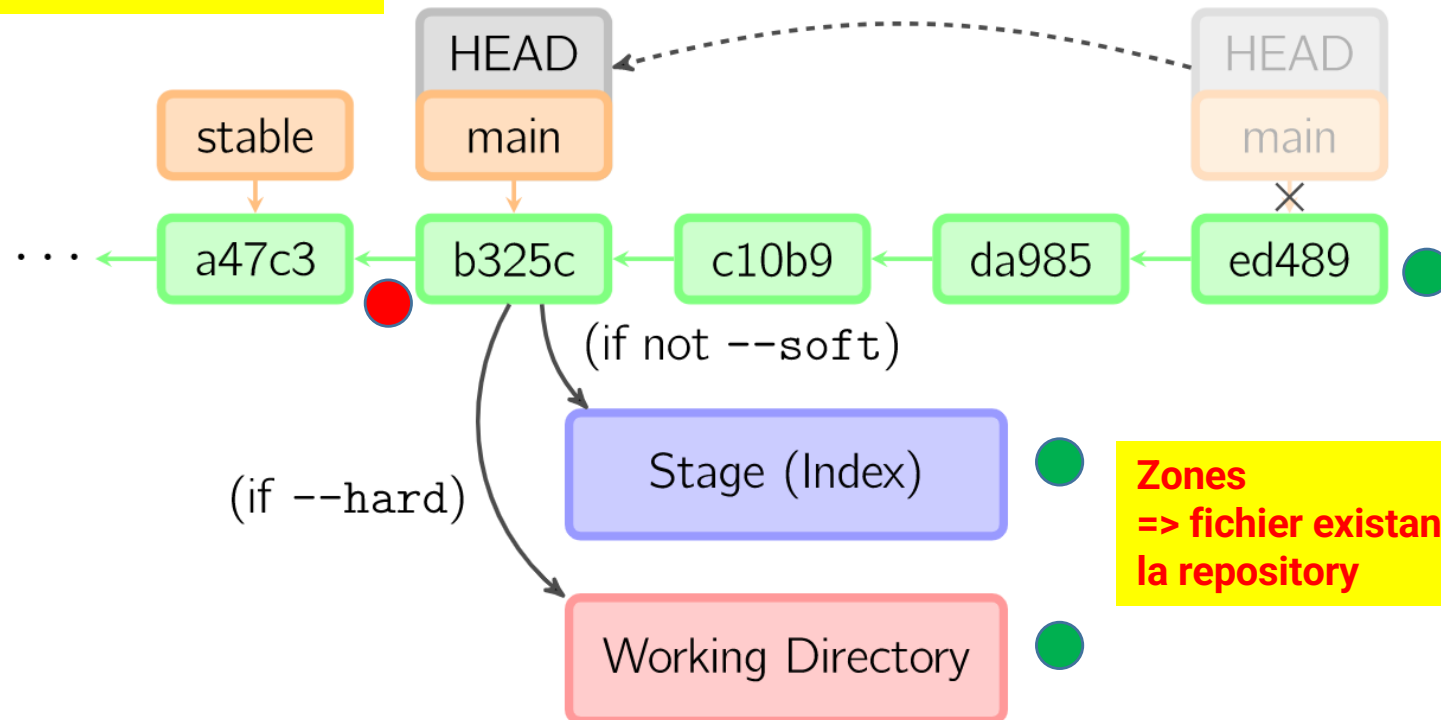
```
git reset [--soft|--mixed|--hard] [tag|sha1]
```



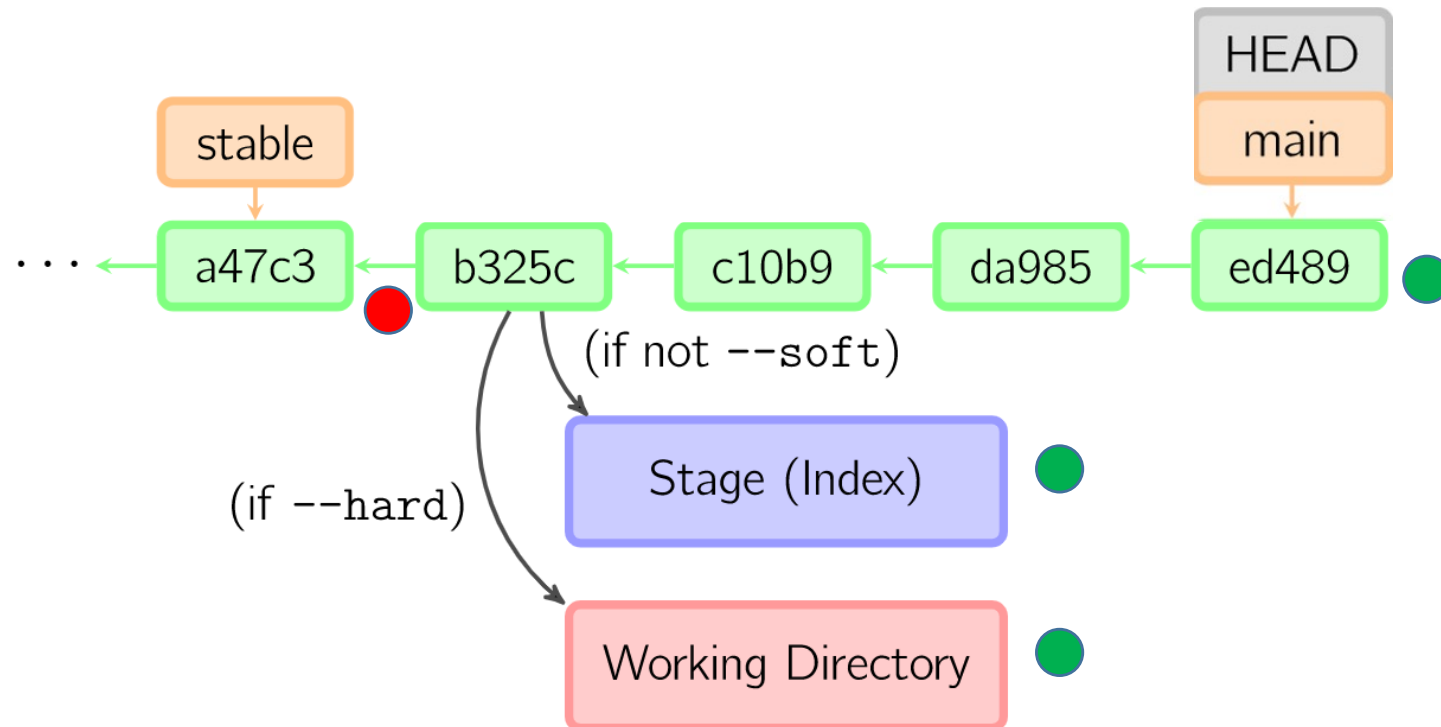


`git reset --soft HEAD~3`

Référence
=> déplacement HEAD
+ branche attachée

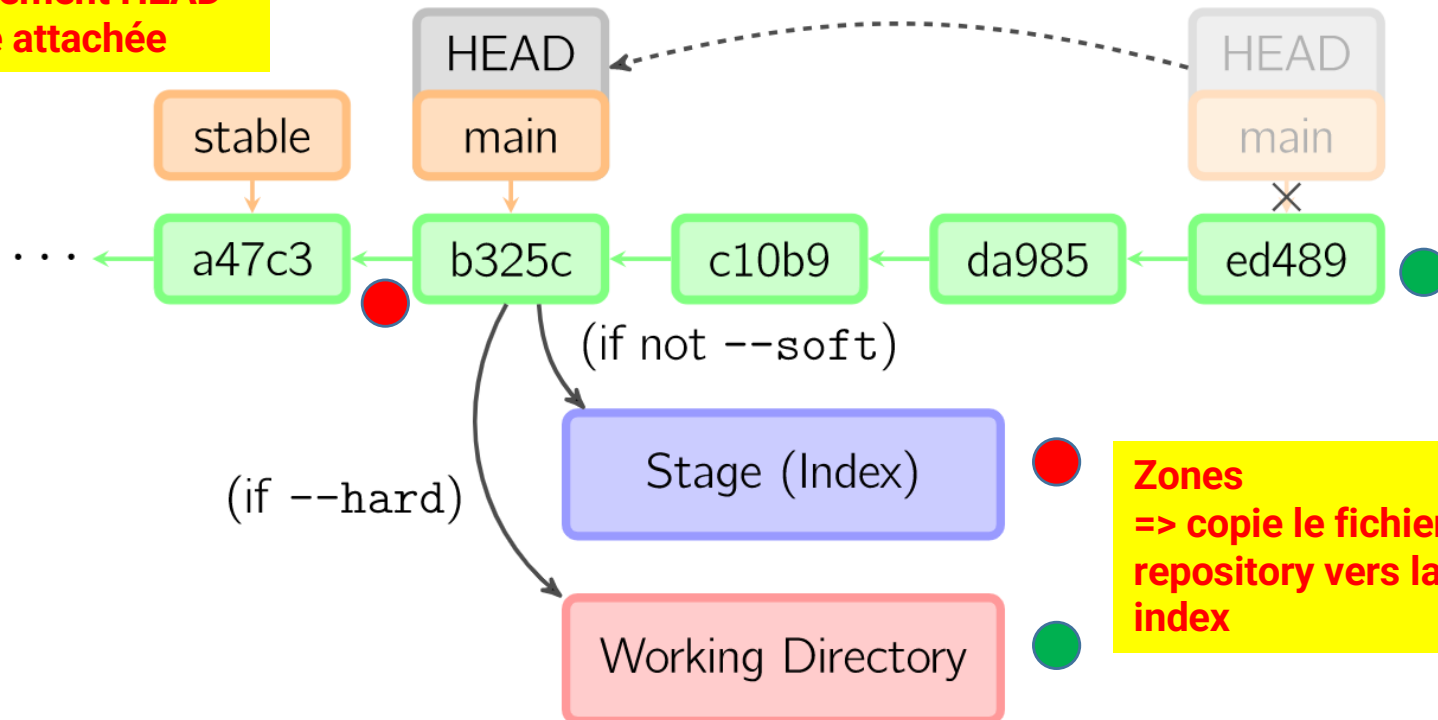


Zones
=> fichier existant dans
la repository

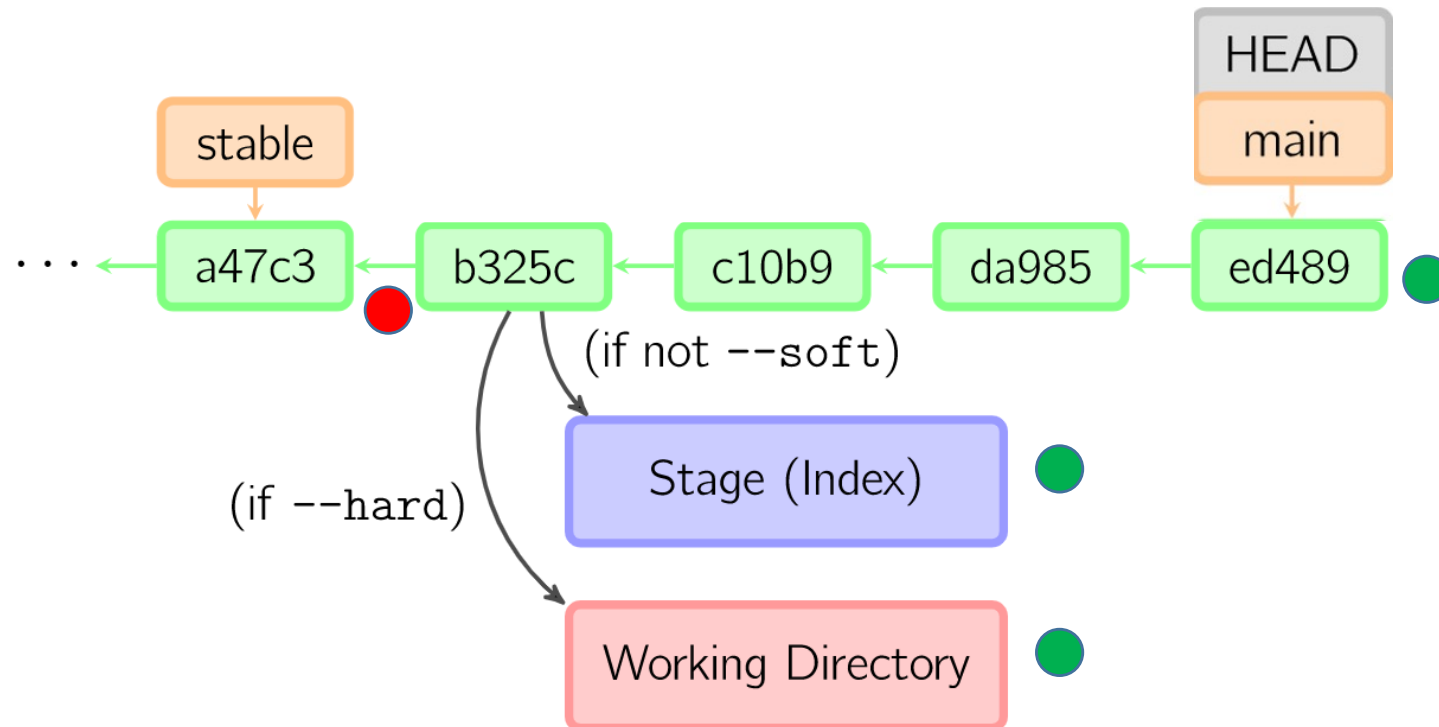


`git reset --mixed HEAD~3`
ou
`git reset HEAD~3`

Référence
=> déplacement HEAD
+ branche attachée

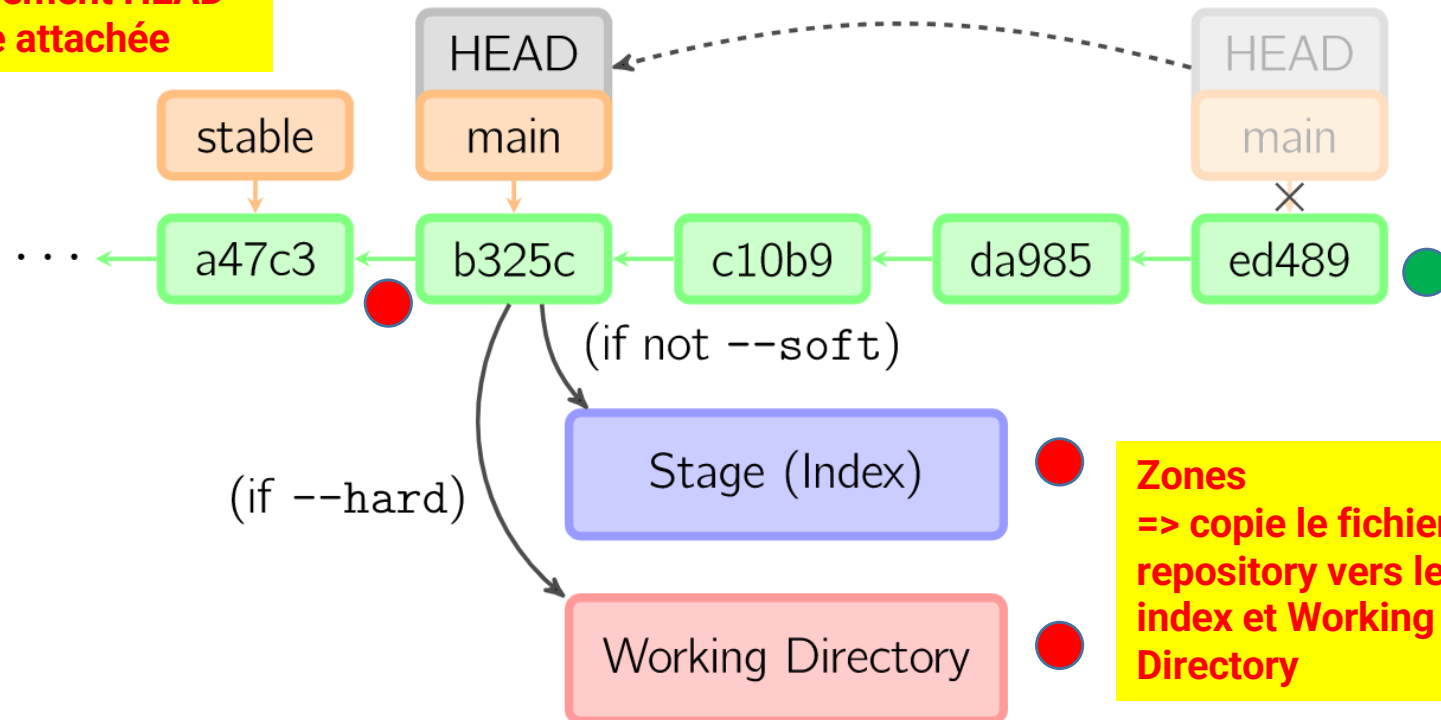


Zones
=> copie le fichier du
repository vers la zone
index

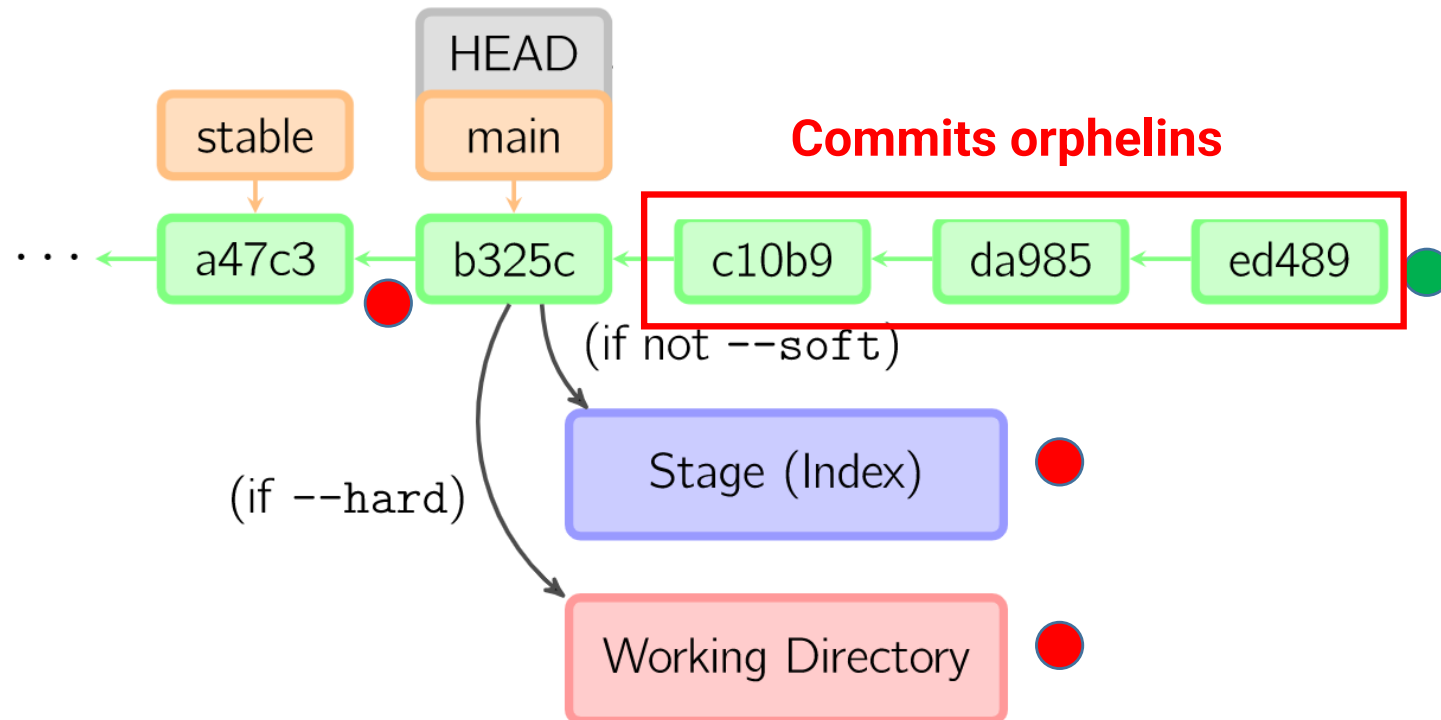


`git reset --hard HEAD~3`

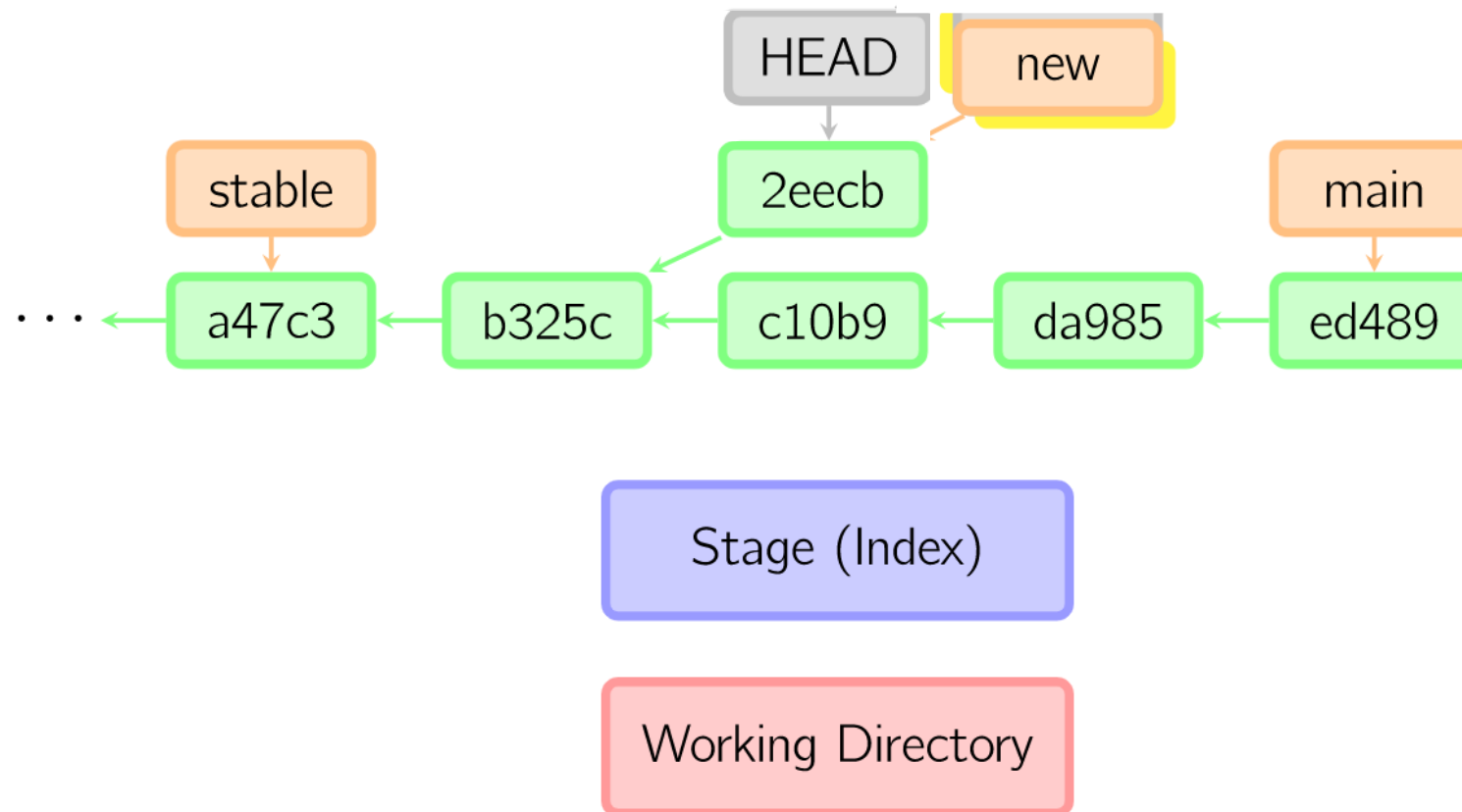
Référence
=> déplacement HEAD
+ branche attachée



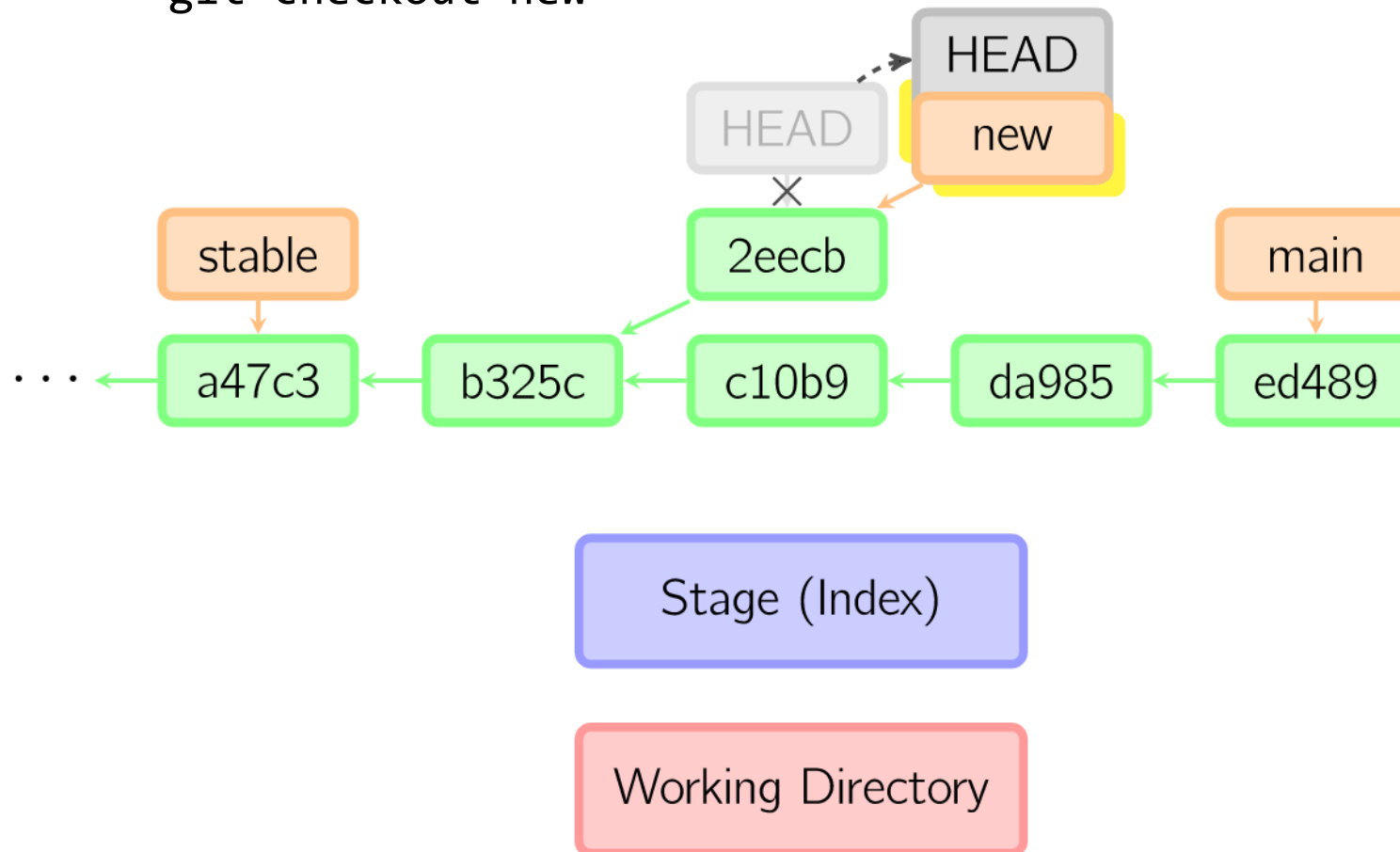
```
git reset --hard HEAD~3
```



`git branch new`



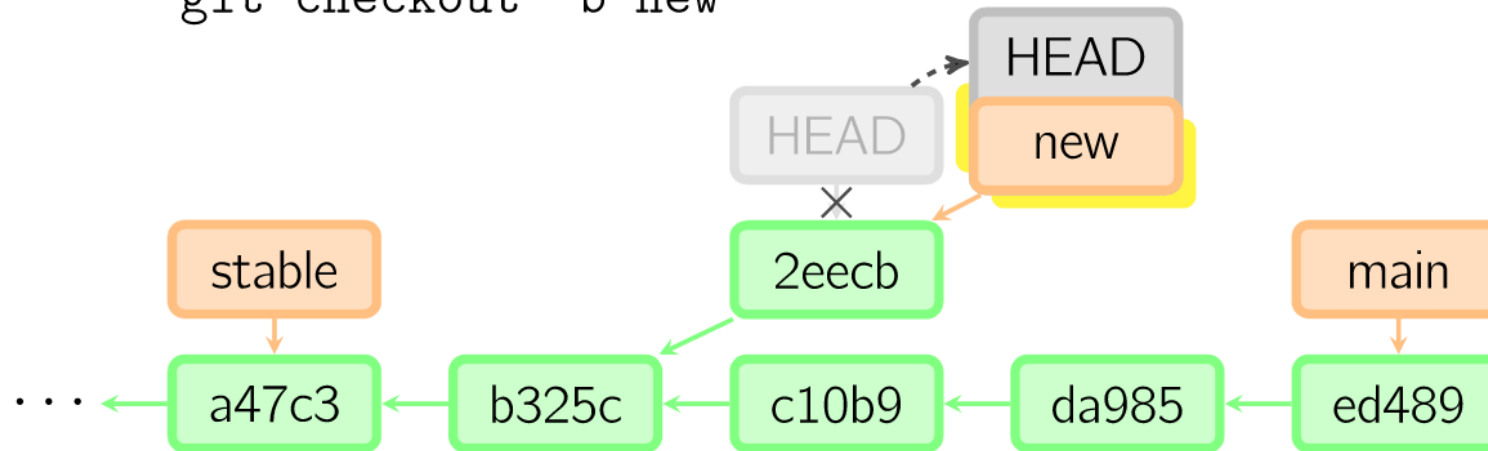
`git checkout new`



- Lister toutes les branches locales
`git branch`
- Master ou main : la branche par défaut
* : branche courante
- Créer une branche "nomBranche"
`git branch nomBranche`
- Basculer sur la branche "nomBranche"
`git checkout nomBranche`
- Créer et basculer en une seule commande
`git checkout -b nomBranche`
= `git branch nomBranche + git checkout nomBranche`

- Renommer une branche
`git branch -m ancienne nouvelle`
- Supprimer une branche
`git branch -d nomBranche`
- Supprimer sans vérification
`git branch -D nomBranche`

`git checkout -b new`



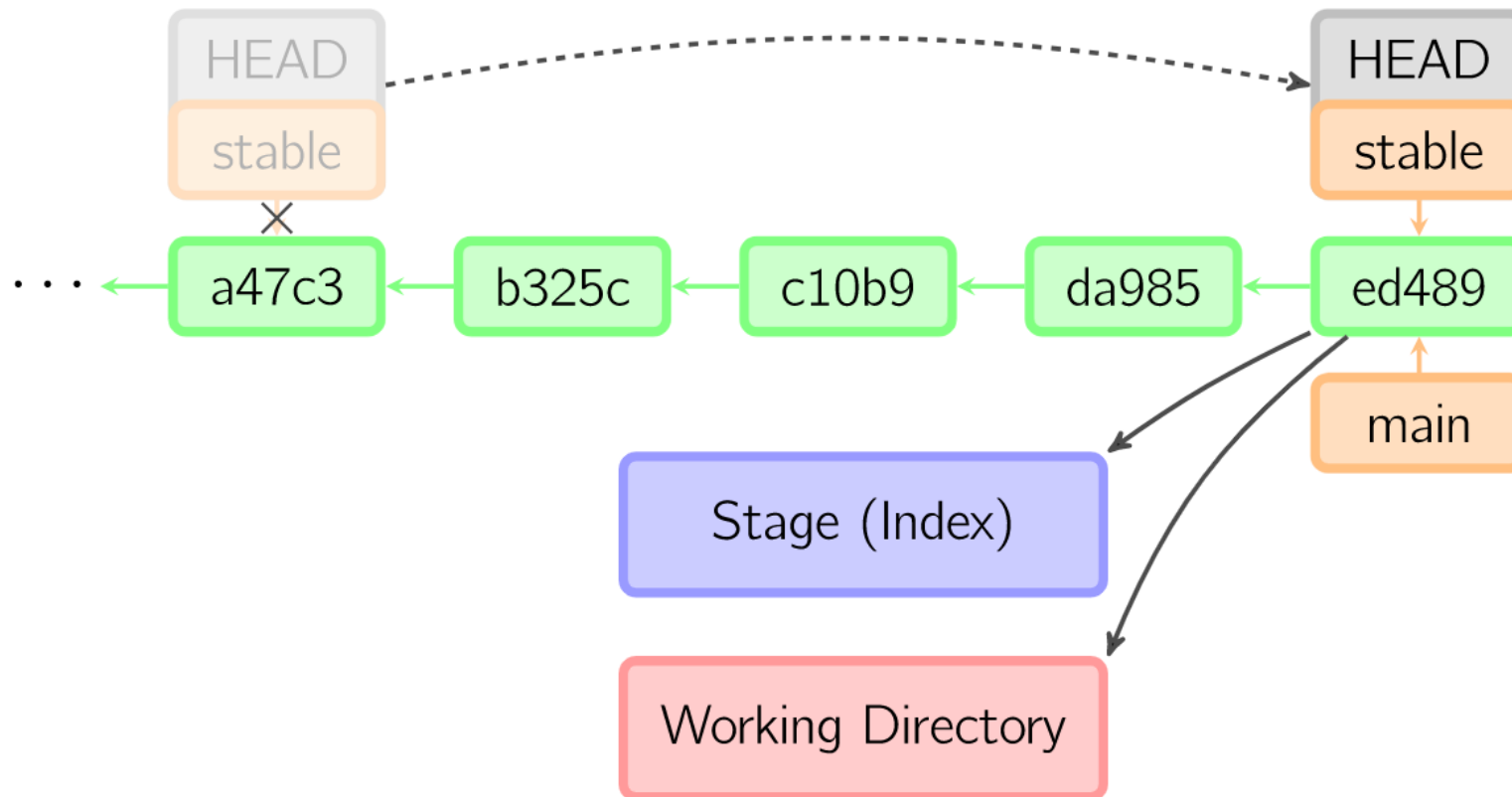
Stage (Index)

Working Directory

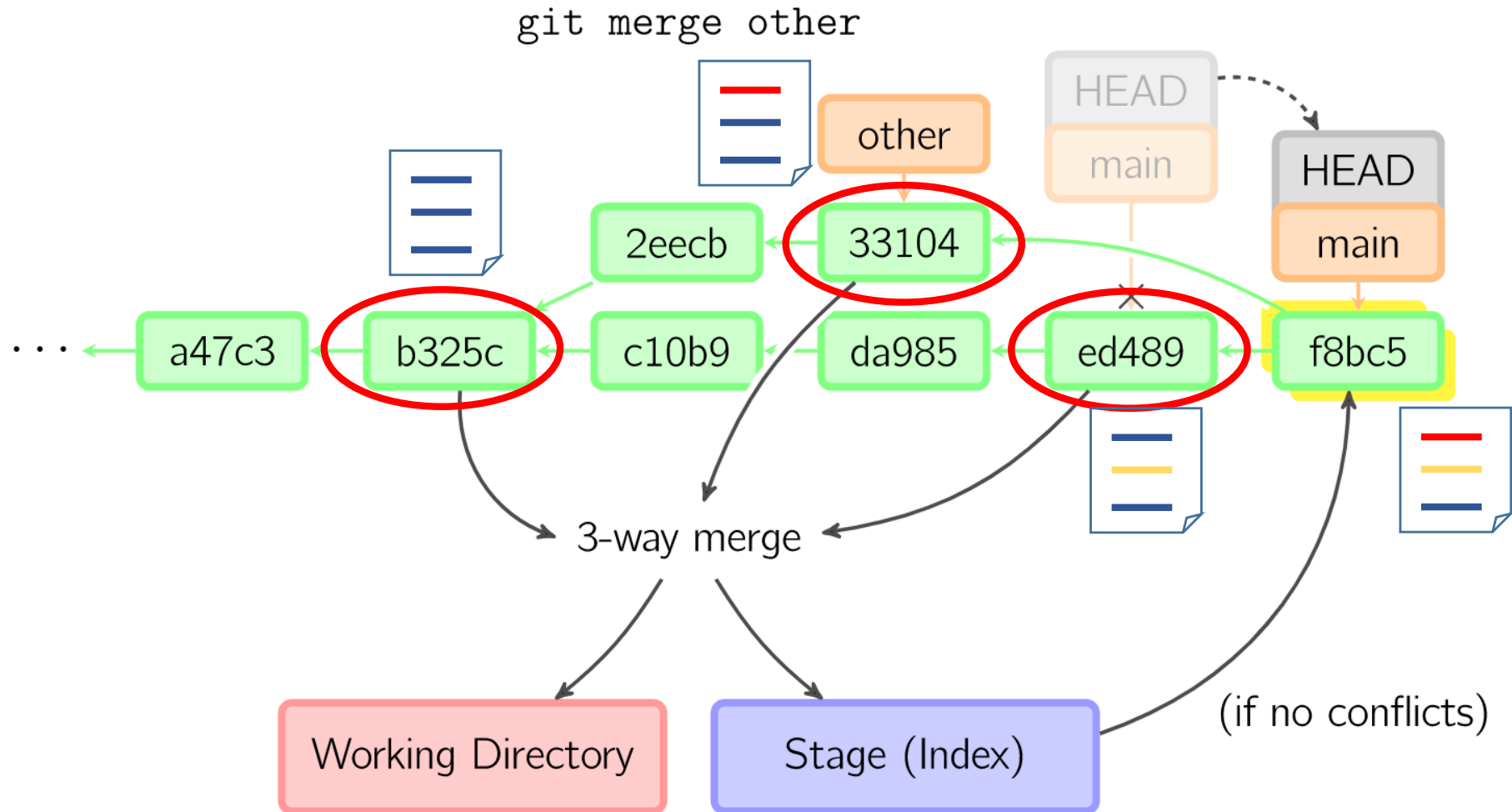
Démo

Commandes – fusion avance rapide–(Fast forward merge)

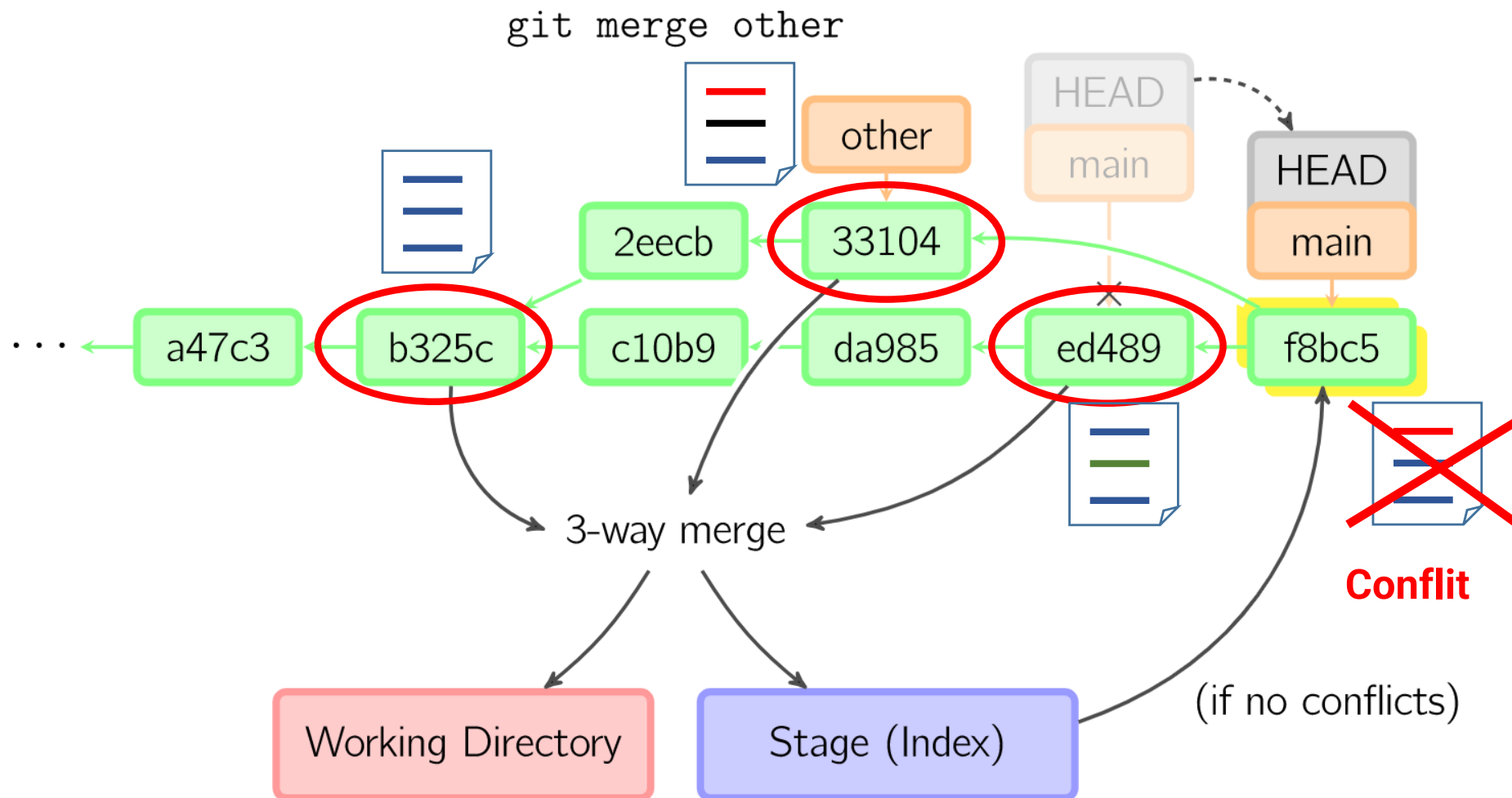
`git merge main`



Commandes – fusion à trois voies – (three-way merge)



Commandes – fusion à trois voies – (three-way merge)



Les trois possibilités de résolution d'un conflit :

- Annuler la fusion (abort)
`git merge -abort`
- Résoudre le conflit manuellement :
Editer le fichier confluel /corriger le conflit / ajouter à l'index / commiter le fichier
- Utiliser un outil de fusion (p4merge, Meld, Kdiff, ...)

TRI

Stash

Git stash

Git stash save nom

Git stash list

Git stash apply

Git stash pop

Collaboration

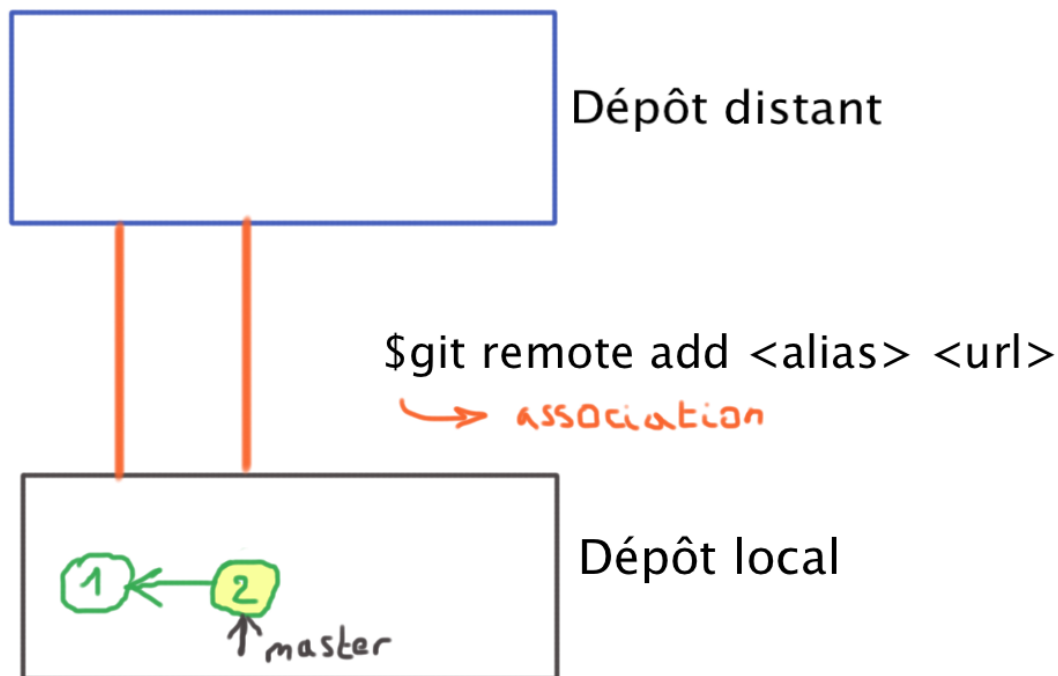
Réseau local

- Dépôt de type « bare »
- GitLab

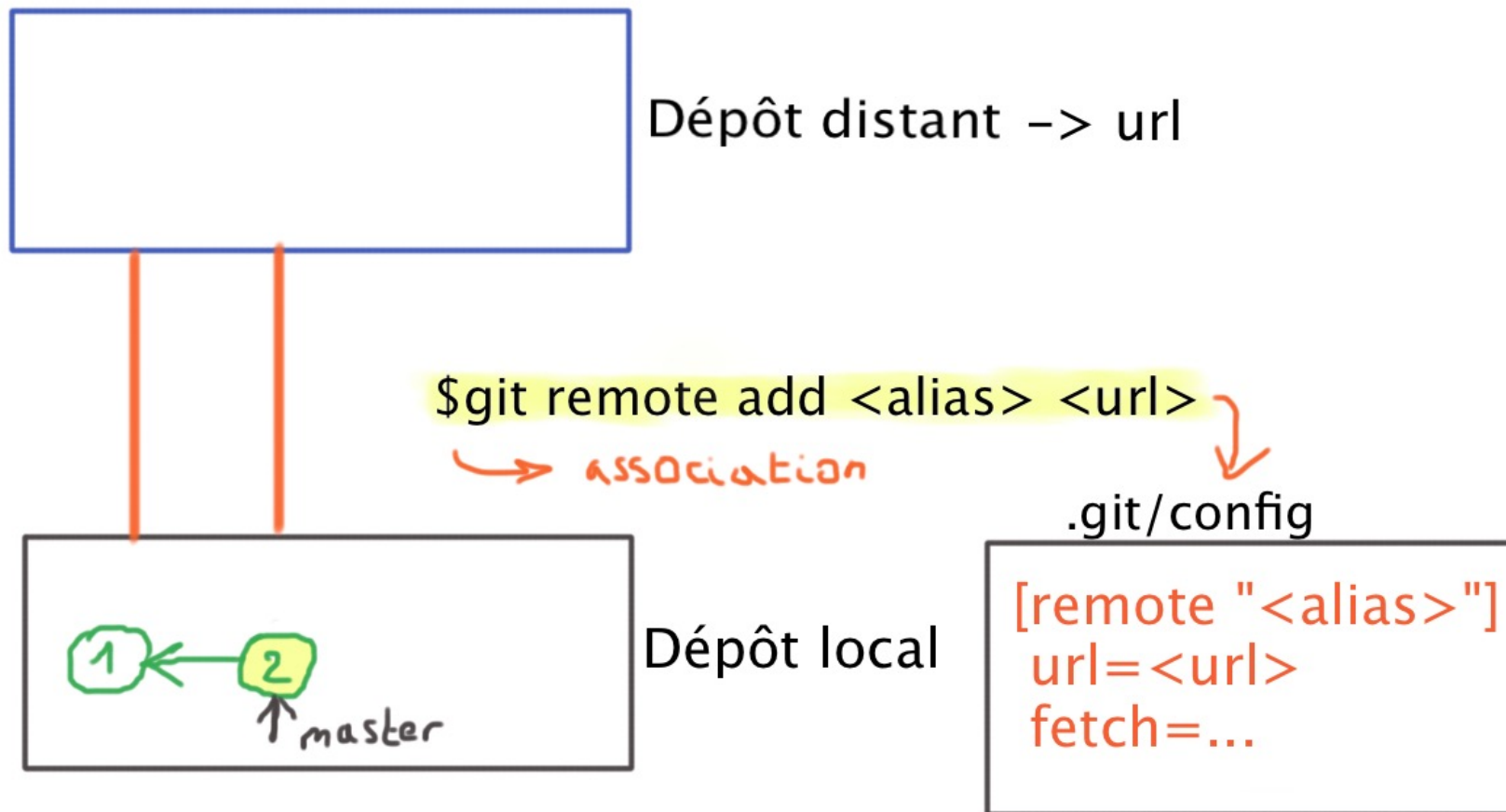
Github (plateforme sur le web):

- **URL** : [https ://github.com/](https://github.com/)
- **Dépôts publics** : gratuits
- **Dépôts privés** : payants
- **Très populaires** => projets Open Source
- **Exemples** : jQuery, Symfony, Ruby on Rails. . .

Crée une nouvelle connexion à un dépôt distant. Quand on a ajouté un dépôt distant, on peut utiliser `<alias>` comme raccourci pour `<url>` dans les autres commandes Git.

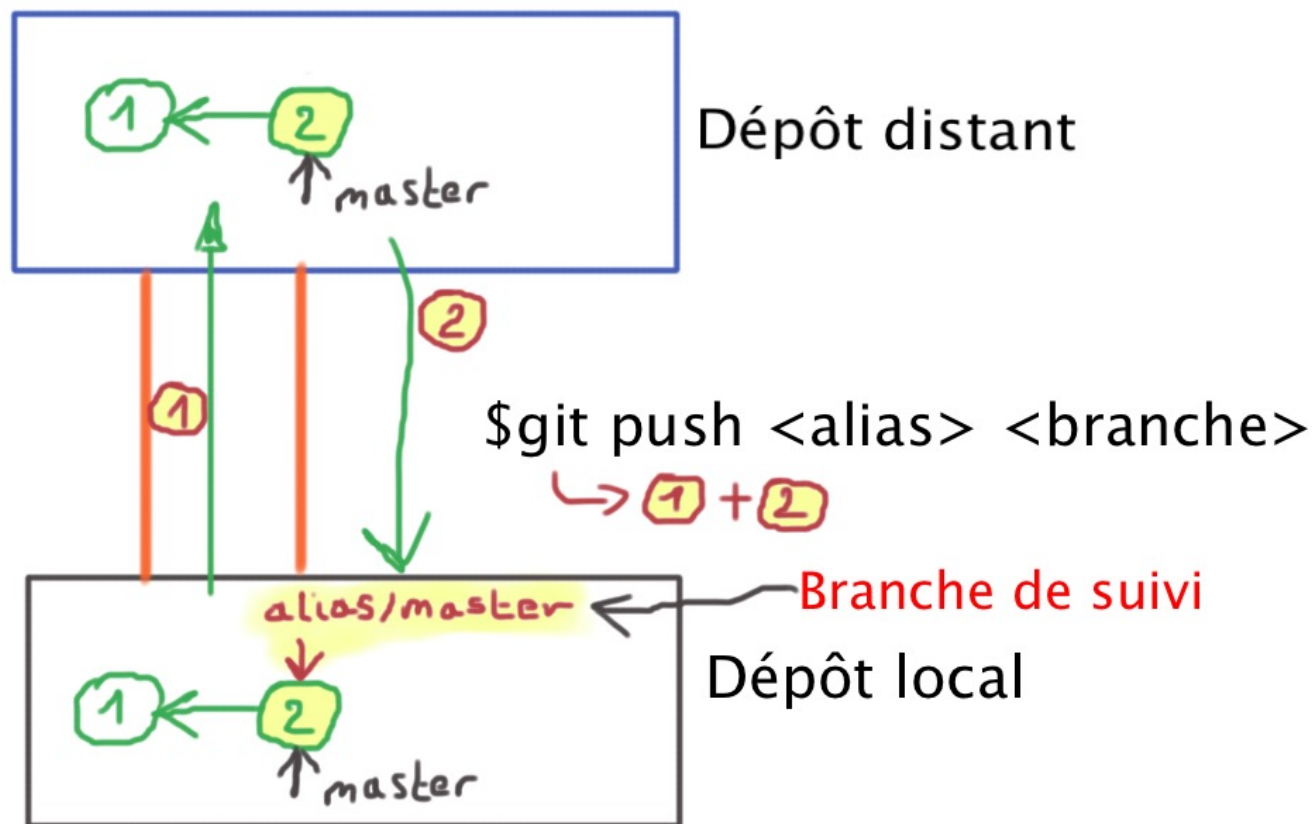


Gestion des dépôts distants - Ajouter un dépôt



Pousse la branche spécifiée vers <distant>, avec tous les commits et objets internes nécessaires. Crée une branche locale dans le dépôt de destination.

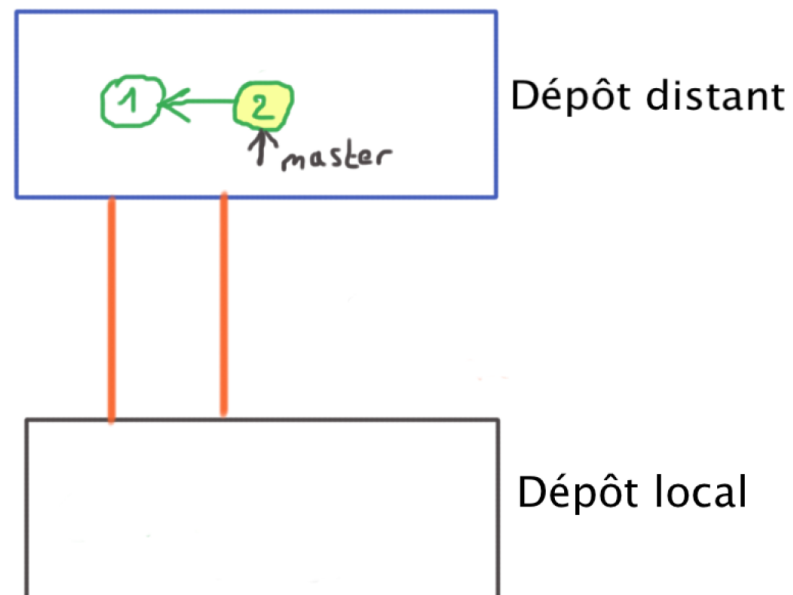
```
$ git push <distant> <branche>
```



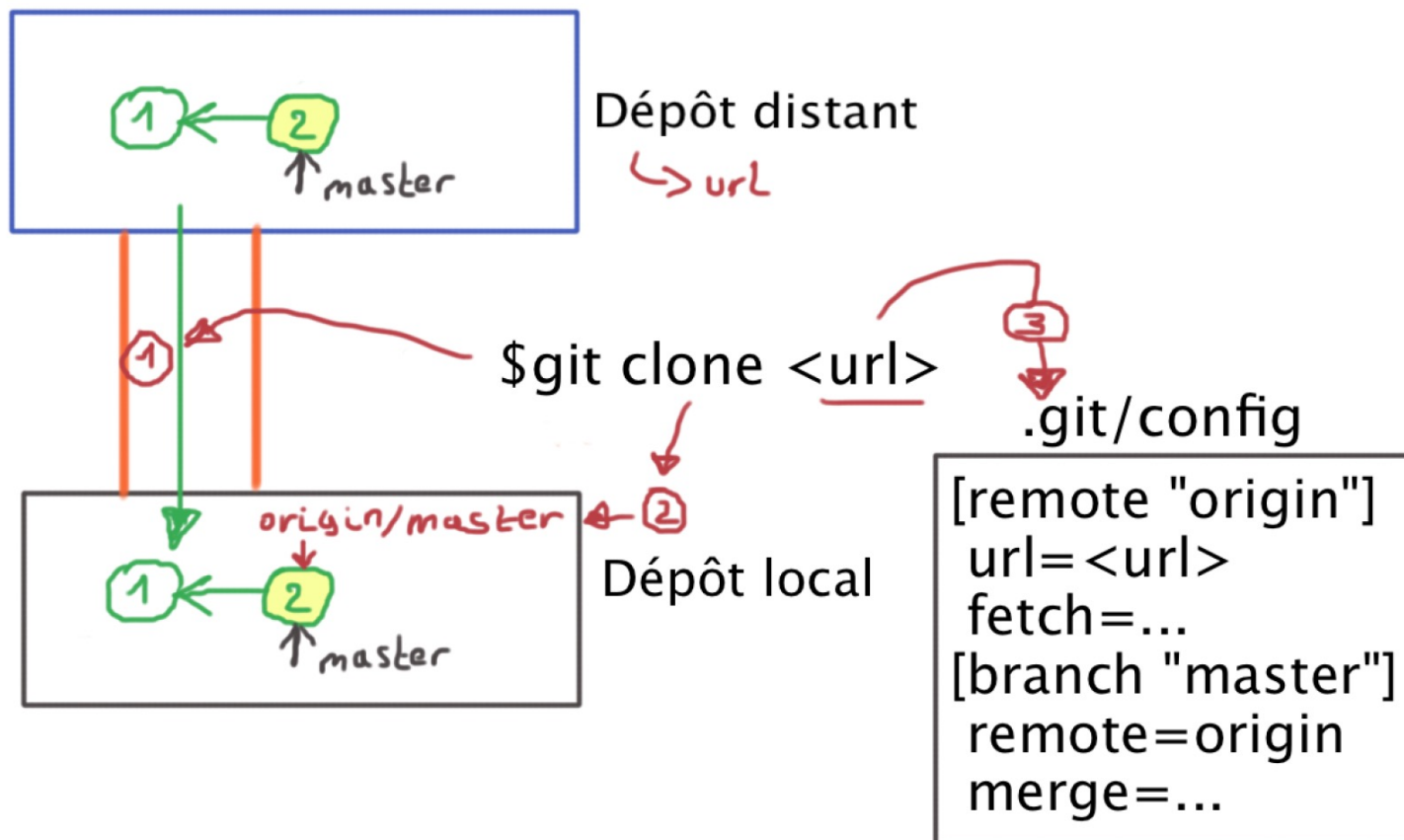
- Les branches de suivi sont des **branches locales** qui sont en relation directe avec une branche distante.
- Cette branche reflète **l'état de la branche distante** correspondante.
- Les branches distantes **ne peuvent pas être déplacées par l'utilisateur**, elles sont déplacées lors des synchronisations.

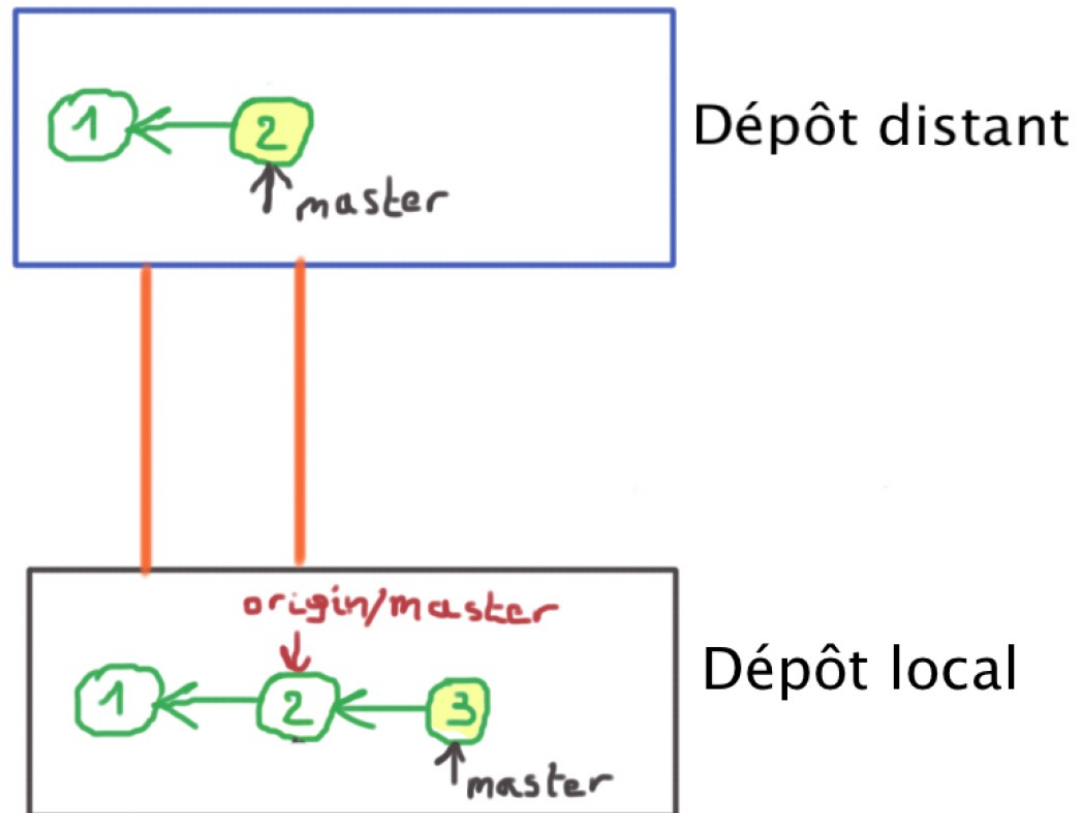
```
$ git clone <dépôt> <répertoire>
```

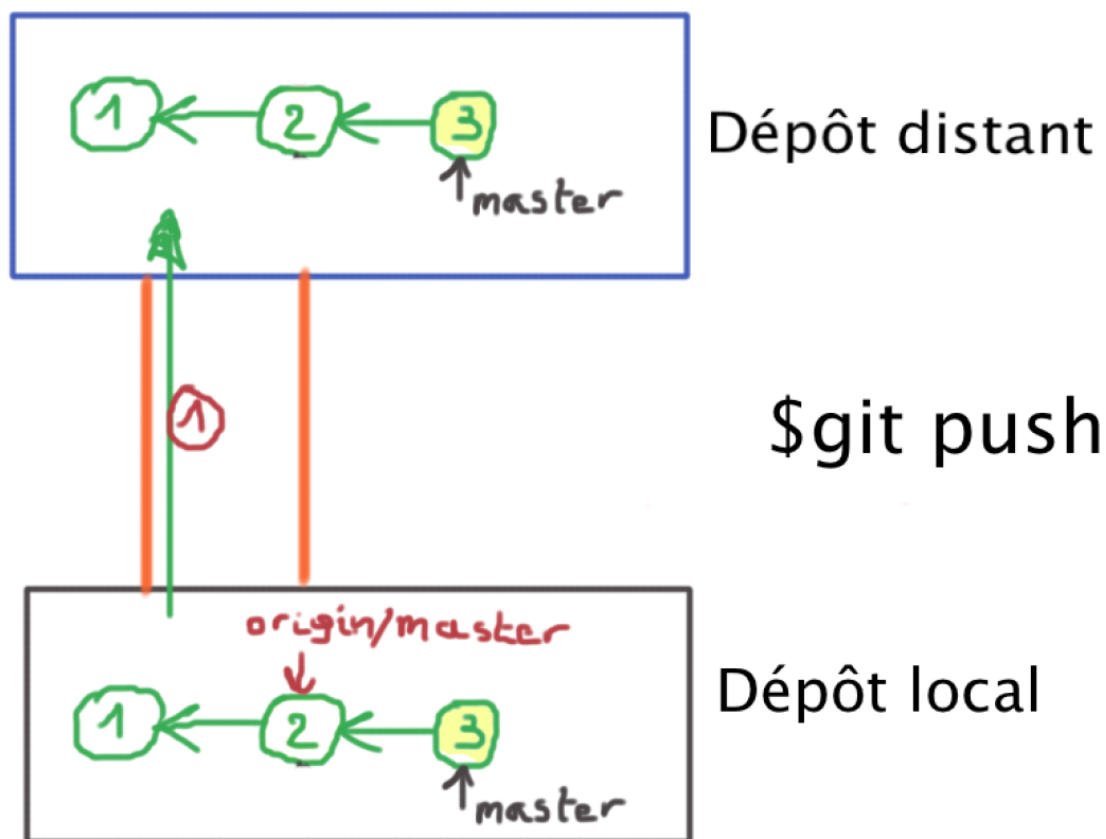
Clone le dépôt situé à l'adresse <dépôt> dans un répertoire intitulé <répertoire> sur la machine locale.

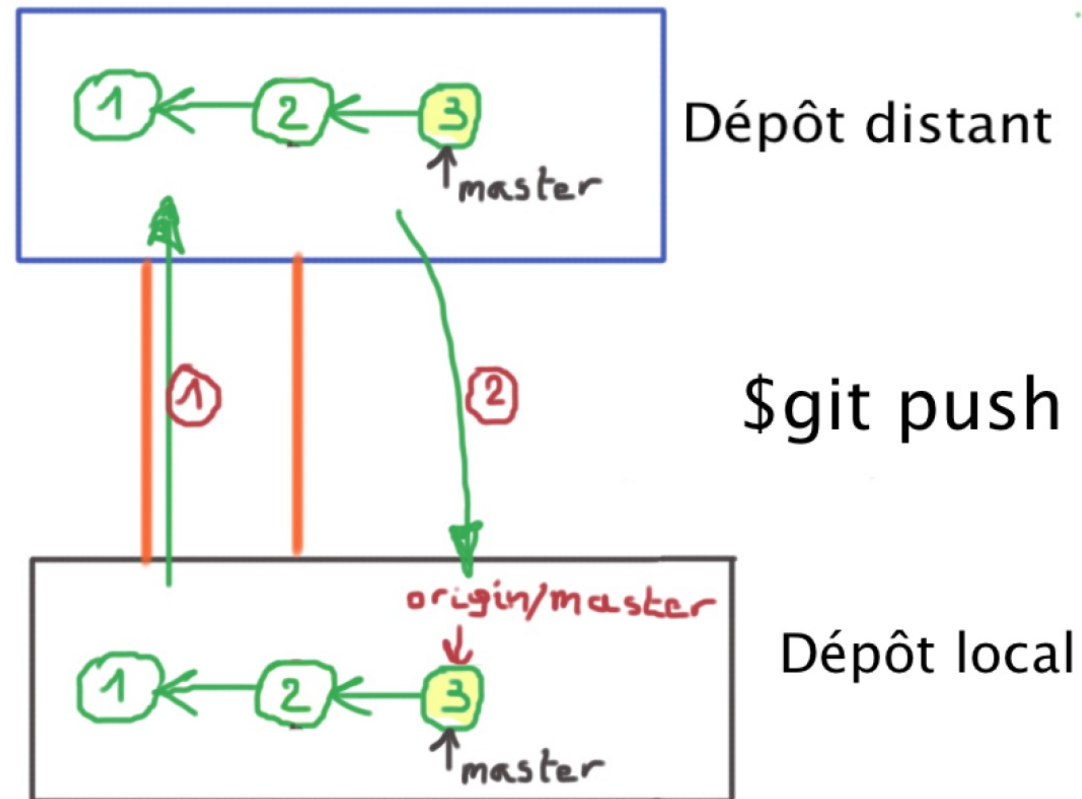


Gestion des dépôts distants – Récupération d'un dépôt





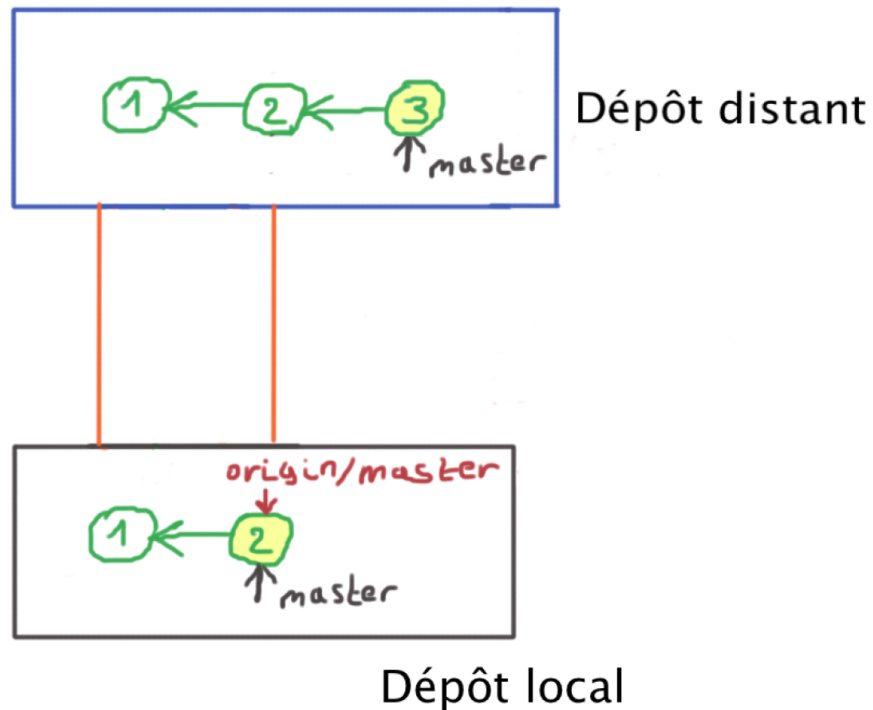




```
$ git fetch <alias> <branche>
```

Récupère la branche du dépôt <alias>. Télécharge aussi tous les commits et fichiers nécessaires depuis le dépôt. =>
Synchronise la branche de suivi avec la branche distante ou mise à jour de la branche de suivi

Gestion des dépôts distants – Synchronisation avec le dépôt distant



.git/config

```
[remote "origin"]  
url=<url>  
fetch=+refs/heads/*:refs/remotes/origin/*  
[branch "master"]  
remote=origin  
merge=...
```

Gestion des dépôts distants – Synchronisation avec le dépôt distant



Dépôt distant

Référence spécifique = Refspec
Format -> <source>:<destination>

\$git fetch



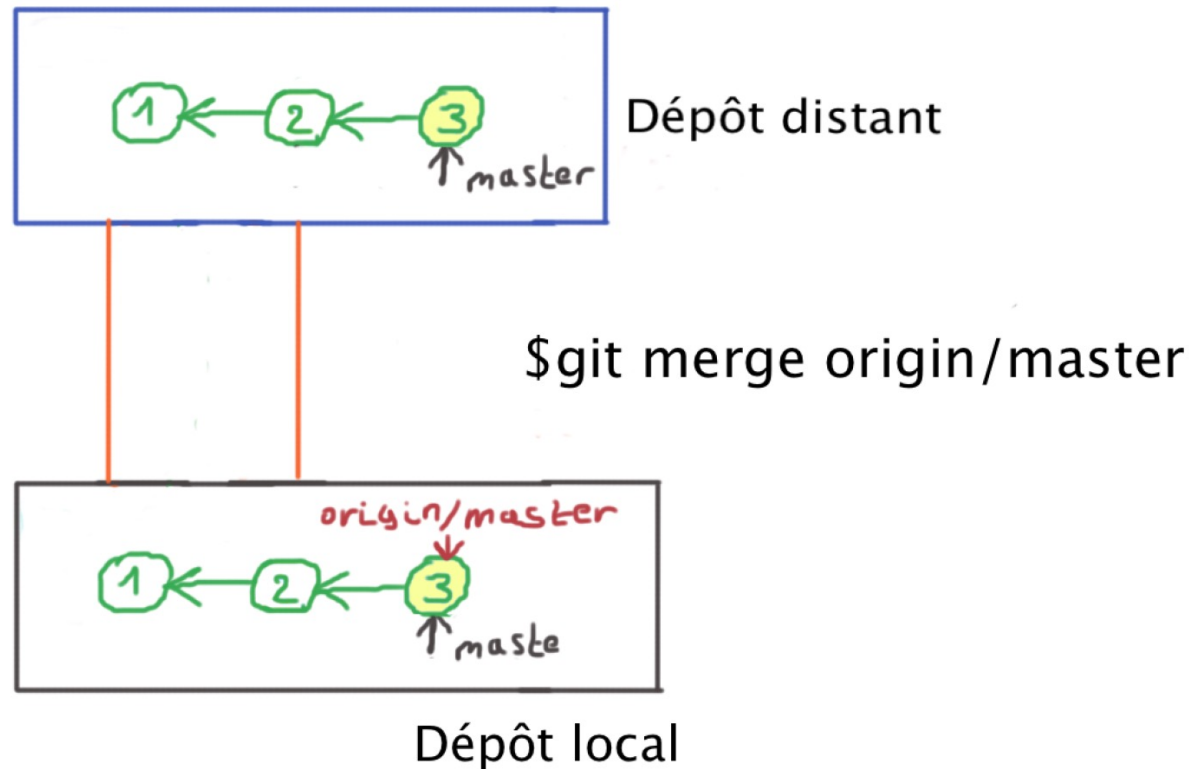
Dépôt local

.git/config

```
[remote "origin"]
url=<url>
fetch=+refs/heads/*:refs/remotes/origin/*
[branch "master"]
remote=origin
merge=...
```

git branch -a : toutes les branches

git branch -r : toutes les branches distantes



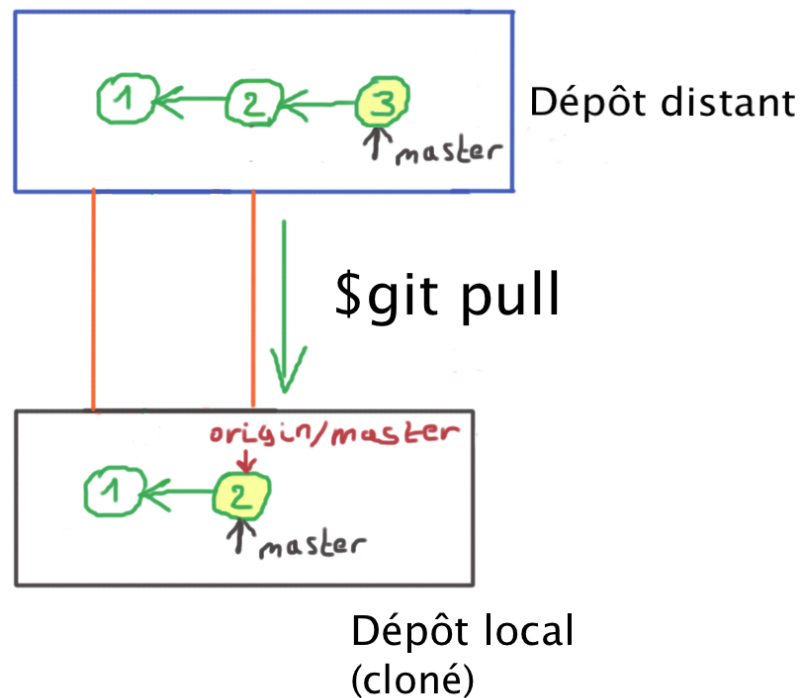
- Création d'un compte
- Accès dépôt distant
 - HTTPS (nom d'utilisateur + mot de passe)
- SSH
 - Génération d'une paire de clé (Publique et privée)
 - Keygen (Linux ou Mac)
 - PuTTY (Windows)
 - Copier la clé publique dans GitHub (contenu du fichier `~/.ssh/id_rsa.pub`)

- Avant de travailler sur votre dépôt local, effectuer une synchronisation avec le dépôt distant.
- Faire un fetch avant de pousser sur le dépôt distant.
- Faire un fetch souvent afin d'être synchro avec le dépôt.

TRI

git pull = (1)git fetch + (2)git merge

Faire un pull sur un dépôt cloné



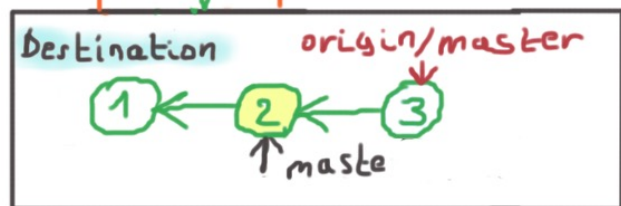
.git/config

```
[remote "origin"]
url= <url>
fetch=+refs/heads/*:refs/remotes/origin/*
[branch "master"]
remote=origin
merge= refs/heads/master
```

Gestion des dépôts distants – Synchronisation avec le dépôt distant - raccourci



Dépôt distant



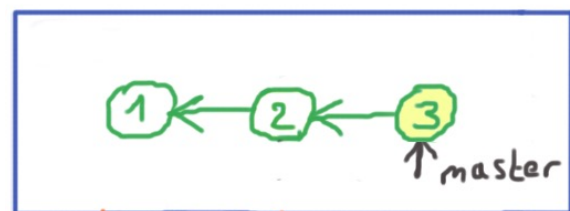
Dépôt local
(cloné)

(1) git fetch

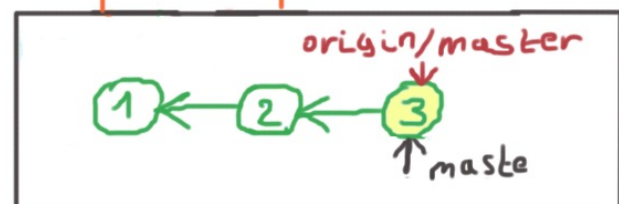
.git/config

```
[remote "origin"]
url=<url>
fetch=+refs/heads/*:refs/remotes/origin/*
[branch "master"]
remote=origin
merge=refs/heads/master
```

Gestion des dépôts distants – Synchronisation avec le dépôt distant - raccourci



Dépôt distant



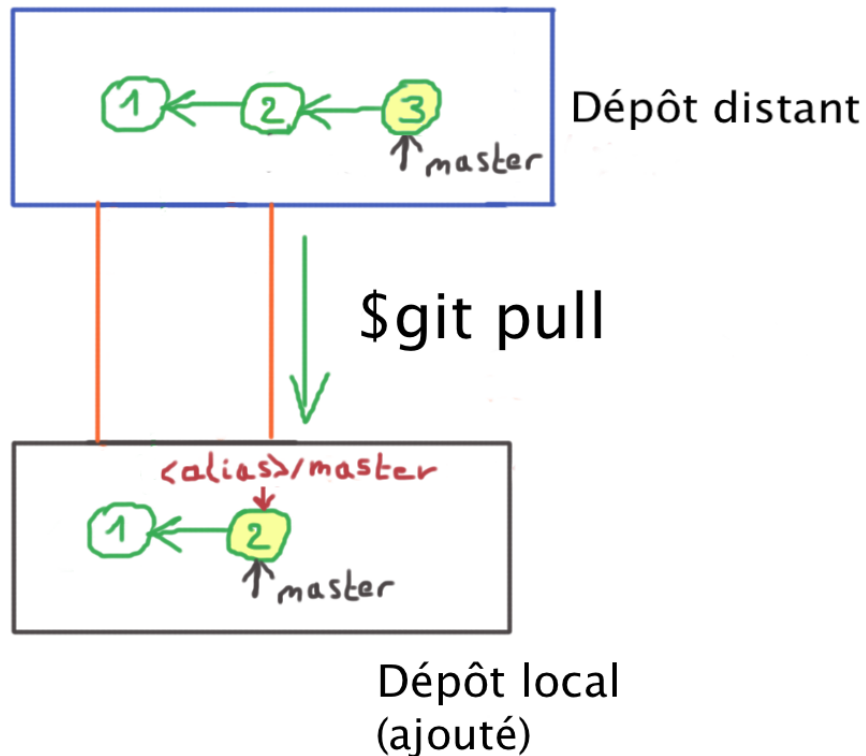
Dépôt local
(cloné)

(2) git merge

.git/config

```
[remote "origin"]
url=<url>
fetch=+refs/heads/*:refs/remotes/origin/*
[branch "master"]
remote=origin
merge=refs/heads/master
```

Faire un pull sur un dépôt ajouté



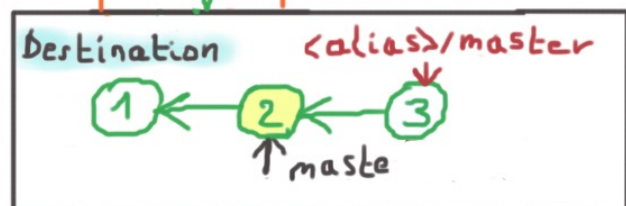
```
[remote "<alias>"]  
url=<url>  
fetch=+refs/heads/*:refs/remotes/<alias>/*
```

Gestion des dépôts distants – Synchronisation avec le dépôt distant - raccourci



Dépôt distant

(1) git fetch

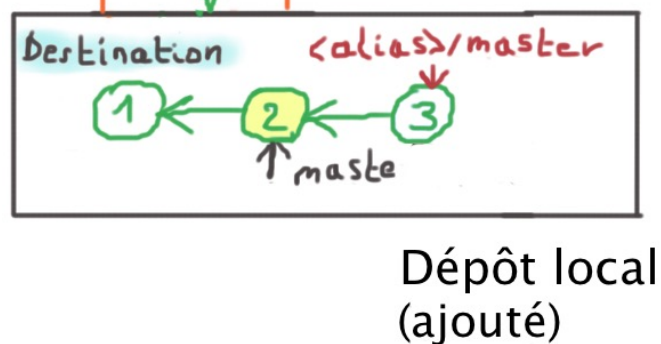
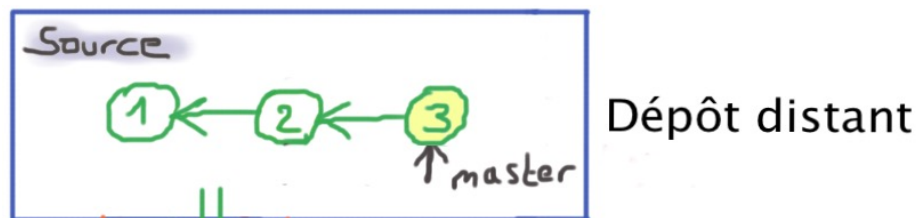


Dépôt local
(ajouté)

.git/config

```
[remote "<alias>"]  
url=<url>  
fetch=+refs/heads/*:refs/remotes/<alias>/*
```

Gestion des dépôts distants – Synchronisation avec le dépôt distant - raccourci



(2) git merge *⇒ ne Fonctionne pas !*

.git/config

```
[remote "<alias>"]
url=<url>
fetch=+refs/heads/*:refs/remotes/<alias>/*
```

}
?

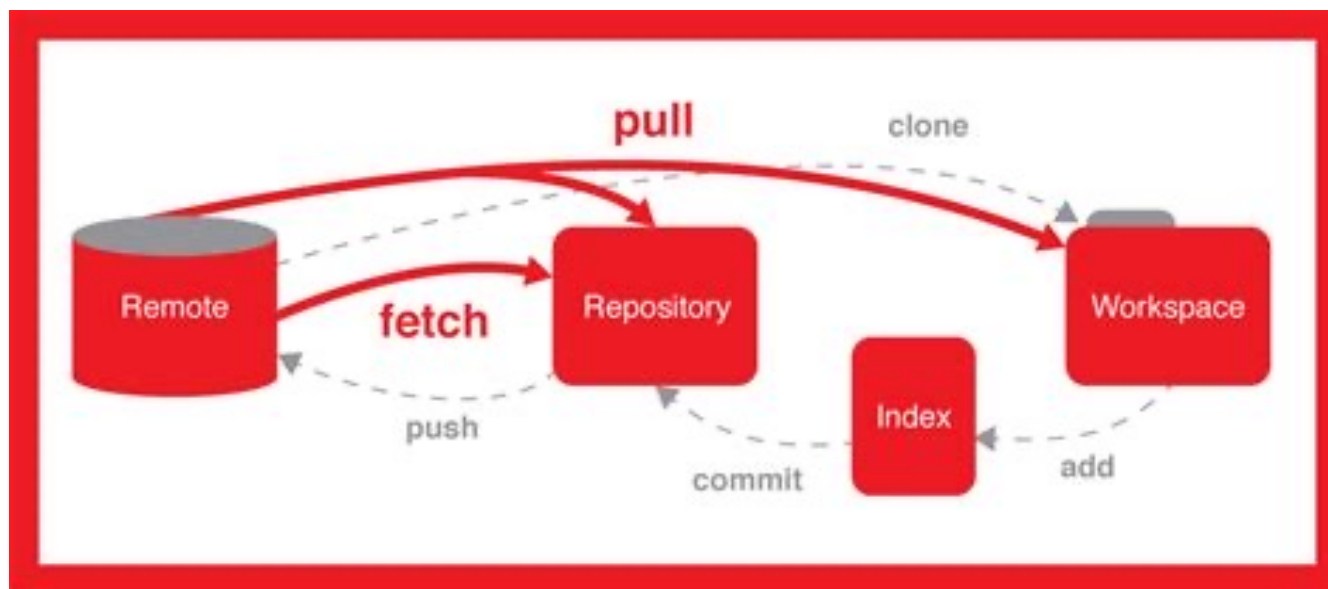
Ajouter un dépôt

```
$ git remote add <alias> <url>
```

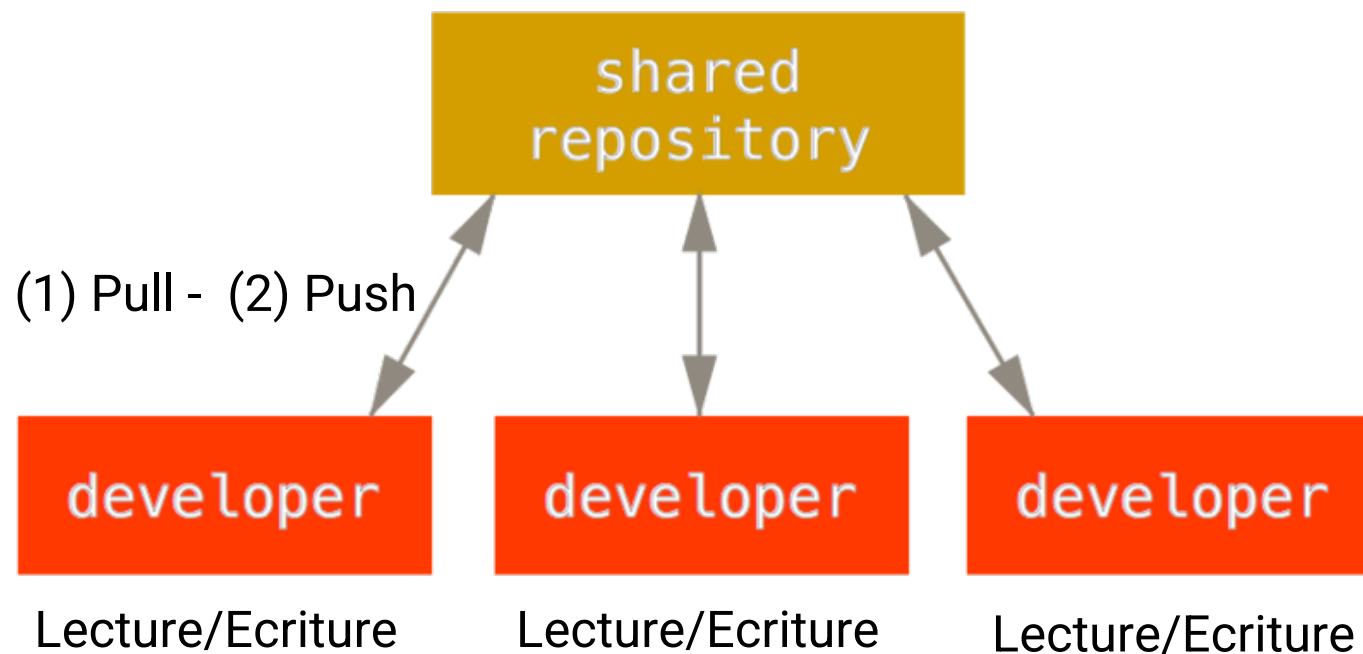
Deux solutions :

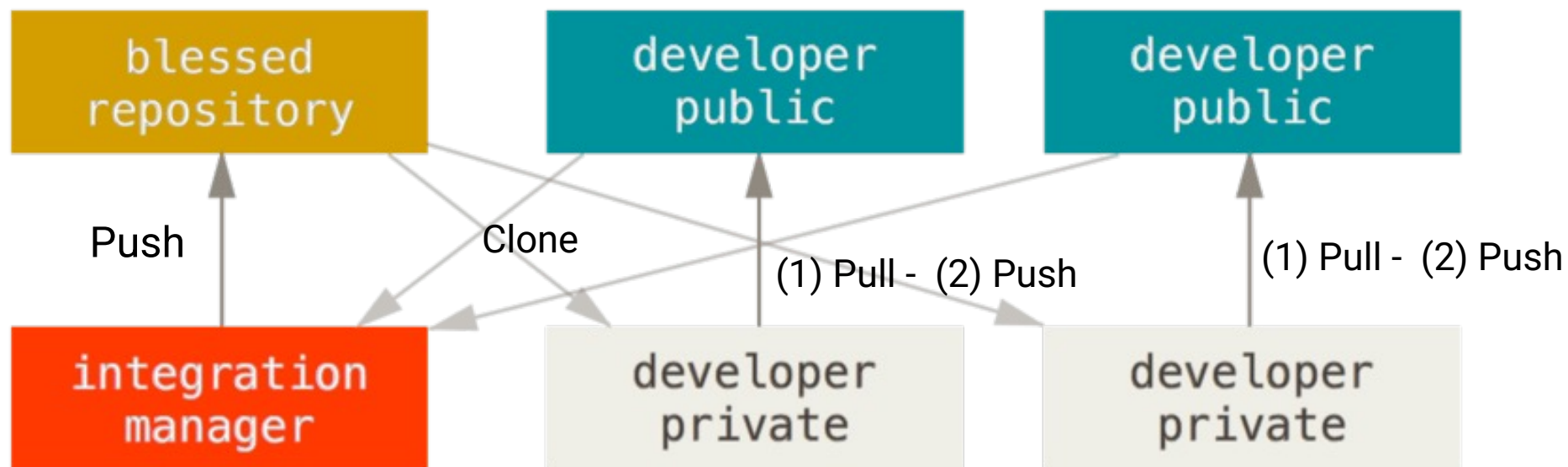
- `git push -u <alias> master`
- `git branch --set-upstream master <alias>/master`

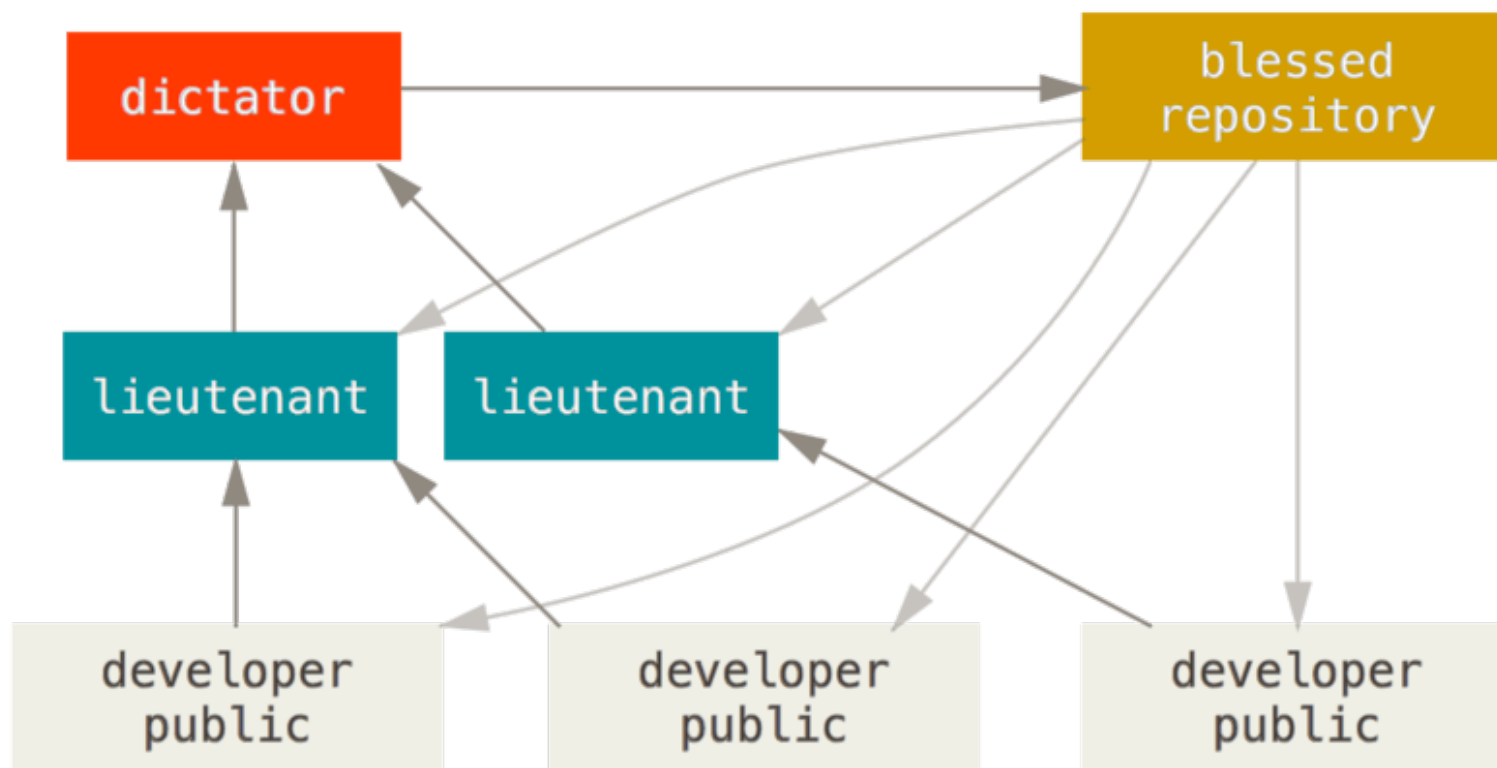
- Supprime la connexion au dépôt distant intitulé <alias>.
\$ git remote rm <alias>
- Mettre à jour les dépôts distants.
\$ git remote update
- Renomme une connexion distante <ancien-nom> en
<nouveau-nom>.
\$ git remote rename <ancien-nom> <nouveau-nom>



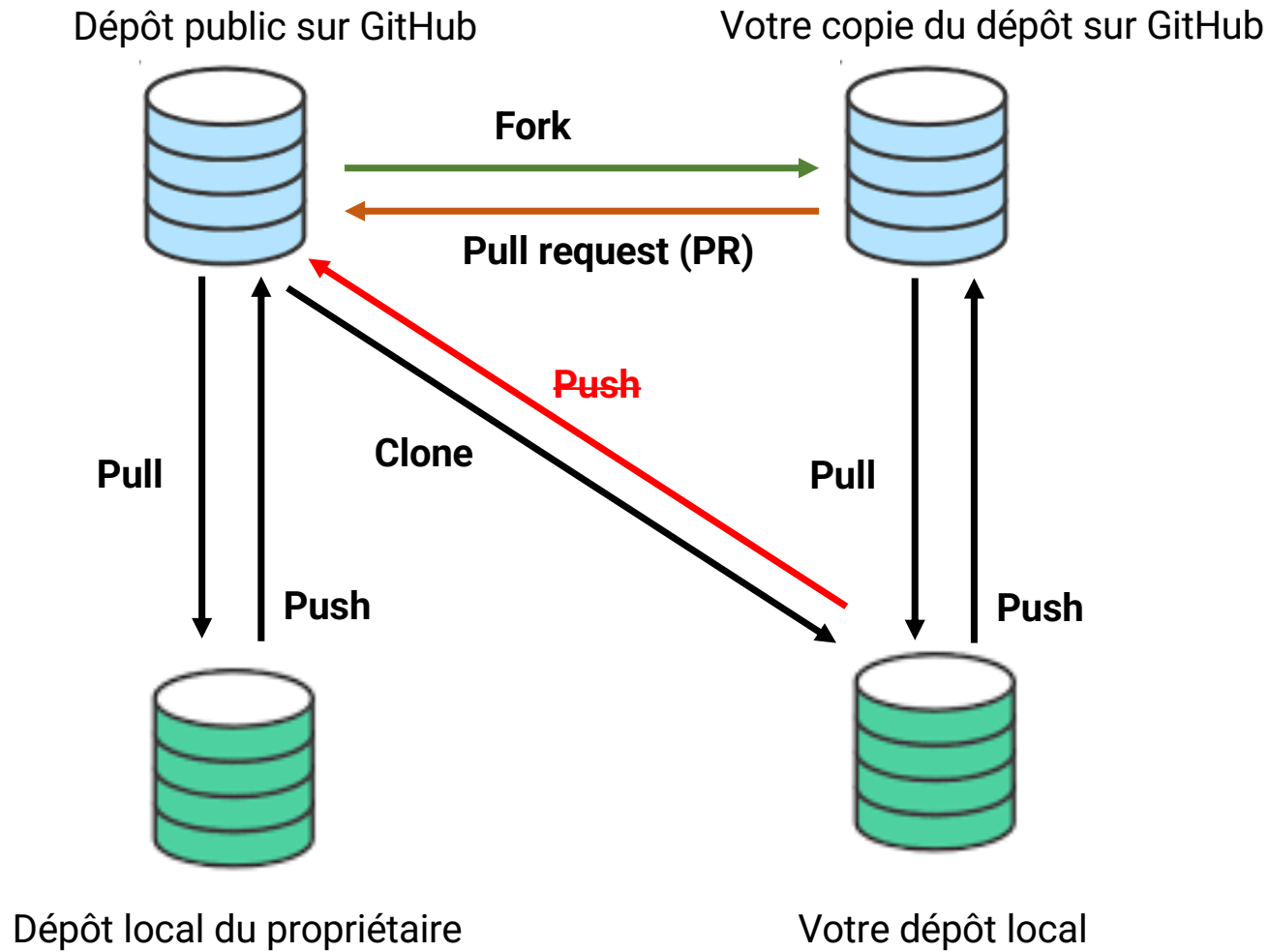
Organisation du travail en équipe

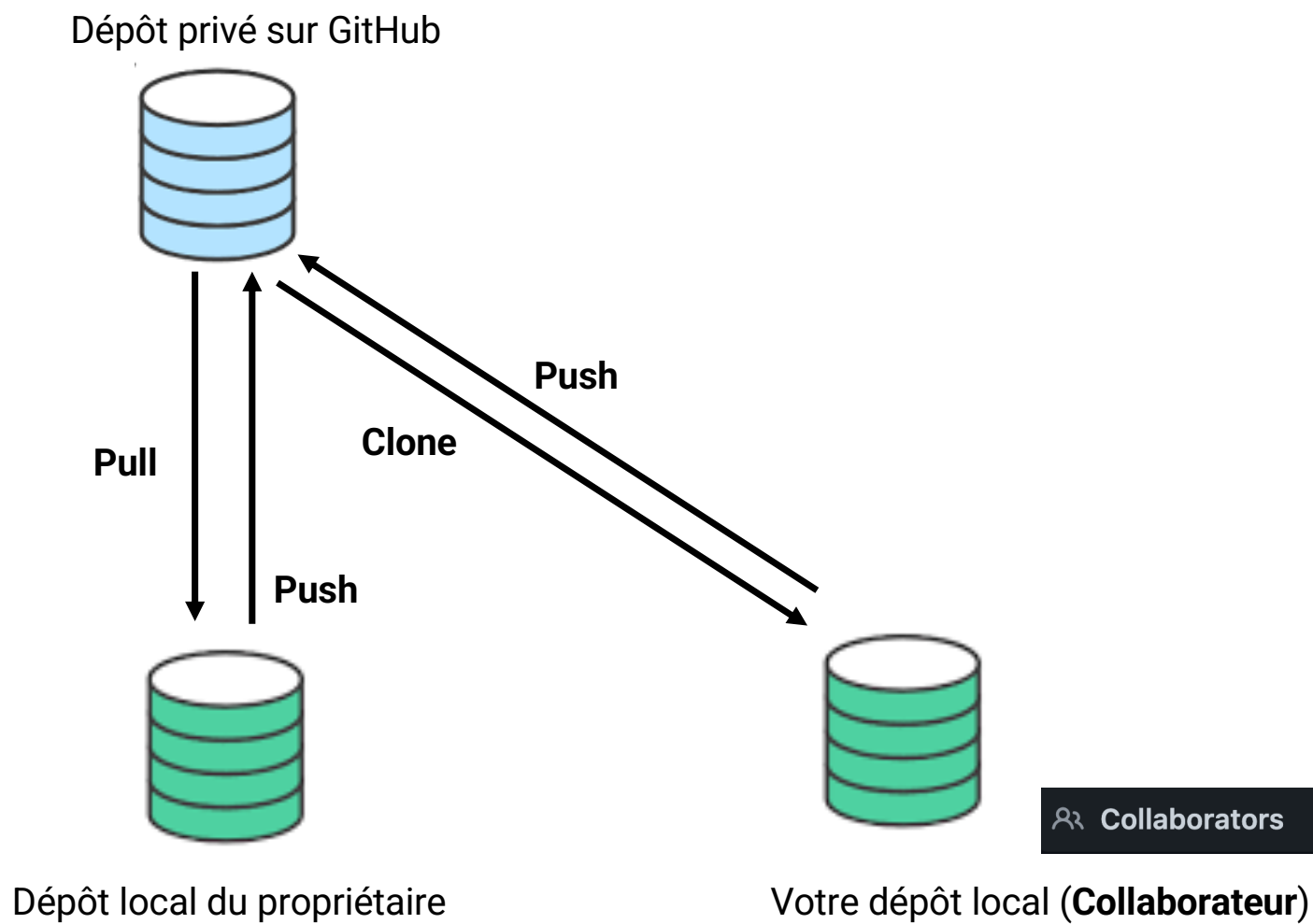






Mode utilisé lors du développement du noyau Linux (+100 collaborateurs)



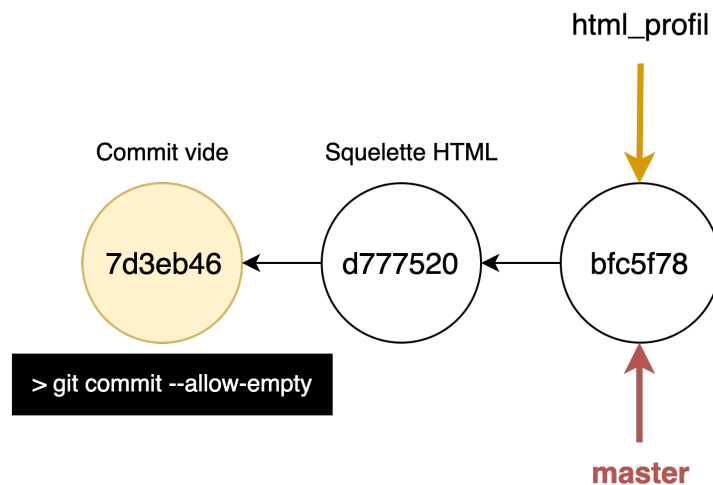


Mettre en ligne son CV à l'aide de GitHub Pages

The screenshot shows the GitHub repository settings page for GitHub Pages. The left sidebar contains a navigation menu with the following items: General, Access, Collaborators, Moderation options, Code and automation, Branches, Tags, Actions, Webhooks, Environments, Pages (highlighted), Security, Code security and analysis, and Deploy keys. The main content area is titled 'GitHub Pages' and includes the following sections:

- General:** A message stating 'Your site is live at <https://trigianni.github.io/siteweb/>' and 'Last deployed by [github-pages](#) 1 minute ago'. A 'Visit site' button is present.
- Build and deployment:**
 - Source:** A dropdown menu set to 'Deploy from a branch'.
 - Branch:** A message stating 'Your GitHub Pages site is currently being built from the master branch. [Learn more.](#)' Below this, a dropdown menu is set to 'master' (highlighted with a yellow circle), followed by a folder icon and a dropdown set to '/ (root)'. A 'Save' button is to the right.
- Footer:** A message stating 'Your site was last deployed to the [github-pages](#) environment by the [pages build and deployment](#) workflow.'

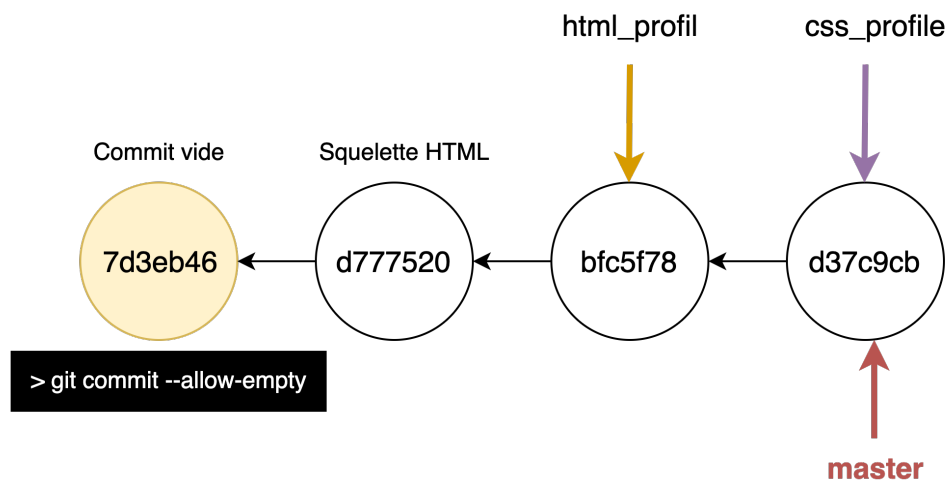
Dépôt local



Master : branche de production

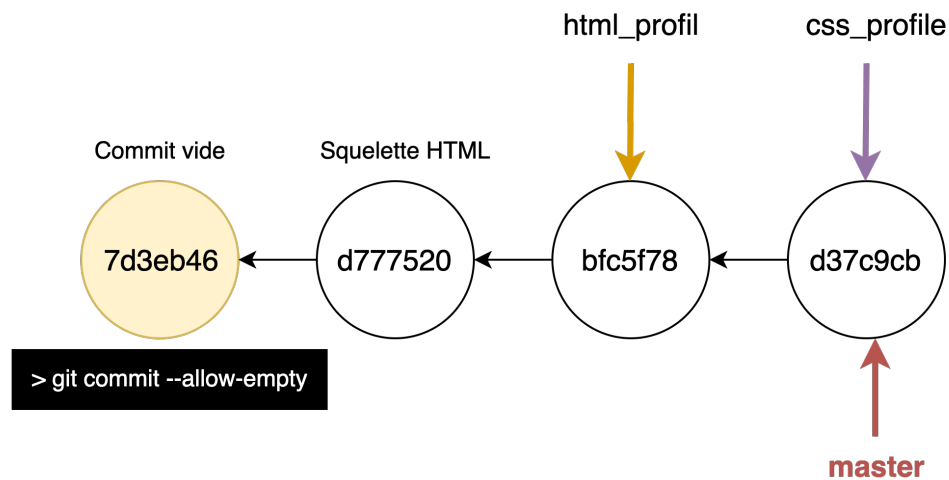
html_profil : branche de travail

Dépôt local



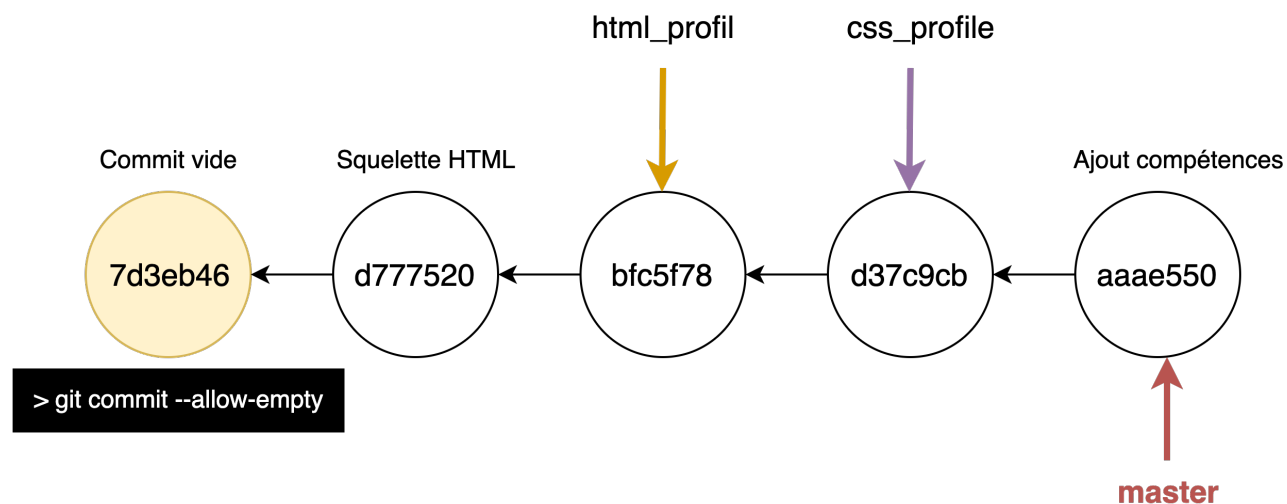
Dépôt distant - GitHub (origin)

Pousser les commits sur le dépôt distant

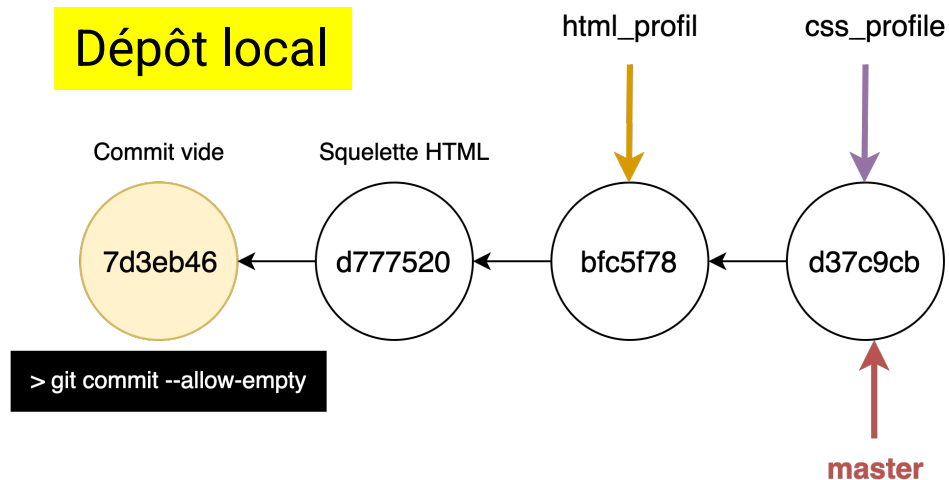


Dépôt distant - GitHub (origin)

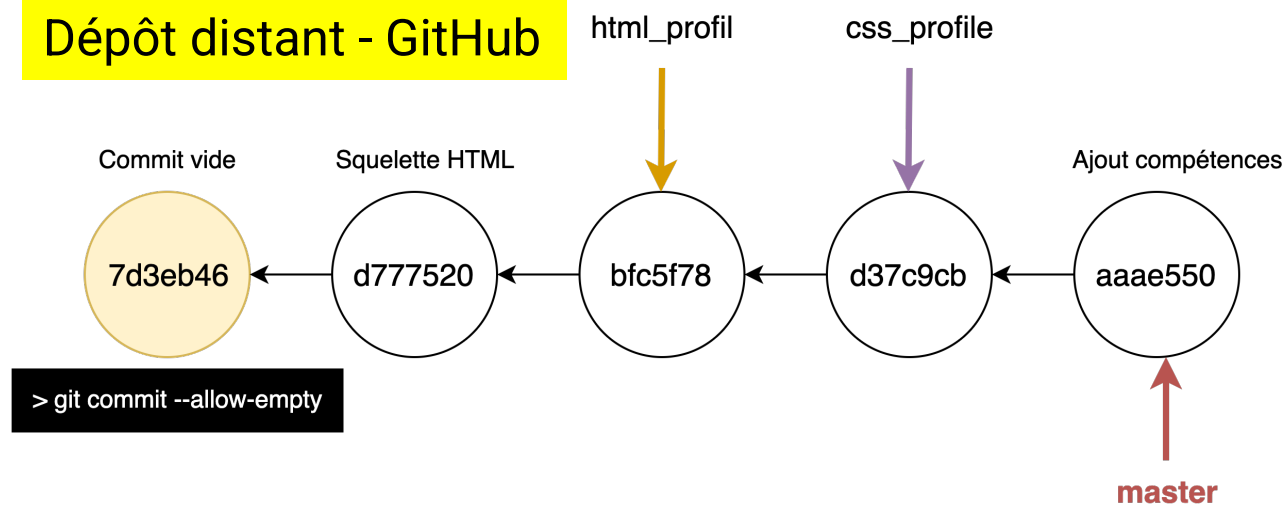
Création d'un commit avec GitHub



Dépôt local



Dépôt distant - GitHub



Dépôt local

Récupération du commit distant

